LEARNING TO IGNORE WHAT MATTERS: JEPAS IN STOCHASTIC WORLDS

Anonymous authors
Paper under double-blind review

ABSTRACT

Joint-embedding predictive architectures (JEPAs) learn a world model by predicting the representation of the next observation instead of the observation itself. This is meant to let them ignore details that do not matter. We show that in stochastic environments they also ignore what an agent needs in order to act. A JEPA's loss is lowest for features that are easy to predict, and the features an agent depends on are often the hardest to predict: its own position when hidden wind pushes it, and obstacles that appear at random times. We find two ways in which a JEPA trained end to end loses these features. The encoder removes them, because the prediction loss also trains the encoder through the prediction target. With wind, the encoder settles on a representation that carries no information about the agent; a short calculation predicts its scale, and the trained models match it. The predictor removes them even when the encoder keeps them: trained with squared error, it outputs the average future, and an obstacle that may or may not appear is missing from that average. We test both mechanisms by training agents entirely inside a JEPA and inside a generative world model with the same data and compute, and evaluating the agents in the real environment. Agents trained inside the JEPA almost never get through walls whose openings are revealed late; agents trained inside the generative model almost always do. Other prediction targets move this failure to a different environment instead of removing it, and a JEPA encoder matches the generative model only when the gradient through its prediction target is blocked and its predictor is replaced by the generative model's sampler. In Craftax, planning inside the JEPA almost never crafts a wooden pickaxe, while planning inside the generative model does so in most episodes. In all of these failures the JEPA did not collapse and predicted its own representations with low error, so neither property shows that a JEPA is fit for control.

1 INTRODUCTION

Most of what a camera sees is irrelevant to the agent carrying it: the pattern of a carpet, the flicker of sensor noise, the leaves moving on a tree. Joint-embedding predictive architectures (JEPAs) promise a world model that looks past such things (LeCun, 2022; Assran et al., 2023; Bardes et al., 2024): an encoder and a predictor are trained together so that the predictor forecasts the encoder's own next output, the model never has to paint a pixel, and the encoder is free to drop whatever it cannot predict. LeWorldModel (Maes et al., 2026) made this a minimal recipe for control: a squared prediction loss, a regularizer that keeps the latents Gaussian so that they cannot collapse, and nothing else.

In a stochastic world, that freedom has a price. One training loss decides both *what* to encode and *how* to predict it, and it favors whatever is predictable: in deep linear models a JEPA learns first the features its context predicts best, where reconstruction learns the highest-variance ones (Littwin et al., 2024). Yet what an agent controls is often the least predictable part of the scene, pushed by forces the camera does not see and met by obstacles that arrive at moments nobody can foresee. Selecting for predictability then selects against the state that control needs, at every stage of a world model (Section 2). The *encoder* keeps what the predictor can already predict, and its regularizer fills the remaining directions with noise when the world offers noise. The *predictor*, trained with a squared loss, outputs the average future, and the average of futures in which an obstacle arrives at different moments shows no obstacle at all.

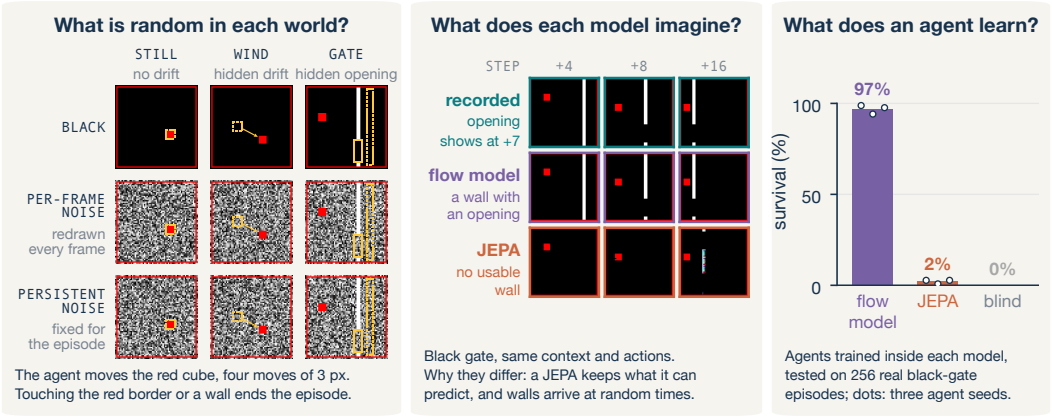


Figure 1: **In the black gate only the flow model imagines the wall and its opening, and only agents trained inside it survive.** Left: the nine worlds; dashed, where the last 12 recorded actions alone would put the cube (wind) and the wall three steps before its opening shows (gate). Middle: one held-out context continued as recorded, as sampled by the flow model and as predicted by the grid JEPA. Right: real survival of agents trained inside each model (Table 2).

We test this against a generative world model built from the ingredients a JEPA avoids, a tokenizer trained to reconstruct frames and a flow-matching predictor that *samples* the next latent (Lipman et al., 2023; Hafner et al., 2025), trained on the same data under the same compute budget. Our worlds are small visual control tasks in which each kind of randomness can be switched on by itself, plus the open-world game Craftax (Matthews et al., 2024). We ask what each model's latent retains about the state and what an agent trained entirely inside its imagination can do in the real world, and we try to repair the JEPA. Our findings follow the chain from encoder to agent:

1. **The JEPA encoder keeps what is predictable, not what matters.** With background noise redrawn every frame and a hidden wind, LeWorldModel's latent settles at a fixed point of its training loss where it is Gaussian noise independent of the state; with no fitted parameter, this point gives the latent variance and prediction error of 23 failing fits to within 0.7%. In the simplest wall world the gradient through the prediction target shrinks the approaching wall to under 0.02% of the latent's variance; blocking that gradient, and nothing else, keeps 32% to 46%. Each other target fixes one failure and opens another.

2. **A squared-loss predictor erases what arrives at random.** With the target gradient blocked, the JEPA's encoder keeps the approaching wall, yet its predictor draws an entering wall on at most 0.5% of the steps where one is due; a random attribute of an object already in view, such as the side of the wall's opening, survives the average.

3. **Only a sampling predictor on an encoder that keeps the state teaches.** Agents trained inside LeWorldModel survive the wall world in 2% of episodes, inside the flow model in 97%; the JEPA encoder with a blocked target gradient reaches 98% with the flow model's sampler and 1% with its own predictor. In Craftax, planning inside the flow model crafts a wooden pickaxe in 68% of episodes, inside the JEPA in 1%: the JEPA's encoder sees the pickaxe, but its predictor does not imagine making it.

Non-collapse and low prediction error coexisted with every one of these failures, so neither is evidence that a world model supports control. Every repair that made the JEPA work turned it into a flow model on a JEPA encoder, which tied the reconstruction model without beating it.

2 Two ways to ignore what matters

The training loss. An encoder f_θ maps an observation x_t to a latent $z_t \in \mathbb{R}^D$, and a predictor g_θ maps recent latents and actions to a forecast of z_{t+1}. LeWorldModel (Maes et al., 2026) trains both

jointly by minimizing

$$\mathcal{L} = \frac{1}{D}\,\widehat{\mathbb{E}}\,\|g_\theta(z_{\le t}, a_{\le t}) - z_{t+1}\|^2 + \lambda\,\mathrm{SIGReg}(\{z_t\}), \tag{1}$$

where SIGReg is a statistical test, computed on random one-dimensional projections of a batch, of whether the latents follow a standard Gaussian; it rules out mapping every frame to the same point. There is no stop-gradient, teacher network, decoder or reward. Two choices hide in this line: the encoder chooses the target z_{t+1} that the predictor must hit, and the loss is a squared error. The first decides what the latent keeps, the second what a rollout keeps.

2.1 Mechanism 1: the encoder keeps what its predictor can predict

For a fixed encoder and an unrestricted predictor, the lowest achievable squared error is the conditional variance of the next latent Z' given the context C and actions A, $\inf_g \mathbb{E}\|g(C, A) - Z'\|^2 = \mathbb{E}\,\mathrm{trCov}(Z' \mid C, A)$: whatever the past cannot predict is charged in full. Joint training lets the encoder lower that charge by choosing what Z' contains, and nothing in Equation 1 asks it to keep the controlled state; a Gaussianized episode-constant nuisance alone reaches zero loss (Proposition 2, extending Sobal et al., 2022). Two parts of the recipe turn this freedom into deletion.

The gradient through the target deletes what the predictor has not learned. In deep linear JEPAs with a stop-gradient target, features are learned in order of how well the context predicts them, and given time the rest is learned too (Littwin et al., 2024). In LeWorldModel the gradient also flows through the target. Hold the latent's total variance fixed, as the regularizer roughly does, and let s_k be the share of that variance carried by content k and u_k the fraction of it the predictor does not yet predict. The target-side gradient then moves the shares by replicator dynamics (Appendix A.2),

$$\dot{s}_k = a\, s_k\,(\bar{u} - u_k), \qquad \bar{u} = \textstyle\sum_j s_j u_j, \tag{2}$$

so a content that the predictor predicts worse than average loses share, and because the predictor learns a content at a rate proportional to its share, a content that starts small shrinks before it is learned. The regularizer cannot stop this: shrinking k of D directions changes its expected value only at second order, by $O(k/D^2)$ (Lemma 1). The order in which features are learned becomes the order in which they are deleted. A stop-gradient target removes this force.

The regularizer fills every direction, with noise if the world offers it. When every frame carries fresh noise, the encoder can output white noise, $z_t = \sqrt{v}\,\xi_t$ with $\xi_t \sim \mathcal{N}(0, I_D)$ independent of states, actions and the past. The prediction loss is then v, and the expected regularizer is a known function $S_B(v)$ of the batch size B, so the training loss $v + \lambda S_B(v)$ is minimized at a scale $v^\star(\lambda)$, $v^\star(0.09) = 0.734$; with a stop-gradient target the prediction term exerts no force on the scale, and the minimum moves to $\arg\min_v S_B(v) = 0.984$ (Proposition 3). This fixed point carries no information about the state, and the loss is flat to second order along every direction that would add a state feature, so whether training leaves it depends on how predictable those features are: without wind the cube's next position follows from the actions, while hidden wind makes fine position features tens to hundreds of times less predictable (Appendix C). Noise fixed for the episode offers no such point, because it is predictable after the first frame.

2.2 Mechanism 2: the squared loss keeps only the mean

Even when the state is kept, a squared loss learns the conditional mean $m(c, a) = \mathbb{E}[Z' \mid c, a]$, and a rollout fed its own predictions imagines the average future. Whether the average still shows what matters depends on the kind of randomness. When the randomness is an attribute of an object already in view, such as the side of a wall's opening, every outcome contains the object, and so does their average. When the randomness is *when* an object arrives, the outcomes share nothing.

Proposition 1 (Arrival erasure)**.** *Let an object enter the scene at a random step J after the history h, with $P(J = j \mid h) \le q$ for every j, and let different values of J put the object on disjoint pixels at every future step. Then at every step k the conditional-mean frame $\mathbb{E}[x_{t+k} \mid h]$ shows the object at no more than q times its true intensity at any pixel.*

The mean frame is $\sum_j P(J = j \mid h)\, x_{t+k}(j)$, and at the object's pixels in $x_{t+k}(j)$ only the term j contains it. Once the arrival step passes with no object drawn, the imagined history is again empty,

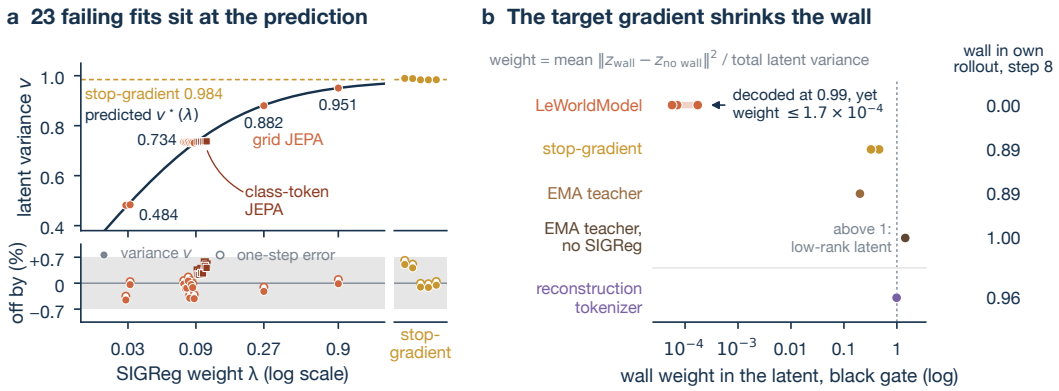


Figure 2: **Both halves of mechanism 1, measured.** **(a)** Latent variance of the 23 fits whose cube read-out is at chance (circles: grid JEPA; squares: class-token JEPA; right: stop-gradient fits) against the white-noise scale $v^\star(\lambda)$ of Proposition 3 (line; no fitted parameter). **(b)** Black gate: the wall's weight in the latent (Section 4) for each prediction target and the reconstruction tokenizer, and the decoded wall at the model's own eighth predicted step.

so a mean rollout stays in the empty branch. In our wall world, seen from eight wall-free frames, the first wall enters after each of 1 to 15 steps with probability 0.063, so $q = 0.063$ (Appendix E). For a latent predictor the proposition holds only through the decoder, which need not map an average latent to an average frame, so Section 4 measures it on the trained encoders.

A predictor that samples, as a flow-matching predictor does, repairs mechanism 2: it draws one arrival time per rollout instead of their average. Neither this nor a model of expected values (Wan et al., 2019; Grimm et al., 2021) touches mechanism 1: if the encoder has dropped the state, a perfect distribution over the remaining features is a perfect distribution over nothing useful.

3 Worlds, Models and Two Tests

Worlds with one knob per kind of randomness. The agent steers a red 6×6-pixel block, the cube, in a 63×63 color image; each of four actions moves it three pixels, and touching the border ends the episode with a penalty of -1, the only reward (episodes last at most 128 steps). The *background* is black, or gray noise redrawn every frame (*per-frame noise*, like sensor static) or drawn once per episode (*persistent noise*, like a patterned floor). *Wind* adds a hidden drift to every move, a slow random walk the agent never observes, so even a perfect memory leaves the cube's next position uncertain; without it the world is *still*. In the *gate* worlds, walls sweep toward the cube one after another, each with an opening at the top or the bottom that shows only when the wall is close; the agent must wait for the opening, then commit to it, and every strategy that ignores the screen dies. The three backgrounds crossed with still, wind and gate give nine worlds. Craftax-Classic (Matthews et al., 2024) is a two-dimensional open-world game in which progress takes multi-step plans. All models of a world learn from the same recordings: uniformly random play in our worlds, a pretrained agent's play in Craftax (Appendices M and N).

Three models, one budget. The *grid JEPA* uses LeWorldModel's loss (Equation 1) at its published weight $\lambda = 0.09$ on a 4×4 grid of 32-dimensional vectors per frame (512 numbers), with a predictor that attends over space and time and forecasts the next grid from the last eight latents and actions. The *class-token JEPA* ports LeWorldModel's published design, one 192-dimensional vector per frame. The *flow model* is a tokenizer trained to reconstruct frames, with the grid JEPA's encoder architecture and latent shape, and a causal flow-matching predictor that samples the next latent (Lipman et al., 2023; Hafner et al., 2025), trained jointly with reward and termination heads; while imagining, it feeds each generated frame back into its context mixed with noise, at the *signal level* 0.9 used in training. Every model trains from scratch for about ten minutes on one A100 (Appendix N); the JEPA's failures persist at three times its budget (Appendix R). Decoders, read-outs and heads that a JEPA needs are fitted afterwards on the frozen model, outside its budget. To repair

Table 1: **Test 1: what each model's latent keeps.** Pixel error of the cube's position read from real encodings (ENC) and from the model's own prediction eight steps ahead (PRED); above 15 is chance; mean of three models (Table 11). With wind an ideal forecaster errs by 7.90 (mean) or 11.76 (sample) at step eight. Gate rows also score the wall (Table 12). Craftax: Fréchet distance $\times 10^3$ (last frame repeated: 13.3).

	class-token JEPA		grid JEPA		flow	
world	ENC	PRED	ENC	PRED	ENC	PRED
black, still	0.09	18.4	0.10	3.35	0.12	0.12
black, wind	0.00	8.79	0.13	11.5	0.13	11.1
per-frame noise, still	20.8	19.9	0.12	4.83	0.26	0.33
per-frame noise, wind	18.0	19.6	17.6	20.4	0.25	12.5
persistent noise, still	18.7	20.0	0.76	1.83	0.59	1.79
persistent noise, wind	16.0	19.5	0.67	11.0	0.25	13.4
black gate	6.73	17.2	0.10	3.24	0.17	0.38
gate + per-frame noise	20.2	20.3	4.74	14.5	0.26	0.43
gate + persistent noise	18.5	21.0	1.09	3.37	0.27	1.03
Craftax-Classic	18.6	24.0	8.09	11.4	0.07	1.27

the grid JEPA we change only its prediction target: no gradient through the target (*stop-gradient*), a moving-average copy of the encoder as in I-JEPA and V-JEPA (*EMA*), and EMA without SIGReg (Appendix D).

Test 1: what the latent keeps. A latent can hold the state and still lose it in prediction, so we read the cube's position from the encoding of a real frame (ENC) and from the model's own prediction eight steps ahead of real context (PRED). One read-out is fitted on the latent and one on frames from a decoder fitted to the frozen model, and we report the better held-out error in pixels, so that a weak decoder cannot fail a model (above 15 is chance). In the gate worlds we also measure the approaching wall's decoded intensity (1 when fully present) and the AUROC for the side of its opening; in Craftax, the Fréchet distance between decoded predictions and real frames.

Test 2: what an agent learns inside the model. One pixel-based agent, a convolutional network with a recurrent memory (1.7M parameters), is trained by PPO (Schulman et al., 2017) entirely inside each model: from real recorded moments, the model supplies the frames through its decoder and its own heads supply rewards and terminations. The agent is then tested in the real world on the same 256 fixed episodes and scored by the share it survives, next to two blind strategies (moving at random, alternating left and right). As a control, the same agent is trained on *real* experience seen through each model's encoder and decoder, the model's *interface*; if it succeeds there, a later failure belongs to the imagination.

Reading the results. Table colors are verdicts with thresholds fixed in advance (Appendices M and P): **green** keeps the state or learns the task, **red** is at chance or no better than blind play, **amber** lies in between. No seed or checkpoint is selected. One rerun of an agent moved survival from 5% to 94%, since training on a GPU is not bit-reproducible (Appendix F); we report every run and call differences inside that spread ties.

4 What the models keep

Our causal claims rest on rows that differ in one property of the world, with the model held fixed.

Wind reverses the effect of the background. In still worlds the grid JEPA keeps the cube under either background, and per-frame noise is the *easier* one (0.12 pixels, against 0.76 with persistent noise), as the JEPA promises. Add hidden wind and the roles reverse. With persistent noise the cube is still located to 0.67 pixels; with per-frame noise the error is at chance (17.6) in all three trained models, with architecture, data, compute and read-out identical. The failure is already in the encoding of real frames: every read-out we tried is at chance, and the decoded encoding of a real

frame shows no cube, while the flow model's shows it (Figure 8, Appendix L). Two and three times the compute change nothing; at $\lambda = 0.03$ one of three per-frame fits keeps the cube, and at 0.9 both wind worlds are at chance (Appendix R).

The blind latent is the regularizer's white-noise fixed point. Every failing fit sits at the point of Section 2.1: its latent is Gaussian (kurtosis 2.96 to 3.03), carries nothing about the cube ($|R^2| \leq 0.015$), and has one-step error and variance both at $v^\star(\lambda)$ to within 0.7%, in all 23 failing fits (the grid JEPA at four regularizer weights and at two and three times the budget, the class-token JEPA in both per-frame-noise cube worlds, and the stop-gradient refits at $v^\star = 0.984$; Appendix C). The information is erased, not hidden. The point is not what the loss prefers (the $\lambda = 0.03$ fit that keeps the cube scores four times better under the failing fits' loss) but where optimization stops: still and windy fits alike reach it within a few hundred updates; the still fits leave it after $1{,}000$ to $1{,}500$ updates, the windy fits at $\lambda \geq 0.09$ never do in up to $25{,}500$. The class-token JEPA, LeWorldModel's published single-vector design, loses the cube in all six noisy worlds, and on a black, still background its rollout loses the cube by the eighth step with an error of only 0.04% of the latent variance: the training loss permits exact information at an arbitrarily small scale (Proposition 4; Appendix M).

In the black gate, the encoder deletes the wall. The black gate has one random quantity, the side of the opening. A decoder fitted to the grid JEPA's latent redraws the approaching wall at 99% of its intensity, yet the JEPA's own rollout has lost the wall by the eighth step in all three models (Table 12). The encodings of the same future with and without the wall differ, in mean squared distance, by 0.006% to 0.017% of the latent's total variance (the wall's *weight*; Figure 2b): a decoder can amplify so small a direction, but a squared-error predictor gains almost nothing by keeping it. Every predictor we trained on this encoder loses the wall, the flow model's sampler included; every predictor trained on the flow model's tokenizer, an encoder of the same architecture, keeps it (0.90 to 0.98 at step eight; Appendix S). This is the selection of Equation 2: block the gradient through the target, change nothing else, and the wall's weight rises to 32% to 46% and the JEPA's own rollout keeps the wall (0.72 to 0.89 at step eight, opening side at AUROC 1.00; three models).

Averaging keeps a wall in view and erases a wall that arrives. With a solid wall on screen, the decoded average of the encodings of its two possible futures (opening at the top or the bottom, rendered from the reconstructed simulator state) shows the wall at 96% to 100% of its intensity (Proposition 5). With no wall on screen, the decoded average over all 30 possible futures shows no entering wall at any of 20 steps on the tokenizer and the stop-gradient JEPA, three models each, as Proposition 1 predicts (Appendix E). Trained predictors behave as the averages do: open loop, the stop-gradient JEPA draws an entering wall on at most 0.5% of the steps where one is due, the flow model on 64% to 81%.

No prediction target keeps the state in every world. Changing only the grid JEPA's target removes one of the two forces of Section 2.1 and exposes the other (Table 5, Appendix D). Stop-gradient and EMA targets keep the black-gate wall but sit at the white-noise fixed point in both per-frame-noise cube worlds, the still one included, and lose the cube in the persistent gate. The EMA JEPA without SIGReg has no fixed point and is the only JEPA that keeps the windblown cube, in all three models, but in the persistent gate the episode's texture fills its latent (effective rank 252 of 512, against 11 to 18 elsewhere) and its rollout drops the wall at the first step. The reconstruction tokenizer keeps the state in all five worlds.

Craftax repeats the ranking. The flow model's eight-step predictions are about nine times closer to real frames than the grid JEPA's by Fréchet distance; the grid JEPA's barely beat repeating the last frame (11.4 against 13.3). Probes fitted after training to read four inventory facts (holding wood, holding two wood, next to a crafting table, holding a wooden pickaxe) reach held-out AUROC 0.97 to 1.00 on every flow latent and 0.63 to 0.99 on the JEPA's (Appendix H).

5 What an agent learns inside the models

Both interfaces carry what the task needs, with one exception. Trained on real experience seen through the JEPA's own decoder, the agent learns all three gate worlds: it survives 95% to 98%

Table 2: **Test 2: survival of one agent trained inside each model, and on real frames through each model's decoder.** Share (%) of 256 fixed test episodes survived, mean ± standard deviation over three agent seeds (Table 13). Blind: the better of two strategies that ignore the screen; alternating left and right survives the still world. Flow model at signal level 0.9 (0.8: Appendix G).

		trained inside the model		trained on real frames via decoder	
world	blind (best)	grid JEPA	flow	grid JEPA	flow
per-frame noise, still	100	85±18	97±2	100±0	100±0
per-frame noise, wind	0	0±0	58±41	0±0	72±11
persistent noise, wind	0	2±1	41±42	76±18	86±4
black gate	0	2±1	97±2	97±2	81±25
gate + per-frame noise	0	0±0	88±13	66±26	83±21
gate + persistent noise	0	0±0	5±6	94±3	73±37

Table 3: **Black gate: an agent needs an encoder that keeps the wall and a predictor that samples.** Real survival (%) of agents trained inside each frozen encoder paired with each predictor, models 0 / 1 / 2. Flow sampler: the flow model's predictor. Squared loss: the JEPA's own predictor, or on the tokenizer the flow architecture regressing the next latent. Blind strategies: 0%; more runs: Appendix F.

encoder	flow sampler	squared loss
reconstruction tokenizer	98.8 / 97.7 / 74.2	29.7 / 19.1 / 12.1
stop-gradient JEPA	98.4 / 98.8 / 97.7	1.2 / 1.6 / 0.0
LeWorldModel	1.2 / 1.6 / 3.1	2.7 / 0.8 / 2.7

of episodes in the black gate, 91% to 96% with persistent noise and 49% to 95% with per-frame noise, where blind strategies never survive; through the flow decoder, 31% to 97%. The exception is per-frame noise with wind, where the JEPA's encoder has lost the cube (Section 4): through its decoder the agent never survives and dies after 17 steps, as a blind random walk does, while through the flow decoder it survives 65% to 84%. Everywhere else, whatever goes wrong next is a failure of imagination.

Inside the JEPA, agents never learn the gate. With the same agent, budget and test episodes, agents trained inside the JEPA survive 2% of the black gate's episodes, against 97% inside the flow model. They are not random: they stay alive for 42 steps on average, where blind strategies last 24, having learned the one danger their imagination still showed them (the border). With per-frame noise the JEPA agents never survive, while the flow agents survive 72% to 98% of episodes in seven of nine runs at two signal levels (5% and 40% in the other two). Planning with the JEPA's predictor, as LeWorldModel was designed to do, is no better than random in either noisy gate world (Appendix Q).

Repairing the encoder is not enough; the predictor must sample. Table 3 crosses three encoders with two predictors in the black gate, where every wall an agent meets enters during the episode. The stop-gradient JEPA's encoder keeps the wall, yet agents inside that JEPA survive at most 1.6% of episodes, because its squared-loss predictor draws no entering wall; with the flow model's sampler on the same frozen encoder they survive 98%, as often as inside the flow model itself (97%). On LeWorldModel's encoder no predictor helps (1% to 3%): the wall is no longer in the latent. A squared-loss predictor on the reconstruction tokenizer lands in between (4% to 62% over eight runs): it never draws a first wall, as Proposition 1 requires, but draws the later ones, which enter one step after the previous wall leaves the screen. A JEPA encoder thus matches reconstruction only once it has become a flow model on a JEPA encoder.

The failing imaginations are exploitable. An agent trained inside a model also optimizes the model's errors. The squared-loss predictor on the tokenizer draws the opening on a side that depends on the agent's recent actions, and its agents learn to put the imagined opening on the cube's side (69% to 97% of imagined reveals, against 48% to 55% in the real game; Spearman −0.90 with real

survival over eight policies). Inside the JEPA with per-frame noise, agents exploit its heads: the termination head turns optimistic from the fourth imagined step, and agents reach imagined returns up to $+0.48$ where the only reward is a penalty for dying, the head-error term of Proposition 5 (Figure 12; Appendix U).

With persistent noise, the flow model teaches once it trusts its own frames less. In the gate world with persistent noise the flow agents survive only 5% of episodes, although both interfaces carry the task. When the flow model reads back its own generated frames at signal level 0.8 instead of 0.9, the same three models teach agents that survive 99.6%, 91.4% and 97.7% of episodes, and no other world moves beyond its run-to-run spread (Appendix G).

Without walls, the representation decides first. With per-frame noise and wind, where the encoder lost the cube, every JEPA agent dies in every episode, after 10 to 16 steps, sooner than a blind random walk (17); the flow agents survive 58% on average (73% at signal 0.8). With persistent noise and wind, where the encoder kept the cube, the JEPA agents outlive blind play (27 steps against 17 and 18) yet survive only 2% of episodes, the flow agents 41% (83% at 0.8). The EMA JEPA without SIGReg, which keeps the windblown cube, trains agents that survive the per-frame wind world in 73%, 93% and 96% of episodes, a tie with the flow model (64% to 90% at signal 0.8, 13% to 95% at 0.9): that world has no random arrival, and the average of a random walk stays close to a typical path for a few steps. The same JEPA loses the wall in the persistent gate (Section 4); no target we tried works everywhere.

Every model in every world. Appendix K runs all six models in all nine worlds at seed 0, with Test 2's agent and with two planners that search each model's imagination: toward a goal image, as LeWorldModel plans, and with the model's own reward heads. Agents trained inside the flow model survive 89% to 100% of episodes in every world but the gate with persistent noise. Inside the five JEPAs, agents beat the blind strategies by more than five points only in the black wind world (14% to 88%) and, for the EMA teacher without SIGReg, with per-frame noise (73% with wind, 9% in the gate). Even the still world with persistent noise defeats them, although alternating left and right survives every episode there and the grid JEPA's decoder carries the cube (98% on real frames): agents trained inside the five JEPAs survive 0% to 15% of its episodes. The planners agree with wind: the flow model's heads keep the cube alive in 77% to 80% of episodes, the grid JEPA's in 0% to 8%. Planning toward a goal image separates nothing: with wind it fails inside the flow model (0%) and succeeds in four of the fifteen JEPA cells (44% to 77%). In the gate worlds no planner exceeds 2%, the flow model's included.

In Craftax, the JEPA's predictor does not imagine progress. One cross-entropy-method planner searches five-step action sequences in each model's imagination, scored by the inventory probes of Section 4 instead of reward heads (Appendix H). Within 150 steps the flow planner crafts a wooden pickaxe in 68% of episodes and the JEPA planner in 1% (Figure 5). The JEPA's encoder sees the pickaxe (probe AUROC 0.84 to 0.99), but of the change in the latent that crafting really causes, its predictor imagines 0.3% to 1.4%, about as much as when crafting is impossible; the flow model imagines 84% to 93% (decoded frames: Figure 6). On real transitions, a policy trained with the true reward reaches 4.9 achievements per episode on the JEPA latent, against 10.7 on the flow latent and 10.1 on pixels. Reward heads fitted after training make neither model a usable training environment (Appendix H).

6 Related work

Joint-embedding world models. LeWorldModel, the recipe we study, learns an encoder and an action-conditioned predictor jointly with a squared loss and a Gaussian regularizer (Maes et al., 2026; Balestriero & LeCun, 2025); DINO-WM and V-JEPA 2 predict on frozen pretrained features (Zhou et al., 2024; Assran et al., 2025), and PLDM adds regularizers to joint training and plans in its latent (Sobal et al., 2025). We judge a model by what an agent trained inside it can do. Sobal et al. (2022) showed that an encoder can latch onto a predictable nuisance (Proposition 2); with fresh randomness in the controlled state the effect of a persistent nuisance reverses, which bounds that finding rather than contradicting it. Other variants add temporal, reachability or task-state structure (Bai & Xiong, 2026; Li et al., 2026; Yan et al., 2026; Liu et al., 2026), and Terver

et al. (2026) observe that conditional means blur multimodal futures. VJEPA makes the predictor variational (Huang, 2026) and Flow-JEPA pairs LeWorldModel's encoder with a flow-matching predictor (Huo et al., 2026); such predictor repairs help a stop-gradient encoder in our black gate, not LeWorldModel's (Table 3).

What to keep: predictability, variance or value. In deep linear models a JEPA learns features in order of how well its context predicts them, and a masked autoencoder in order of their variance (Littwin et al., 2024); our encoder results are the control consequence of that order. Joint embedding provably beats reconstruction when irrelevant features have large variance (Van Assel et al., 2025), and reconstruction can spend its capacity on high-variance detail (Balestriero & LeCun, 2024); both concern a latent too small for everything, where a JEPA did no better in our tests (Section 7). Model-based agents separate or discard visual distractors (Fu et al., 2021; Wang et al., 2022; Zhang et al., 2021; Zhu et al., 2023), which the agent does not control; our wind puts the randomness into the controlled state, where discarding the unpredictable discards the state. Self-predictive representation learning needs a stop-gradient or moving-average target to avoid collapse (Tang et al., 2023; Ni et al., 2024); that choice also decides which state the encoder keeps.

Generative imagination, and prediction versus control. Generative world models support control learned in imagination (Hafner et al., 2020; Alonso et al., 2024; Hafner et al., 2025), with their own perception, action-conditioning and rollout failures (Hansen & Wang, 2026); FlowWM learns stochastic futures in feature space without actions (Porcher et al., 2026). A model good at prediction need not be good for control (Lambert et al., 2020); value-aware and value-gradient-weighted model learning, expectation models and value equivalence fit what a class of decisions needs (Farahmand et al., 2017; Voelcker et al., 2022; Wan et al., 2019; Grimm et al., 2020; 2021), and TD-JEPA predicts policy-conditioned successor features (Bagatella et al., 2026).

7 Discussion

One rule, three stages. Every failure we measured follows from one rule: a JEPA keeps what it can predict. Its encoder deletes a controlled state that is hard to predict or settles at a white-noise fixed point, and other targets only move this failure to another world; its squared-loss predictor drops what arrives at a random time; and an agent trained inside a model learns whatever the model gets wrong. Reconstruction and a sampling predictor kept the state in every world we built.

When reconstruction is not enough. Under reconstruction, a latent too small for everything keeps what carries the most pixel variance. With two large bouncing blocks and a small mark that shows the opening's side early, a 512-number reconstruction latent keeps the cube and the mark, which LeWorldModel and the grid JEPA lose; at 16 numbers reconstruction loses them too, and LeWorldModel keeps the predictable blocks instead. Weighting reconstruction by the gradient of a critic's value (Voelcker et al., 2022) restores them; reward heads at the usual weight do not (Appendix I). A starved latent needs to know what matters, and predictability does not tell it.

Scope and limitations. Our recipes share an encoder architecture but differ in predictor, updates and supervision, so our causal claims rest on within-recipe contrasts: the four targets and the crossing of encoders with predictors. VICReg-family regularizers and multi-step joint predictors such as Flow-JEPA's are untested. The wind reversal depends on the regularizer's weight and seed, and our agents need the decoder (Appendices R and Q). Most behavioral results rest on one model per world with three agent seeds, agent training is not reproducible run to run, and the black gate's JEPA was chosen during development. The flow model's sampler setting decides the persistent gate, and its latent loses a still cube on a persistent colored background (Appendix M).

Outlook. In a stochastic world the controlled state, random shocks and arrivals included, is information to model rather than noise to predict away. Reconstructing it and sampling its future is the simpler way to keep it; where capacity is short, a feature's value, not its predictability, should decide.

REPRODUCIBILITY STATEMENT

Environments, models, agents, evaluation code and the scripts that generate both main tables will be released with the code. Every model number comes from a fixed final checkpoint and every agent number from a final checkpoint evaluated on stored test episodes; the training scripts enforce the compute budgets. The protocol and its verdict thresholds were fixed before the experiments, every later amendment is dated in the repository, model, data and source hashes are recorded beside every run, and a checker maps every number in this paper to the artifact it comes from. The code was extended twice during the experiments, to add the two noisy gate worlds and the longer agent budget of Appendix T, both times only by adding to the shared code path.

USE OF LARGE LANGUAGE MODELS

Language-model coding agents helped write some of the experiment code; the authors specified the protocols, audited the code and results, and are responsible for every claim. For retrieval and discovery, language models helped find related work. For writing, language models helped to polish and proofread the text, which the authors revised. The authors take full responsibility for all content and every claim.

REFERENCES

Eloi Alonso, Adam Jelley, Vincent Micheli, Anssi Kanervisto, Amos Storkey, Tim Pearce, and François Fleuret. Diffusion for world modeling: Visual details matter in atari. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

Mahmoud Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, and Nicolas Ballas. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning, 2025.

Marco Bagatella, Matteo Pirotta, Ahmed Touati, Alessandro Lazaric, and Andrea Tirinzoni. TD-JEPA: Latent-predictive representations for zero-shot reinforcement learning. In *International Conference on Learning Representations*, 2026. URL https://arxiv.org/abs/2510.00739.

Jiaxin Bai and Jiaxuan Xiong. Temporal-distance JEPA: Plan-aware representation learning for latent world model predictive control, 2026. URL https://arxiv.org/abs/2607.25337.

Randall Balestriero and Yann LeCun. How learning by reconstruction produces uninformative features for perception. In *International Conference on Machine Learning (ICML)*, 2024.

Randall Balestriero and Yann LeCun. LeJEPA: Provable and scalable self-supervised learning without the heuristics, 2025.

Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video, 2024.

Boyuan Chen, Diego Martí Monsó, Yilun Du, Max Simchowitz, Russ Tedrake, and Vincent Sitzmann. Diffusion forcing: Next-token prediction meets full-sequence diffusion. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, Shane Legg, and Koray Kavukcuoglu. IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In *International Conference on Machine Learning (ICML)*, 2018.

Amir-massoud Farahmand, Andre Barreto, and Daniel Nikovski. Value-aware loss function for model-based reinforcement learning. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 1486–1494, 2017.

Jesse Farebrother, Jordi Orbay, Quan Vuong, Adrien Ali Taiga, Yevgen Chebotar, Ted Xiao, Alexander Irpan, Sergey Levine, Pablo Samuel Castro, Aleksandra Faust, Aviral Kumar, and Rishabh Agarwal. Stop regressing: Training value functions via classification for scalable deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2024.

Xiang Fu, Ge Yang, Pulkit Agrawal, and Tommi Jaakkola. Learning task informed abstractions. In *International Conference on Machine Learning (ICML)*, 2021.

Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre H. Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Daniel Guo, Mohammad Gheshlaghi Azar, Bilal Piot, Koray Kavukcuoglu, Rémi Munos, and Michal Valko. Bootstrap your own latent: A new approach to self-supervised learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Christopher Grimm, André Barreto, Satinder Singh, and David Silver. The value equivalence principle for model-based reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Christopher Grimm, André Barreto, Gregory Farquhar, David Silver, and Satinder Singh. Proper value equivalence. In *Advances in Neural Information Processing Systems*, volume 34, pp. 7773–7786, 2021. URL https://proceedings.neurips.cc/paper/2021/file/400e5e6a7ce0c754f281525fae75a873-Paper.pdf.

Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations (ICLR)*, 2020.

Danijar Hafner, Wilson Yan, and Timothy Lillicrap. Training agents inside of scalable world models, 2025.

Nicklas Hansen and Xiaolong Wang. Hallucination in world models is predictable and preventable, 2026. URL https://arxiv.org/abs/2606.27326.

Matteo Hessel, Ivo Danihelka, Fabio Viola, Arthur Guez, Simon Schmitt, Laurent Sifre, Theophane Weber, David Silver, and Hado van Hasselt. Muesli: Combining improvements in policy optimization. In *International Conference on Machine Learning*, 2021.

Yongchao Huang. VJEPA: Variational joint embedding predictive architectures as probabilistic world models, 2026.

Yanchen Huo, Ziying Song, and Yadan Luo. Flow-JEPA: Flow matching for robust latent dynamics in JEPA world models, 2026.

Nathan Lambert, Brandon Amos, Omry Yadan, and Roberto Calandra. Objective mismatch in model-based reinforcement learning. In *Conference on Learning for Dynamics and Control (L4DC)*, 2020.

Yann LeCun. A path towards autonomous machine intelligence, 2022. Version 0.9.2, OpenReview.

Wenyuan Li, Guang Li, Keisuke Maeda, Takahiro Ogawa, and Miki Haseyama. Predictive but not plannable: RC-aux for latent world models, 2026. URL https://arxiv.org/abs/2605.07278.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.

Etai Littwin, Omid Saremi, Madhu Advani, Vimal Thilak, Preetum Nakkiran, Chen Huang, and Joshua Susskind. How JEPA avoids noisy features: The implicit bias of deep linear self distillation networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Muyuan Liu, Yue Huang, Zheng Liang, and Xiang Gao. Toward physically grounded JEPA world models for goal-conditioned robotic planning, 2026. URL https://arxiv.org/abs/2609.03565. IROS 2026 Workshop on Physical World Models for Scaling Embodied AI.

Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels, 2026.

Michael Matthews, Michael Beukman, Benjamin Ellis, Mikayel Samvelyan, Matthew Jackson, Samuel Coward, and Jakob Foerster. Craftax: A lightning-fast benchmark for open-ended reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2024.

Tianwei Ni, Benjamin Eysenbach, Erfan Seyedsalehi, Michel Ma, Clement Gehring, Aditya Mahajan, and Pierre-Luc Bacon. Bridging state and history representations: Understanding self-predictive RL. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://openreview.net/forum?id=ms0VgzSGF2.

François Porcher, Nicolas Carion, Karteek Alahari, and Shizhe Chen. Flow matching in feature space for stochastic world modeling, 2026. URL https://arxiv.org/pdf/2606.29059v2.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.

Vlad Sobal, Jyothir S V, Siddhartha Jalagam, Nicolas Carion, Kyunghyun Cho, and Yann LeCun. Joint embedding predictive architectures focus on slow features, 2022. URL https://arxiv.org/abs/2211.10831.

Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models, 2025.

Yunhao Tang, Zhaohan Daniel Guo, Pierre Harvey Richemond, Bernardo Avila Pires, Yash Chandak, Remi Munos, Mark Rowland, Mohammad Gheshlaghi Azar, Charline Le Lan, Clare Lyle, András György, Shantanu Thakoor, Will Dabney, Bilal Piot, Daniele Calandriello, and Michal Valko. Understanding self-predictive learning for reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2023. URL https://proceedings.mlr.press/v202/tang23d.html.

Basile Terver, Tsung-Yen Yang, Jean Ponce, Adrien Bardes, and Yann LeCun. What drives success in physical planning with joint-embedding predictive world models? *Transactions on Machine Learning Research*, 2026. URL https://arxiv.org/abs/2512.24497.

Hugues Van Assel, Mark Ibrahim, Tommaso Biancalani, Aviv Regev, and Randall Balestriero. Joint embedding vs reconstruction: Provable benefits of latent space prediction for self supervised learning, 2025.

Claas A. Voelcker, Victor Liao, Animesh Garg, and Amir-massoud Farahmand. Value gradient weighted model-based reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2022.

Yi Wan, Muhammad Zaheer, Adam White, Martha White, and Richard S. Sutton. Planning with expectation models. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 3649–3655, 2019.

Tongzhou Wang, Simon S. Du, Antonio Torralba, Phillip Isola, Amy Zhang, and Yuandong Tian. Denoised MDPs: Learning world models better than the world itself. In *International Conference on Machine Learning (ICML)*, 2022.

Haodong Yan, Jiaguan Zhu, Mingyuan Jia, Ruiqing Yin, Junjie He, Zhide Zhong, Junfeng Li, Jinxuan Lu, Hengtao Li, Tianran Zhang, Jiayi Chen, Wenxuan Song, Wen Chen, Yuxiang Gao, and Haoang Li. Is forward prediction enough? physical state grounding for JEPA world models, 2026. URL https://arxiv.org/abs/2608.06799.

Amy Zhang, Rowan McAllister, Roberto Calandra, Yarin Gal, and Sergey Levine. Learning invariant representations for reinforcement learning without reconstruction. In *International Conference on Learning Representations (ICLR)*, 2021.

Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning, 2024.

Chuning Zhu, Max Simchowitz, Siri Gadipudi, and Abhishek Gupta. RePo: Resilient model-based reinforcement learning by regularizing posterior predictability. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

A WHAT THE OBJECTIVE DOES NOT GUARANTEE

The experiments concern fitted finite networks. The statements below are about populations, fixed encoders and reduced models: they say which guarantees follow from prediction error and a Gaussian latent marginal alone, they describe the two forces of Section 2.1, and they separate the three places a downstream error can come from. Where the main text leans on one of them, it leans only on the part stated here. Propositions 2 and 4 are *counterexamples*: they show that a property cannot be inferred from the objective. Lemma 1 and the replicator form describe how a target gradient selects content, in a reduced model. Proposition 3 is *exact* for its family, and its prediction is tested on every failing fit (Appendix C). Proposition 5 is an *identity*: it tells the diagnostics of Section 4 what they measure.

A.1 NON-COLLAPSE DOES NOT SELECT THE CONTROLLED STATE

Write observations as $x_t = R(p_t, b)$, where p_t is controlled state and b is an episode-constant nuisance recoverable from the image. Assume that b is independent of states and actions in the training data. At deployment, assume it is independent of the initial state and transition randomness, and does not directly affect dynamics or reward. Let SIGReg_∞ be the nonnegative population characteristic-function discrepancy, which vanishes for $\mathcal{N}(0, I_D)$ (Appendix B).

Proposition 2 (Gaussian nuisance optimum; extending Sobal et al. (2022))**.** *If the encoder class contains a map $f(R(p, b)) = \varphi(b)$ with $\varphi(b) \sim \mathcal{N}(0, I_D)$ and the predictor class contains the last-latent copy map, their population prediction loss and SIGReg_∞ are both zero. This is a global optimum with covariance I_D and $I(z_t; p_{1:T}) = 0$ under the training distribution. A deployed controller observing only these latents, time, and its own actions has the value of a randomized state-blind policy, one that may depend on time, its own past actions and private randomness but not on the state.*

The argument is simple: nuisance changes across episodes, satisfying the marginal constraint, but is constant within each episode, making prediction exact. Its deployment value follows by giving a state-blind policy the same private nuisance draw and controller randomness. The Gaussianization assumption requires a rich nuisance and a realizable measurable map; a finite texture does not literally instantiate it. The finite-batch regularizer also differs from the population functional. A complementary innovation bound in Appendix B shows that an encoder preserving all state distances at a fixed scale must incur prediction error for unpredictable state innovations. What the proposition says about the fits of Section 4 is only this: a healthy marginal and a low loss are compatible with an encoder that carries nothing a controller needs, so neither can be read as evidence that the state is kept. The failing per-frame-plus-wind cell is *not* an instance of the construction, since its loss is high, and the proposition does not explain why persistence protects the cube; the matched design measures that.

A.2 A TARGET GRADIENT SELECTS BY PREDICTABILITY

Write $S_B(v)$ for the expected value of the implemented regularizer (Equation 9) on a batch of B independent rows drawn from $\mathcal{N}(0, vI_D)$. Every projection $u^\top z$ is then $\mathcal{N}(0, v)$, and $\mathbb{E}|\widehat{\varphi}_B - \varphi|^2 = (1 - |\varphi|^2)/B$ for the empirical characteristic function, so

$$S_B(v) = \sum_k q_k w_k \left[B\big(e^{-\tau_k^2 v/2} - w_k\big)^2 + 1 - e^{-\tau_k^2 v} \right] \tag{3}$$

for any number of projections. For $B = 128$: $S_B(1) = 1.052$, $\arg\min_v S_B(v) = 0.984$, $S'_B(1) = 0.48$, $S''_B(1) = 30.4$.

Lemma 1 (Shrinking a few directions barely changes the regularizer)**.** *Let the rows be $\mathcal{N}(0, I_D - \Delta P)$ with P a rank-k orthogonal projector. Then the expected regularizer is $\mathbb{E}_u S_B(1 - \Delta\, u^\top P u)$ over uniform unit projections u, and*

$$\mathbb{E}_u S_B(1 - \Delta\, u^\top P u) - S_B(1) = -S'_B(1)\,\Delta\,\frac{k}{D} + \tfrac{1}{2} S''_B(1)\,\Delta^2\,\frac{k(k+2)}{D(D+2)} + O(\Delta^3). \qquad (4)$$

If the predictor predicts a fraction $1 - \bar{u}$ of the shrunk content, shrinking it saves $\Delta\, k\, \bar{u}/D$ of prediction loss. The first-order term of the regularizer has the same sign (since $S'_B(1) > 0$), and the second-order resistance is smaller by a factor of order k/D; at $\lambda = 0.09$ and $D = 512$ it is below $6 \times 10^{-6}\,\Delta^2 k(k+2)$. So for $k \ll D$ the regularizer does not protect a content the predictor has not learned.

Replicator form. In a reduced model, let the latent consist of independent contents in orthogonal directions with amplitudes a_k, total variance $\sum_k a_k^2 = 1$ held fixed (the regularizer's scale term), shares $s_k = a_k^2$, and let the current predictor leave a fraction u_k of content k unpredicted. The prediction loss is $\sum_k a_k^2 u_k$; its gradient with respect to the target's amplitudes, projected onto the constraint, gives $\dot{a}_k = -2\eta a_k(u_k - \bar{u})$ with $\bar{u} = \sum_j s_j u_j$, hence Equation 2 with $a = 4\eta$. With a stop-gradient or moving-average target this term is absent. If the predictor learns content k at a rate proportional to its share, a content whose competitors are already predicted decays at rate about a while it is unlearned, and it is learned only if its initial share exceeds a threshold proportional to the ratio of the encoder's selection speed to the predictor's learning speed; below it, share and knowledge decay together. These are statements about the reduced model; Section 4 tests their prediction by removing the target gradient and nothing else.

A.3 The white-noise fixed point

Proposition 3 (White-noise fixed point)**.** *Let the encoder output $z_t = \sqrt{v}\,\xi_t$ with $\xi_t \sim \mathcal{N}(0, I_D)$ independent across time and of states, actions and past latents, for example a Gaussianizing function of per-frame noise pixels. Then (i) every predictor has prediction loss at least v, with equality for the constant 0; (ii) the expected regularizer is $S_B(v)$; (iii) within this family the objective $v + \lambda S_B(v)$ is minimized at $v^\star(\lambda)$ with $1 + \lambda S'_B(v^\star) = 0$, giving $v^\star = 0.484,\ 0.734,\ 0.882$ and 0.951 at $\lambda = 0.03,\ 0.09,\ 0.27$ and 0.9; (iv) with a stop-gradient target the prediction term exerts no force on the target's scale, and the stationary scale is $\arg\min_v S_B(v) = 0.984$. At this point the latent carries no information about the state.*

Along any direction that adds a zero-mean state feature at amplitude ε, the added target variance costs ε^2 times its trace in prediction loss and saves exactly as much in the regularizer, by (iii); the objective is therefore flat to second order, and whether training leaves the point is decided at fourth order by how predictable the feature is from the history (Appendix C).

A.4 Decodability does not set a useful prediction scale

Even preserving all state information leaves its geometry unconstrained. For any $\varepsilon > 0$, we can place the two values of a relevant bit less than 2ε apart while keeping the *entire* latent distribution exactly standard Gaussian.

Proposition 4 (Exact information at arbitrarily small scale)**.** *For every $\varepsilon > 0$ and $D \geq 1$, there is a measurable observation process with episode-constant nuisance b and a fresh independent fair state bit p_t, and an encoder f_ε, such that $z_t \sim \mathcal{N}(0, I_D)$ and both p_t and b are exactly recoverable almost surely. For fixed b, $0 < \|f_\varepsilon(R(1,b)) - f_\varepsilon(R(0,b))\| < 2\varepsilon$. The optimal next-latent predictor is independent of the next bit and has coordinate-mean squared error that is positive and below ε^2/D. With reward $\mathbf{1}\{a_t = p_t\}$, a controller that observes z_t before acting attains value 1 per step, while every state-blind policy attains $1/2$.*

The construction partitions one Gaussian coordinate into narrow intervals and uses the bit to select the lower or upper half of each interval's conditional probability mass. The nuisance fixes the

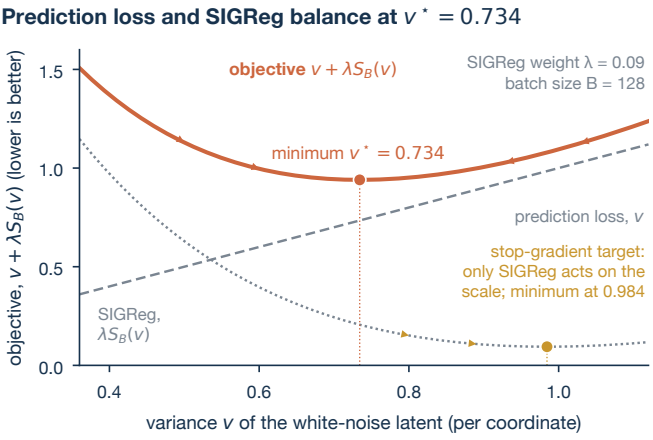


Figure 3: **The prediction loss and SIGReg set the scale of a white-noise latent.** For a latent $z_t = \sqrt{v}\,\xi_t$ that carries no information, the prediction loss is v (dashed) and the expected regularizer is $\lambda S_B(v)$ (dotted; $\lambda = 0.09$, $B = 128$). Their sum, the objective (solid), is lowest at $v^\star = 0.734$ (Proposition 3). With a stop-gradient target the prediction loss exerts no force on the scale, which settles at the minimum of the regularizer alone, 0.984. Arrows: the direction in which each force moves v.

interval and the within-half quantile. This preserves the Gaussian marginal and exact decoding while making the two states arbitrarily close. The full proof is in Appendix B. It uses infinite-precision measurable coding; the next bit is intrinsically unpredictable. It therefore separates information from scale, without proving avoidable prediction error or explaining deletion of the predictable gate wall. Nor is f_ε what the objective selects: in the same world the nuisance-only encoder of Proposition 2 attains exactly zero, so f_ε is an (ε^2/D)-approximate optimum, and the construction bounds what the objective *permits*, not what it prefers. The class-token cell of Section 4, where a rollout error of 0.0004 of the latent variance destroys a read-out that is exact on clean latents, is the finite analogue of this scale mismatch, observed and then reproduced by injecting noise of that size; the proposition says only that the objective could not have prevented it.

A.5 Where a downstream error comes from

Fix a frozen encoder and a context c, the latent and action history the predictor reads, and let Z' be the next latent with conditional law P_c and conditional mean $m(c) = \mathbb{E}[Z' \mid c]$. Let $h : \mathbb{R}^D \to \mathbb{R}$ be a task quantity of the next latent (the probability that the transition it starts is fatal, its reward, a decoded pixel statistic) and $\widehat{h}$ a fitted head for it. What imagination has to supply is $u(c) = \mathbb{E}[h(Z') \mid c]$. A deterministic predictor $\widehat{g}$ supplies $\widehat{h}(\widehat{g}(c))$; a sampler with conditional law $\widehat{Q}_c$ supplies $\mathbb{E}_{\widehat{Q}_c}\widehat{h}(\widehat{Z}')$.

Proposition 5 (Three terms)**.** *For every context c,*

$$\widehat{h}(\widehat{g}(c)) - u(c) = \underbrace{\big[\widehat{h} - h\big](\widehat{g}(c))}_{\text{head error, off distribution}} + \underbrace{h(\widehat{g}(c)) - h(m(c))}_{\text{predictor approximation}} + \underbrace{h(m(c)) - \mathbb{E}[h(Z') \mid c]}_{\text{read-out gap of the mean}}. \tag{5}$$

If h is L-Lipschitz, the second term is at most $L\,\|\widehat{g}(c) - m(c)\|$ in absolute value and the third at most $L\sqrt{\mathrm{trCov}(Z' \mid c)}$; if h is twice differentiable with $\|\nabla^2 h\| \le M$, the third is at most $\frac{M}{2}\mathrm{trCov}(Z' \mid c)$, and it vanishes when h is affine on the support of P_c or when P_c is a point mass. For a sampler,

$$\mathbb{E}_{\widehat{Q}_c}\widehat{h}(\widehat{Z}') - u(c) = \mathbb{E}_{\widehat{Q}_c}\big[\widehat{h} - h\big](\widehat{Z}') + \Big(\mathbb{E}_{\widehat{Q}_c} h(\widehat{Z}') - \mathbb{E}_{P_c} h(Z')\Big), \tag{6}$$

where the second term is at most $L\,W_1(\widehat{Q}_c, P_c)$ in absolute value, with W_1 the Wasserstein-1 distance.

The identity is a rearrangement and the bounds are Lipschitz continuity, a second-order Taylor remainder and Kantorovich–Rubinstein duality (Appendix B); its use is in what it separates. The

read-out gap of the mean is mechanism 2 of Section 2: it needs both a spread conditional law and a read-out that is not affine across that spread, and it is the only term a distributional predictor removes, at the price of the transport term in Equation 6. The *predictor approximation* term is what a trained network adds on top of the ideal conditional mean; a small latent error controls it only relative to the read-out's sensitivity at that scale (Equation 7), which Proposition 4 shows the objective does not constrain. The *head error* is evaluated at the predicted latent, a point the head was never fitted on, and is the term the imagination policies of Section 5 are paid by. In a rollout the three terms recur at every step with the imagined history in place of c; nothing here bounds their compounding.

Two consequences organize the diagnostics. For a wall already in view in the black gate, the two possible futures render the same solid wall until the opening is revealed, so P_c is a point mass on everything but the opening, the third term is exactly zero for any read-out of the shared wall, and a wall missing from the predicted frame is the second term. For a wall still to arrive, P_c is spread over the arrival time, and the third term is the whole wall (Proposition 1); Section 4 measures both cases by decoding the oracle mean through the model's decoder. And the sampler's version replaces the read-out gap by a transport term but keeps the head term, so where sampling does not repair imagination, the head term or the transport term dominates, and the decomposition says which two quantities to measure there.

Why scale matters for a downstream read-out. For any fixed L-Lipschitz downstream score q and any coupled real and predicted next latents,

$$\left|\mathbb{E}q(Z') - \mathbb{E}q(\widehat{Z}')\right| \le L\sqrt{\mathbb{E}\|Z' - \widehat{Z}'\|^2}. \tag{7}$$

Prediction error controls score error only relative to the read-out's sensitivity, and exact decodability alone supplies no bound on that sensitivity. With $\widehat{Z}' = m(c)$ the right-hand side is $L\sqrt{\mathbb{E}\mathrm{trCov}(Z' \mid c)}$, the read-out gap of Proposition 5; and a planner needs an argmax rather than an expectation, which survives whenever twice the right-hand side is below the read-out's gap between the best and second-best action. We do not estimate L or claim a control guarantee for the fitted networks.

B Proofs

B.1 Empirical objective and population idealization

For a training batch of B windows and H predicted time positions, the implemented loss is

$$\mathcal{L}_{\mathrm{pred}} = \frac{1}{BHD}\sum_{b,h}\|\widehat{z}_{b,h+1} - z_{b,h+1}\|^2. \tag{8}$$

The notation in Equation 1 averages over time and uses this coordinate-mean convention. One-step targets are teacher-forced at every temporal position, with a shorter history near the start of a window. For encoded windows of T frames, P random unit projection directions u_p, K knots $\tau_k \in [0,3]$, and $w_k = e^{-\tau_k^2/2}$, the regularizer is

$$R = \frac{B}{TP}\sum_{t,p,k} q_k w_k \left[\left(\frac{1}{B}\sum_b \cos(\tau_k u_p^\top z_{b,t}) - w_k\right)^2 + \left(\frac{1}{B}\sum_b \sin(\tau_k u_p^\top z_{b,t})\right)^2\right], \tag{9}$$

where $\Delta = 3/(K-1)$, with $q_k = \Delta$ at the endpoints and 2Δ in the interior. The characteristic function is averaged across batch examples separately at each temporal position; the factor B is part of the implementation with $\lambda = 0.09$.

The population statements use the nonnegative discrepancy

$$\mathrm{SIGReg}_\infty(\mathcal{L}(z)) = \int_{\mathbb{S}^{D-1}}\int_0^3 w(\tau)\left[\left(\mathbb{E}\cos(\tau\langle z,u\rangle) - w(\tau)\right)^2 + \left(\mathbb{E}\sin(\tau\langle z,u\rangle)\right)^2\right] d\tau\, d\sigma(u), \tag{10}$$

where $w(\tau) = e^{-\tau^2/2}$ and σ is uniform on the sphere. Only the fact that this functional vanishes for a standard Gaussian is needed. It is not a theorem about exact minima of the finite-batch objective.

B.2 NUISANCE OPTIMUM AND THE COST OF INNOVATIONS

Proposition 2.

Proof. Let β recover the nuisance from the observation, set $f^\star = \varphi \circ \beta$, and let $g^\star$ copy the last latent. Then $z_t = \varphi(b) = z_{t+1}$, so every prediction residual vanishes. The Gaussian marginal gives covariance I_D and zero population regularizer. Both objective terms are nonnegative, so their sum is globally minimized at zero. Training-distribution independence gives $I(\varphi(b); p_{1:T}) = 0$.

For deployment, draw nuisance B and the controller's random seed U at reset, independently of the initial state and transition randomness. Run the original controller in the environment with nuisance B. Let a state-blind policy with private seed (B, U) feed constant latent history $\varphi(B)$ to a copy of that controller and reproduce its actions with U. Coupled to the same initial state and transition randomness, the policies take the same first action. If their actions agree through step t, their states, supplied latent histories, and seeds agree, so the next actions agree as well. Induction gives identical trajectories, returns, and terminations. This does not assume independence between B and states subsequently influenced by its induced actions. □

Corollary 1 (Innovation cost for uniformly state-resolving encoders)**.** *Let $p_{t+1} = p_t + \mu(a_t) + w_t$, $\mathcal{H}_t = \sigma(p_{\le t}, b, a_{\le t})$, and $\Sigma_t = \mathrm{Cov}(w_t \mid \mathcal{H}_t)$. Assume finite second moments of states and encoded observations, $x_t = R(p_t, b)$, and $\|f(R(p, b)) - f(R(p', b))\| \ge \kappa \|p - p'\|$ for all admissible p, p', b, with $\kappa > 0$; the nuisance is $\mathcal{H}_t$-measurable, so it is fixed under the conditioning below. Every square-integrable predictor of latent and action history satisfies*

$$\frac{1}{D}\mathbb{E}\|g - f(x_{t+1})\|^2 \ge \frac{\kappa^2}{D}\mathbb{E}\mathrm{tr}\Sigma_t. \tag{11}$$

When $\mathbb{E}\mathrm{tr}\Sigma_t > 0$, this exceeds the nuisance encoder's zero prediction loss under the assumptions of Proposition 2.

The assumption is stronger than injectivity or task sufficiency: every state distance must be preserved at a uniform scale. Innovation is measured after conditioning on state history and nuisance; persistent wind is not automatically fresh innovation.

Remark 1 (The bound has a ceiling). The Gaussian marginal caps κ. If p and b are independent and $\mathrm{Cov}(z) = I_D$ with $\mathbb{E}z = 0$, two independent state draws P, P' sharing one nuisance give latents with $\mathbb{E}\|z - z'\|^2 \le 2\mathbb{E}\|z\|^2 + 2\mathbb{E}\|z'\|^2 = 4D$, while the expansion gives $\mathbb{E}\|z - z'\|^2 \ge \kappa^2\mathbb{E}\|P - P'\|^2 = 2\kappa^2\mathrm{trCov}(p)$. Hence $\kappa^2 \le 2D/\mathrm{trCov}(p)$ and the certified cost in Equation 11 is at most $2\,\mathbb{E}\mathrm{tr}\Sigma_t/\mathrm{trCov}(p)$, twice the ratio of the world's fresh noise to its state spread, in the per-coordinate units of Table 1. In the wind worlds $\mathbb{E}\mathrm{tr}\Sigma_t \approx 0.35$ and $\mathrm{trCov}(p) \approx 300$ square pixels, so the corollary can certify at most 0.0024 of prediction loss. The persistent-noise pair of Section 4 pays $0.0030 - 0.0014 = 0.0016$ extra for the wind and keeps the cube; the per-frame pair pays 0.733 extra, three hundred times the certified price, and loses it. The corollary prices the incentive; it does not explain the failing cell.

Corollary 1.

Proof. Put $Z = f(x_{t+1})$ and $\mathcal{C}_t = \sigma(z_{\le t}, a_{\le t})$. Because observations and latents are measurable functions of state and nuisance, $\mathcal{C}_t \subseteq \mathcal{H}_t$. Any predictor using a shorter window is also $\mathcal{C}_t$-measurable. Conditional least squares and the tower property give

$$\begin{aligned}\mathbb{E}\|g - Z\|^2 &\ge \mathbb{E}\|Z - \mathbb{E}[Z \mid \mathcal{C}_t]\|^2 \\ &= \mathbb{E}\|Z - \mathbb{E}[Z \mid \mathcal{H}_t]\|^2 + \mathbb{E}\|\mathbb{E}[Z \mid \mathcal{H}_t] - \mathbb{E}[Z \mid \mathcal{C}_t]\|^2 \\ &\ge \mathbb{E}\mathrm{trCov}(Z \mid \mathcal{H}_t).\end{aligned}$$

The cross term vanishes because its second factor is $\mathcal{H}_t$-measurable, by $\mathcal{C}_t \subseteq \mathcal{H}_t$, and its first has conditional mean zero given $\mathcal{H}_t$. Take conditionally independent copies P, P' of p_{t+1} given $\mathcal{H}_t$. Such copies exist for Euclidean state spaces, using their regular conditional distributions. The nuisance b and current state p_t are fixed under this conditioning. The conditional independent-copy

variance identity yields

$$\begin{aligned}\mathrm{trCov}(Z \mid \mathcal{H}_t) &= \tfrac{1}{2}\mathbb{E}[\|f(R(P,b)) - f(R(P',b))\|^2 \mid \mathcal{H}_t] \\ &\geq \tfrac{\kappa^2}{2}\mathbb{E}[\|P - P'\|^2 \mid \mathcal{H}_t] \\ &= \kappa^2 \mathrm{trCov}(p_{t+1} \mid \mathcal{H}_t) = \kappa^2 \mathrm{tr}\Sigma_t.\end{aligned}$$

The final equality follows because $p_t + \mu(a_t)$ is $\mathcal{H}_t$-measurable. Taking expectations and dividing by D proves Equation 11. Averaging these inequalities gives the corresponding bound for a multi-time prediction objective. Proposition 2 supplies the zero-loss comparator when its assumptions also hold. □

B.3 Conditional prediction and geometric scale

Lemma 2 (Conditional mean)**.** *For a fixed encoder and data law with $\mathbb{E}\|z'\|^2 < \infty$, the unique squared-error minimizer over measurable square-integrable predictors from history c and action a is $m(c,a) = \mathbb{E}[z' \mid c, a]$, up to null sets.*

Proof. Let $m = \mathbb{E}[z' \mid c, a]$. For every square-integrable measurable $g(c,a)$, expand $g - z' = (g - m) + (m - z')$. The cross term has expectation zero since $\mathbb{E}[m - z' \mid c, a] = 0$. Hence

$$\mathbb{E}\|g - z'\|^2 = \mathbb{E}\|g - m\|^2 + \mathbb{E}\mathrm{trCov}(z' \mid c, a).$$

The second term does not depend on g. The first is nonnegative and vanishes exactly when $g = m$ almost surely. □

Proposition 5.

Proof. Adding and subtracting $h(\widehat{g}(c))$ and $h(m(c))$ gives Equation 5. If h is L-Lipschitz, $|h(\widehat{g}(c)) - h(m(c))| \leq L\|\widehat{g}(c) - m(c)\|$, and by Jensen's inequality $|h(m(c)) - \mathbb{E}[h(Z') \mid c]| \leq \mathbb{E}[|h(m(c)) - h(Z')| \mid c] \leq L\,\mathbb{E}[\|Z' - m(c)\| \mid c] \leq L\sqrt{\mathrm{trCov}(Z' \mid c)}$. If h is twice differentiable with $\|\nabla^2 h\| \leq M$, the integral form of Taylor's theorem gives $h(Z') = h(m) + \nabla h(m)^\top (Z' - m) + \rho$ with $|\rho| \leq \frac{M}{2}\|Z' - m\|^2$; the linear term has conditional mean zero, so $|\mathbb{E}[h(Z') \mid c] - h(m)| \leq \frac{M}{2}\mathbb{E}[\|Z' - m\|^2 \mid c] = \frac{M}{2}\mathrm{trCov}(Z' \mid c)$. If h is affine on the support of P_c the remainder is zero, and if P_c is a point mass then $Z' = m(c)$ almost surely. For the sampler, adding and subtracting $\mathbb{E}_{\widehat{Q}_c} h(\widehat{Z}')$ gives Equation 6, and the Kantorovich–Rubinstein duality $W_1(\widehat{Q}, P) = \sup_{\mathrm{Lip}(\phi)\leq 1} |\mathbb{E}_{\widehat{Q}}\phi - \mathbb{E}_P\phi|$ applied to $\phi = h/L$ gives the transport bound. All expectations are finite because h is Lipschitz and the latents have finite second moments. □

The finite-sample regularizer. The finite-sample regularizer differs from Equation 10. For independent identically distributed samples with characteristic function ϕ and empirical estimate $\widehat{\phi}_n$,

$$\mathbb{E}|\widehat{\phi}_n(t) - w(t)|^2 = |\phi(t) - w(t)|^2 + \frac{1 - |\phi(t)|^2}{n}. \tag{12}$$

Indeed, $\mathbb{E}\widehat{\phi}_n = \phi$ and $\mathbb{E}|\widehat{\phi}_n - \phi|^2 = (1 - |\phi|^2)/n$, since each complex exponential has modulus one; expanding around ϕ gives the identity. At a Gaussian law, the residual equals $(1 - e^{-t^2})/n$. This is estimator bias, not an optimization floor. Changing the law changes both terms; dependent temporal samples introduce additional covariance terms. The implementation multiplies this statistic by batch size B, so its Gaussian finite-batch bias does not shrink as $1/B$ after that scaling. The second term decreases in $|\phi|$, so the finite-batch statistic pays a small premium for concentration: at a single knot its minimizer over real $\phi \in [0,1]$ is $\phi^\star = wB/(B-1)$, displaced $O(1/B)$ from the Gaussian in the collapse direction. For an exactly Gaussian batch of independent rows the expected value of Equation 9 is $\sum_k q_k w_k (1 - e^{-\tau_k^2}) = 1.05$ with the implementation's 17 knots on $[0,3]$, independent of B; this is the reference against which the fitted values of Section 4 are read. Finite projections and knots, and the bounded range $\tau \leq 3$, do not characterize Gaussianity.

Proposition 4.

Proof. Partition $\mathbb{R}$ into $I_j = [2\varepsilon j, 2\varepsilon(j+1))$, $j \in \mathbb{Z}$. Let $q_j = \Pr(G \in I_j)$ for $G \sim \mathcal{N}(0,1)$. Every q_j is positive and their sum is one. Let F_j be the conditional Gaussian distribution function on I_j, with inverse quantile function Q_j on $(0,1)$. The conditional CDF is continuous and strictly increasing on the interval interior, so its inverse is measurable and takes values in that interior.

Draw J with probabilities q_j, $U \sim \mathrm{Uniform}(0,1)$, and $V \sim \mathcal{N}(0, I_{D-1})$, independently. Set $b = (J, U, V)$; for $D = 1$, omit V. Take $R(p,b) = (p,b)$ and define

$$f_\varepsilon(R(p,b)) = \big(Q_J((U+p)/2), V\big). \tag{13}$$

The mixture $(U + p_t)/2$ is uniform on $(0,1)$, since p_t is an independent fair bit. Conditional on $J = j$, the first coordinate therefore has the Gaussian law restricted to I_j. Mixing with weights q_j recovers $\mathcal{N}(0,1)$. Independence of V gives the joint law $\mathcal{N}(0, I_D)$ and zero population regularizer.

Given the first coordinate y, its interval determines J almost surely. Write $v = F_J(y)$. Then $p_t = \mathbf{1}\{v > 1/2\}$ and $U = 2v - p_t$ almost surely; the other coordinates recover V. Thus the latent recovers both state and nuisance. For fixed b, the two first coordinates are distinct and lie inside the same interval of width 2ε. All remaining coordinates agree, proving the distance bound.

Latent history reveals b, while the next bit remains independent and fair. The next latent consequently has two equiprobable values, say $z^{(0)}(b)$ and $z^{(1)}(b)$. Lemma 2 gives $m(b) = (z^{(0)}(b) + z^{(1)}(b))/2$. Its conditional squared error is $\|z^{(1)}(b) - z^{(0)}(b)\|^2/4$, strictly positive and strictly below ε^2 for every b. The midpoint is square-integrable by Jensen's inequality, since both state encodings have finite second moments. Taking expectations and dividing by D proves the coordinate-mean loss bound, including strict positivity since the conditional error is positive almost surely. The midpoint depends only on b, so it is independent of p_{t+1}. Finally, a controller that observes z_t before acting decodes the current bit and plays $a_t = p_t$. A state-blind action is independent of the current fair bit, giving success probability $1/2$. The construction is not selected by the objective: the nuisance (J, U, V) alone maps to $(Q_J(U), V) \sim \mathcal{N}(0, I_D)$, an encoder of the form of Proposition 2 with exactly zero prediction loss and zero regularizer in the same world, so f_ε is an (ε^2/D)-approximate global optimum that the objective strictly prefers to discard. □

B.4 Selection, the white-noise point and arrivals

Equation 3.

Proof. For rows $z_b \sim \mathcal{N}(0, vI_D)$ and a unit direction u, $y_b = u^\top z_b \sim \mathcal{N}(0, v)$ independently over b, so $\mathbb{E}\cos(\tau y_b) = e^{-\tau^2 v/2}$, $\mathbb{E}\sin(\tau y_b) = 0$ and $\mathrm{Var}\cos(\tau y_b) + \mathrm{Var}\sin(\tau y_b) = 1 - e^{-\tau^2 v}$. The bracket of Equation 9 at knot τ_k therefore has expectation $(e^{-\tau_k^2 v/2} - w_k)^2 + (1 - e^{-\tau_k^2 v})/B$; multiplying by B and summing with weights $q_k w_k$ gives Equation 3, which does not depend on u, t or the number of projections. □

Lemma 1.

Proof. Under $\mathcal{N}(0, I_D - \Delta P)$ the projection $u^\top z$ is $\mathcal{N}(0, 1 - \Delta\, x)$ with $x = u^\top P u$, so the argument above gives $S_B(1 - \Delta x)$ for each u. Expand to second order in Δ and use the moments of x for u uniform on the sphere: $x \sim \mathrm{Beta}(k/2, (D-k)/2)$, so $\mathbb{E}x = k/D$ and $\mathbb{E}x^2 = k(k+2)/(D(D+2))$. □

Proposition 3.

Proof. (i) The next latent is independent of everything a predictor reads and has mean 0 and covariance vI_D, so by Lemma 2 the best predictor is the constant 0 with coordinate-mean error v. (ii) is Equation 3. (iii) The objective restricted to the family is the scalar function $v + \lambda S_B(v)$; its stationary points solve $1 + \lambda S_B'(v) = 0$, and the values listed are the numerical minimizers ($\lambda S_B''(v^\star) > 0$ at each). (iv) With the target detached, the encoder receives the prediction loss's gradient only through the predictor's inputs, and the optimal predictor of (i) ignores its inputs, so only the regularizer acts on the scale. The latent is independent of the states by construction. □

Proposition 1.

Proof. Write $x_{t+k}(j)$ for the frame at step k when the object enters at step j, with $x_{t+k}(j) = x^0_{t+k} + o_{t+k}(j)$, where x^0 is the frame without the object and $o(j)$ the object's contribution, supported on the footprint $F_k(j)$. The footprints are disjoint across j, so at a pixel of $F_k(j)$ the mean frame is $x^0_{t+k} + P(J = j \mid h)\, o_{t+k}(j)$, which shows at most a fraction q of the object's intensity. Pixels outside every footprint show no object in any outcome. □

C THE WHITE-NOISE FIXED POINT ON EVERY FAILING FIT

Protocol. Each fit is read in evaluation mode on $1{,}024$ random reset-free windows of its world's held-out archive. v is the per-coordinate variance of the latent, MSE the per-coordinate one-step teacher-forced prediction error, kurtosis the mean over 256 random unit projections, and position R^2 that of a held-out ridge read-out of the cube's position. $v^\star$ is the prediction of Proposition 3, computed from Equation 3 with no fitted parameter. Table 4 lists every fit whose cube read-out is at chance.

Table 4: **Every failing fit sits at the white-noise fixed point.** Ranges over the seeds of a row. The criterion is MSE $= v = v^\star$: the escaped still controls (the last row, and the still joint control of Table 15, seed 0, $R^2 = 0.037$) also have kurtosis 3 and a low *linear* position R^2, but their one-step error is far below their variance.

fits	$v^\star$	v	MSE	kurtosis	position R^2
grid, per-frame wind, $\lambda = 0.09$, seeds 0–2	0.734	0.731–0.734	0.732–0.735	2.98–2.99	-0.006
same, 2× and 3× budget	0.734	0.733	0.734	2.99	-0.006
same, joint controls of Table 15, seeds 0–2	0.734	0.731–0.735	0.732–0.736	2.98–3.00	-0.006 to 0.004
grid, per-frame wind, $\lambda = 0.03$, seeds 1, 2	0.484	0.482–0.484	0.482–0.484	2.98–3.01	± 0.005
grid, per-frame wind, $\lambda = 0.27$	0.882	0.880	0.881	2.99	-0.006
grid, per-frame wind, $\lambda = 0.9$	0.951	0.951	0.952	2.99	-0.006
class token, per-frame still, seeds 0–2	0.734	0.736–0.738	0.737–0.738	2.96–2.97	-0.001 to 0.000
class token, per-frame wind, seeds 0–2	0.734	0.736–0.737	0.737–0.738	2.96–2.99	-0.010
grid, per-frame wind, stop-gradient, seeds 0–2	0.984	0.983–0.989	0.984–0.990	2.99–3.01	-0.006 to -0.004
grid, per-frame still, stop-gradient, seeds 0, 1	0.984	0.984–0.989	0.985–0.990	3.02–3.03	-0.008 to 0.015
partial escape: per-frame still, stop-gradient, seed 2	0.984	0.980	0.891	3.02	0.133
escaped control: per-frame wind, $\lambda = 0.03$, seed 0	–	1.017	0.022	3.01	0.997
escaped control: per-frame still, $\lambda = 0.09$, seed 0	–	0.990	0.0004	2.98	0.035

Why wind keeps the fits there. At the fixed point the objective is flat to second order along any state feature (Section A.3). At fourth order, a feature ϕ added at amplitude ε lowers the prediction loss by $\varepsilon^4\, \Pi(\phi)/(Dv^\star)$, where $\Pi(\phi) = \sum_{k<H} \mathbb{E}_a \|\mathrm{Cov}(\phi_{t+1}, \phi_{t-k} \mid a)\|_F^2$ is its predictability pooled over the $H = 8$ history frames of the grid JEPA's predictor, and the regularizer resists it through its curvature and the feature's excess kurtosis. For a unit Fourier feature of the cube's position at spatial frequency ω, computed exactly from the environment's dynamics, $\Pi = H = 8$ at every frequency in the still world, because the displacement follows from the action. With the hidden wind, $\Pi = 5.2$ at $\omega = 0.1$ rad/px, 0.8 at 0.5, 0.10 at 1.0 and 0.01 at 1.5: the fine features a convolutional encoder starts with are 10 to 800 times less predictable. The training logs follow this picture. Still and windy fits reach the point by update 250; the still fits leave it abruptly between updates $1{,}000$ and $1{,}500$ (the loss falls from 0.73 to 0.02 within 250 updates); the windy fits at $\lambda \geq 0.09$ never leave it in up to $25{,}500$ updates. At $\lambda = 0.03$ all three windy seeds reach the point, seed 0 leaves it after about $1{,}750$ updates and seeds 1 and 2 never do, so the seed split of Appendix R is a race on a flat point, not two different optima. Why a larger λ lengthens the plateau so much, why a stop-gradient target also holds two still fits there, and why the class-token JEPA, whose predictor reads three frames, holds all three of its still fits there, the local analysis does not explain.

D PREDICTION TARGETS: PROTOCOL AND EVERY CELL

Targets. All targets use the grid JEPA's encoder, factored predictor, data sampler, optimizer, schedule and $8{,}500$ updates. *Stop-gradient*: the target encoding is detached from the encoder's gradient.

EMA: the target is the encoding of the same window by a moving-average copy of the online encoder and projector, updated after every step with momentum rising from 0.996 to 1 on the BYOL cosine schedule (Grill et al., 2020); SIGReg stays on the online encoder's latents. With momentum 0 the EMA trainer reproduces the stop-gradient trainer (first loss to 3×10^{-6} relative, loss curves within 1.4% over 500 updates). *EMA without SIGReg*: the same with weight 0. The stop-gradient target without SIGReg collapses (latent variance 0.001 per coordinate), so it is not a fourth arm.

Measures. Wall at step 1 / 8: decoded intensity of the approaching wall in the model's own rollout, 1 and 8 steps after eight real context frames, before the opening is revealed. AUROC at 8: separation of the opening's side in the decoded frame 8 steps after the reveal. Cube ENC / PRED: pixel-probe error on the decoded encoding of real frames and on the eighth predicted frame. Cube read-out: the protocol of Table 15 on the latent. Effective rank: exponential of the entropy of the normalized covariance spectrum on 4,096 held-out frames. *Wall weight*: for 64 held-out contexts before the reveal, the recorded future and the same future without the wall are simulated with the same random keys and encoded; the weight is the mean squared distance between the two encodings, divided by D times the latent's per-coordinate variance, at the first future step. It is not a share of the variance and can exceed 1 for a low-rank latent in which the wall dominates.

Summary. Table 5 summarizes the four targets in five worlds, with the reconstruction tokenizer as the reference row; Table 6 lists every cell and seed.

Table 5: **No prediction target keeps the state in every world; reconstruction does.** The grid JEPA with each target, seed 0, same encoder, predictor, data and budget. Wall: decoded intensity of the approaching wall at the model's own eighth predicted step (1 = fully present). Cube, gate world: pixel-probe error on decoded encodings of real frames; cube, cube worlds: error of a read-out on the latent (the protocol of Table 15; chance ≈ 14.5). Reconstruction: the flow model's tokenizer and predictor, three-model means of Tables 1 and 12. [a]Seeds 1 and 2: 1.00 and 0.50. [b]Three models each: 0.38 to 0.47 and 14.4 to 14.6; persistent gate wall 0.09 to 0.10.

	black gate	gate + per-frame	gate + persistent		per-frame, still	per-frame, wind
prediction target	wall	wall	wall	cube (px)	cube (px)	cube (px)
LeWorldModel (joint)	0.00	0.14[a]	0.36	1.4	0.42	14.4[b]
stop-gradient	0.89	0.95	0.79	12.0	13.7	14.6[b]
EMA teacher, with SIGReg	0.89	0.98	0.78	16.7	13.7	14.4
EMA teacher, no SIGReg	1.00	1.00	0.10[b]	18.4	0.46	0.38[b]
reconstruction (flow)	0.96	0.94	0.89	0.27	0.26	0.25

Wall weights, black gate. LeWorldModel 7.1×10^{-5}, 5.7×10^{-5} and 1.7×10^{-4} (seeds 0 to 2); stop-gradient 0.42, 0.46 and 0.32, with the JEPA's own rollout keeping the wall at 0.89, 0.72 and 0.77 at step 8; EMA 0.20; EMA without SIGReg 1.44; the reconstruction tokenizer 0.99 (seed 0). In the gate with per-frame noise, LeWorldModel's weights are 1.9 to 3.0, and its own predictor keeps the wall at step 8 at 0.14, 1.00 and 0.50 (seeds 0 to 2). The EMA JEPA without SIGReg keeps the persistent gate's wall in its encoder (decoded encoding 0.90, opening AUROC 1.00 on the encodings, all three seeds) and drops it in its predictor from the first step; why was not established. Figure 4 shows the black-gate weights at the latents' own scale.

E AVERAGING OVER ARRIVAL TIME, AND THE FLOW MODEL'S EPISODE CLOCK

When the first wall enters. In the black gate the first wall enters at episode age 22. Eight wall-free frames do not show the age, and 61% of the training windows of eight frames have no wall on screen. Over those windows the number of steps until the wall enters is uniform on 1 to 15 (0.063 each); in the remaining 5.8% the episode ends first. Later walls enter one step after the previous wall leaves the screen, which the history shows. With a 32-frame history no training window lacks a wall.

Table 6: **Every target in every world, seed** 0, and the cells rerun at seeds 1 and 2 because their seed-0 outcome differed from both LeWorldModel's and the stop-gradient target's. Cube-world LeWorldModel rows: read-outs from the scorecard fit, cube read-out from the retrained joint control of Table 15 (same configuration, different run). "–": not scored.

world	target	wall 1 / 8	AUROC 8	cube ENC / PRED (px)	cube read-out (px)	loss / SIGReg	eff. rank
black gate	LeWorldModel	0.00 / 0.00	0.50	0.23 / 3.66	–	0.0006 / 1.17	270
	stop-gradient	1.00 / 0.89	1.00	0.27 / 0.44	–	0.024 / 1.10	328
	EMA	0.99 / 0.89	0.99	0.23 / 0.41	–	0.010 / 1.08	350
	EMA, no SIGReg	1.00 / 1.00	1.00	0.27 / 0.94	–	0.031 / (59.1)	18
gate + per-frame noise	LeWorldModel	0.94 / 0.14	0.53	0.38 / 5.75	–	0.054 / 2.79	47
	stop-gradient	1.00 / 0.95	1.00	0.28 / 0.33	–	0.857 / 1.07	441
	EMA	1.00 / 0.98	0.98	0.29 / 0.45	–	0.873 / 1.06	454
	EMA, no SIGReg	1.00 / 1.00	1.00	0.26 / 0.45	–	0.090 / (93.2)	14
gate + persistent noise	LeWorldModel	0.86 / 0.36	0.97	1.42 / 4.41	–	0.001 / 1.10	198
	stop-gradient	0.92 / 0.79	1.00	11.98 / 16.85	–	0.012 / 0.97	301
	EMA	0.94 / 0.78	1.00	16.65 / 18.65	–	0.027 / 1.01	318
	EMA, no SIGReg	0.11 / 0.10	0.42	18.42 / 20.95	–	0.019 / (35.6)	252
per-frame noise, still	LeWorldModel	–	–	0.27 / 4.46	0.42	0.0007 / 1.26	250
	stop-gradient	–	–	–	13.70	0.988 / 1.02	474
	EMA	–	–	20.62 / 23.58	13.73	0.969 / 1.05	475
	EMA, no SIGReg	–	–	0.25 / 0.31	0.46	0.0004 / (47.7)	16
per-frame noise, wind	LeWorldModel	–	–	17.54 / 48.07	14.41	0.734 / 2.29	464
	stop-gradient	–	–	–	14.59	0.985 / 1.08	475
	EMA	–	–	17.78 / 40.18	14.42	0.968 / 1.03	475
	EMA, no SIGReg	–	–	0.24 / 9.24	0.38	0.017 / (60.5)	11
per-frame noise, wind	LeWorldModel, seeds 0 / 1 / 2	–	–	17.5 / 17.6 / 17.7 (ENC)	14.41 / 14.63 / 14.48	0.734 / 0.734 / 0.733	
	stop-gradient, seeds 0 / 1 / 2	–	–	–	14.59 / 14.46 / 14.49	0.985 / 0.984 / 0.984	
	EMA, no SIGReg, seeds 0 / 1 / 2	–	–	0.24 / 0.24 / 0.24 (ENC)	0.38 / 0.47 / 0.38	0.017 / 0.020 / 0.016	
gate + persistent noise	LeWorldModel, seeds 0 / 1 / 2	0.36 / 0.50 / 0.64 (step 8)	0.97 / 0.93 / 0.99	1.42 / 1.26 / 1.28 (ENC)	–		
	EMA, no SIGReg, seeds 0 / 1 / 2	0.10 / 0.09 / 0.09 (step 8)	0.42 / 0.41 / 0.43	18.4 / 18.7 / 16.9 (ENC)	–		

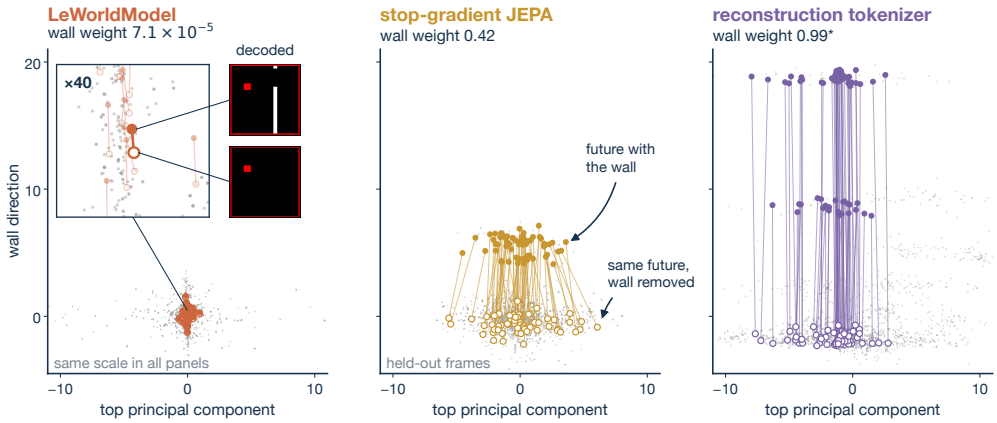


Figure 4: **LeWorldModel keeps the wall only in a direction too small to see; stopping the gradient through the target enlarges it.** Black gate, model seed 0. Each of 64 held-out contexts before the reveal is continued twice under the recorded actions and the same randomness, once as recorded and once with the wall removed; both first future frames are encoded (filled: with the wall; open: without; a line joins each pair). Vertical: the mean direction from the encoding without the wall to the encoding with it. Horizontal: the top principal component of 2,048 held-out encodings (gray) after that direction is removed. Latent units, same scale in all panels; the wall weight is defined above. Inset: LeWorldModel magnified 40 times around the first context, with its fitted decoder applied to the two encodings. Every pair steps along the wall direction, but the two groups overlap (leave-one-out AUROC 0.54). The reconstruction tokenizer encodes the last context frame together with each future.

The oracle. For each evaluation context with no wall on screen we render all 30 futures the context allows (entry after 1 to 15 steps, opening at the top or bottom) under fixed alternating left and right actions, encode them with the frozen encoder, average the encodings with the data's weights and decode the average, at future steps 1 to 20. As a control, the same is done for a wall already in view, averaging over the opening's side only. 32 contexts for seed 0 of the tokenizer and the stop-gradient JEPA and for LeWorldModel's seed 1, 16 for the others.

LeWorldModel's seed-2 decoder does not draw a faint mixture: it turns the averaged latent into one clean wall at one of the possible positions, as a classifier would. The latent form of Proposition 1

Table 7: **The decoded average of an arriving wall.** Largest decoded wall intensity over positions, steps 1 to 20; control: a wall in view averaged over its opening's side, at steps 1 / 10 / 20.

encoder (seeds)	arriving wall, average over entry time and side	wall in view, average over side
reconstruction tokenizer (0, 1, 2)	0.00 at every step, all three	1.00 / 0.99–1.00 / 0.99
stop-gradient JEPA (0, 1, 2)	0.00 at every step, all three	1.00 / 0.98–1.00 / 0.97–0.99
LeWorldModel (0, 1)	0.00 at every step, both	1.00 / 0.84–0.95 / 0.69–0.80
LeWorldModel (2)	0.81–1.00 at steps 3–5 and 19–20, 0.17–0.26 at 6–17	1.00 / 0.99 / 0.97
pixel average, no encoder	0.00 at every step	–

therefore needs a decoder that behaves like an average near the data. This does not help LeWorldModel, whose encoder has already deleted the wall (Appendix D).

Trained predictors. Open loop from 256 uniform training starts under the real actions, the stop-gradient JEPA draws a wall on 0.75 to 0.89 of the steps with a wall already in view at the start, and on 0.000 to 0.005 of the steps with a wall that entered after the start; the flow model on 0.64 to 0.81 of the latter. The stop-gradient JEPA also draws none of 46 later walls, whose timing the history shows; that is the predictor's approximation error, not averaging, since a squared-loss predictor on the tokenizer draws all 46.

The flow model's episode clock. The flow recipe draws half of its training windows from an episode start and gives the first frame of such a window the incoming action 0; the evaluation packet does the same. The flow model reads this as an episode clock. Rolled open loop for 24 steps from wall-free contexts (4 sampling draws $\times$ 128 contexts), its first drawn wall lands exactly 15 steps after the context, the episode-start timing, in 59% to 73% (fit 2) and 15% to 35% (fit 0) of rollouts when the incoming action is 0, at every context age tested, and in at most 1% otherwise. From uniform contexts with the archive's action, the first drawn wall lands anywhere in steps 2 to 16 (10.0 ± 3.9, 12.7 ± 3.3 and 11.0 ± 4.0 for fits 0 to 2) whatever the true entry time (4, 8 or 12 steps): there the model samples the arrival. This does not enter Test 2: the agents' imagined episodes start from uniform archive contexts with the archive's actions, and the world model is not used in the real evaluation. It does bound Proposition 1 for this recipe: a squared-loss predictor trained with the flow model's window sampler would see $q \approx 0.5$ at episode starts.

F Encoders crossed with predictors in the black gate

Cells. Reconstruction tokenizer with flow sampler: the flow recipe's models 0 to 2. Tokenizer with squared loss: the flow recipe's predictor architecture trained to regress the next latent on the frozen tokenizer, heads trained jointly. Stop-gradient JEPA with its own predictor: the refits of Appendix D, with heads refitted afterwards. Stop-gradient JEPA and LeWorldModel with the flow sampler: the flow recipe's predictor trained on the frozen JEPA latents with jointly trained heads, decoded by the JEPA's decoder (the predictor swap of Appendix S). Each predictor is trained on the frozen encoder. Agent seed k runs on model k, except LeWorldModel's own predictor, which is the model of Table 2 with agent seeds 0 to 2. Test 2 is the registered driver, unchanged except for the model it loads.

Every run. Real survival, walls passed per episode in brackets. Tokenizer with flow sampler, models 0 / 1 / 2: 0.988 (3.97), 0.977 (3.93), 0.742 (3.22); model 0 with agent seeds 1 and 2: 0.941, 0.977. Tokenizer with squared loss: models 0 / 1 / 2 0.297 (2.45), 0.191 (1.28), 0.121 (1.61); model 0 with agent seeds 1 and 2: 0.605, 0.039; agent seed 0 on models 1 and 2: 0.621, 0.070; a rerun of model 0, agent seed 0: 0.035 (eight runs, mean 0.248). Stop-gradient JEPA with flow sampler: 0.984 (3.96), 0.988 (3.96), 0.977 (3.91); a rerun of model 0: 0.945. Stop-gradient JEPA with its own predictor: 0.012, 0.016, 0.000. LeWorldModel with flow sampler: 0.012 (0.53), 0.016 (0.63), 0.031 (0.73). LeWorldModel's own predictor (model 1, agent seeds 0 to 2): 0.027, 0.008, 0.027.

Decoded-real controls. The same agent trained on real transitions seen through each encoder and decoder: tokenizer 0.957, 0.516, 0.953 (model 0, agent seeds 0 to 2); stop-gradient JEPA 0.957 and 0.984 (model 0); LeWorldModel 0.965 (model 0), 0.922 (model 2), and 0.977, 0.980, 0.945 (model

1). All but one control survive at least 0.92; the exception (tokenizer, agent seed 1, 0.516) is still far above every run inside a JEPA predictor.

Where the squared-loss tokenizer cell fails. It never draws a first wall (none of about 700) and draws every later wall (46 of 46). At the reveal it draws one sharp opening, not an average, on a side at chance (0.50 to 0.52), but the side depends on the agent's actions: forcing all actions up and then down leaves it unchanged in only 52% to 56% of histories, against 85% to 94% for the samplers, whose side is set by their noise. Its agents learn to exploit this. In closed loop from 256 training starts, the imagined opening is on the cube's side in 69% to 97% of reveals over the eight policies, against 48% to 55% in the real game from the same starts and 47% to 60% in the sampler cells. Sorted by that share, real survival is 0.297, 0.621, 0.605, 0.191, 0.070, 0.121, 0.039 and 0.035 (Spearman -0.90).

Reproducibility. Runs are not bit-reproducible on the GPU: two default-mode runs of the same configuration differ in the first imagination batch, and the difference compounds through 150 updates. XLA's deterministic modes make the scatters reproducible but slow the agent's update about $1{,}600$-fold (the gradient of the policy network's overlapping max-pool becomes a serial scatter), so they cannot be used. Same-seed reruns moved survival from 0.051 to 0.941 and from 0.941 to 0.398 in the gate with per-frame noise, from 0.297 to 0.035 in the squared-loss tokenizer cell and from 0.984 to 0.945 in the stop-gradient sampler cell. We therefore read effects only beyond this spread.

G Context signal, and the flow model's long rollouts

Context signal. While the flow model imagines, every frame it generated is written back into its 32-frame context as $s\,x + (1 - s)\,\epsilon$ and labelled with level s; real context frames stay clean. The model was trained with real frames noised to $s = 0.9$, and the registered runs sample at 0.9. Only the sampler changes with s: the heads keep the recipe's 0.9, and the real evaluation never uses the model.

Table 8: **Flow agents at signal 0.8 and 0.9.** Real survival, every run (agent seeds 0 to 2; in the noisy gates agent seed k on model k). The persistent gate at 0.8 is from the same driver.

world	signal 0.8 (runs; mean)	signal 0.9 (registered runs; reruns)
per-frame noise, still	1.000 / 0.977 / 0.992; 0.990	0.953 / 0.969 / 0.992
per-frame noise, wind	0.637 / 0.645 / 0.902; 0.728	0.945 / 0.129 / 0.652; 0.734
persistent noise, wind	0.957 / 0.766 / 0.770; 0.831	0.891 / 0.207 / 0.137
black gate	0.969 / 0.902 / 0.984; 0.952	0.988 / 0.941 / 0.977
gate + per-frame noise	0.953 / 0.719 / 0.051, 0.941; 0.666	0.980 / 0.941 / 0.730; 0.398, 0.906
gate + persistent noise	0.996 / 0.914 / 0.977; 0.962	0.000 / 0.039 / 0.113; 0.000

Long rollouts in the persistent gate. Open loop under the actions of 64 real episodes, the imagined background's correlation with the episode's texture falls in every paper fit, from 0.55 to 0.57 at steps 1 to 8 to 0.07 to 0.11 at steps 81 to 121 at signal 0.9 (0.16 to 0.36 at 0.8). At 0.9 the cube multiplies in two fits and vanishes in the third. Pinning the first real frames in the context changes nothing, and the break-up starts before the 32-frame window fills. Four times the predictor's updates keep the texture correlated through step 121 (0.42 and 0.39) and stop the multiplication in two of three refits; the third refit's rollouts turn into a lattice of red and cyan blobs within nine steps, although its training loss matches the others'. Agents inside the refits at signal 0.9 survive 0.8% and 1.6% (refit 0, two agent seeds; 1.6% at 0.8), 89.8% (refit 1) and 0% (refit 2). Refit 0 keeps the texture best, but only 22% of its imagined frames show a wall (80% in refit 1), where 88% of real steps have one, so its agent never practises passing walls. A long, high-entropy nuisance is the one place where the flow model paid for modelling what the task does not need. For the agents, signal 0.8 repaired the failure on all three paper models; four times the training repaired it on one of three refits.

H CRAFTAX: PLANNING, REAL-TRANSITION CONTROL AND HEADS

Planning. Models: the three Craftax fits of each family in Table 1. Labelled frames: 1,024 environments $\times$ 1,500 steps of uniformly random play, labels from the true game state (holding wood, holding two wood, standing next to a crafting table, holding a wooden pickaxe). Each model encodes the frames with its own encoder; a linear and a two-layer probe are fitted per label, and the planner uses the two-layer probe unless its mean AUROC on held-out environments (never used for fitting) is below the linear one's. Planner, identical for both families: cross-entropy method over action sequences, population 64, 8 elites, 3 iterations, horizon 5, first action executed, replanning at every real step; objective $\max_{h\le 5} \sum_j w_j\, \sigma(\mathrm{probe}_j(\hat{z}_{t+h}))$ with $w = (1, 1, 3, 6)$. The JEPA imagines with its one-step predictor over its last eight latents; the flow model samples with 4 function evaluations over a sliding window of its last eight frames. Evaluation: the paper's 256 evaluation episodes per model, 150 real steps.

Table 9: **Head-free planning in Craftax.** Percentage of 256 episodes per model that collect wood, place a table and craft a wooden pickaxe within 150 steps, and achievements unlocked per episode.

planner	model	wood	table	pickaxe	achievements
random actions	–	40	6	0	2.18
flow	0 / 1 / 2	95 / 98 / 98	89 / 93 / 94	68 / 66 / 70	4.08 / 4.14 / 4.24
JEPA	0 / 1 / 2	40 / 51 / 79	12 / 18 / 48	1 / 0 / 3	1.54 / 2.15 / 2.97

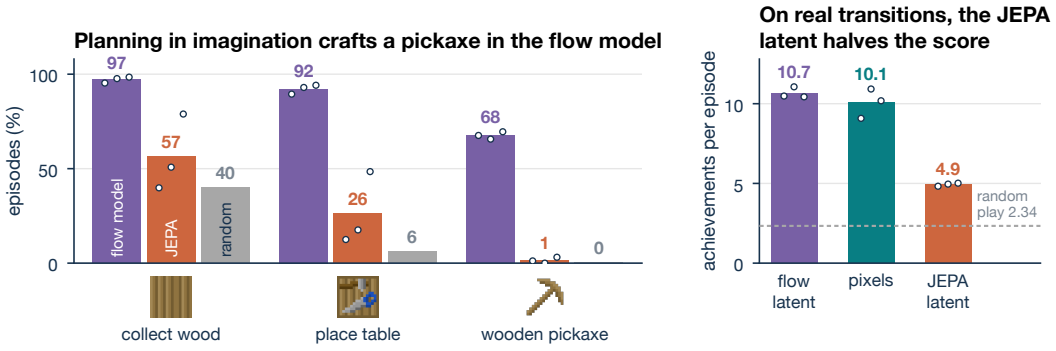


Figure 5: **Craftax: planning in the flow model's imagination crafts a wooden pickaxe; in the JEPA's it almost never does.** Left: share of 256 test episodes per model that collect wood, place a crafting table and craft a wooden pickaxe within 150 steps, when the same probes score imagined latents and the same cross-entropy-method planner searches each model's imagination (no reward head); bars: mean over three models per family, dots: the models; random actions: one run (its pickaxe bar is 1 of 256 episodes). Right: achievements per episode of one latent-input policy trained with PPO on two million real transitions with the true reward, on each frozen encoder's latent or on pixels; dots: agent seeds; dashed: random play. The two panels use different episode lengths and are not comparable.

On 64 episodes per model, a longer horizon (eight steps, replanning every second step) gives the same picture (pickaxe $71 \pm 4\%$ against $2 \pm 3\%$), and so does LeWorldModel's own goal objective, the latent distance to frames just after the next unlock (pickaxe: flow 8%, JEPA 0%). With the two-layer probe the JEPA collects more wood and places more tables than with the linear one on every seed, and with either probe its pickaxe rate stays at 0% to 5%.

What the planner's score sees. From real random-play states in which an action can work (DO in front of a tree; PLACE_TABLE with two wood on a free cell; MAKE_WOOD_PICKAXE next to a table with wood) or cannot, each model imagines the next latent after the action and after a no-op from the same eight-frame context. For the pickaxe, the JEPA's imagined latent change is 0.3% to 1.4% of the real one and its imagined score gain 0.02 to 0.15 (real 1.1 to 2.5), about as much as when crafting cannot work (0.03 to 0.08); the flow model imagines 84% to 93% of the change and

3.8 to 4.6 of a real gain of 5.0 to 5.3, and nothing when crafting cannot work. For the table, the JEPA imagines 10% to 14% of the change and cannot tell one wood from two, so its planners press PLACE_TABLE on 16% to 27% of steps (flow and random play 6%). Decoded frames show the same failure (Figure 6).

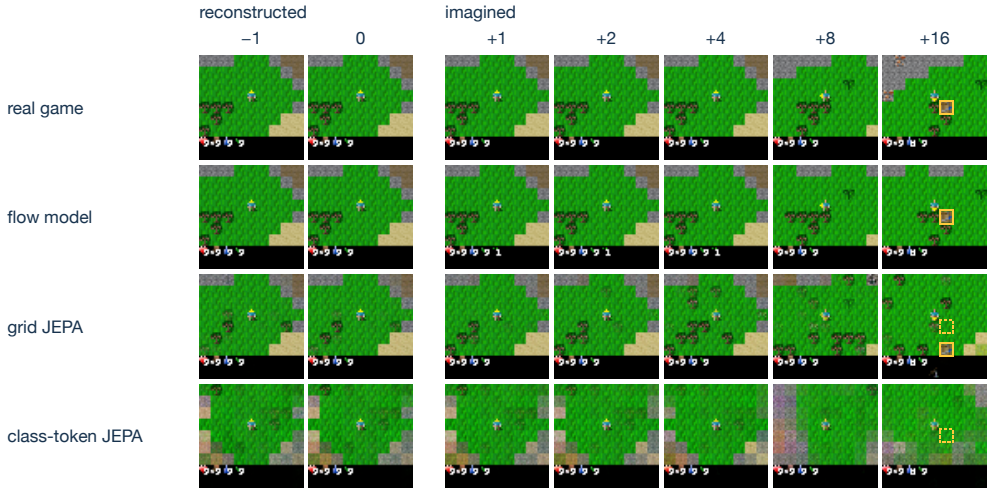


Figure 6: **Craftax: the flow model imagines the crafting table the agent places; the JEPAs do not.** The first held-out episode of Craftax. Every model receives the same eight recorded frames and the next sixteen recorded actions, in which the agent collects wood and places a crafting table at step 14. Left: the last two context frames after each model's encoder and decoder (for the JEPAs, the decoder fitted after training). Right: the frames each model imagines; one sample of the flow model. Solid box: a block that matches the game's crafting-table sprite. Dashed box: where the real table stands, in a frame whose model drew no table there. The grid JEPA draws a table two blocks below the real one; the class-token JEPA draws none.

Real-transition control. The latent-input policy of the imagination runs, trained with PPO on two million real Craftax-Classic transitions with the true reward and termination (no world model, no heads), on the frozen encoder's latent of the real frame; plain pixel PPO for reference. Achievements per episode over 256 evaluation episodes, agent seeds 0 to 2: flow latent 10.48, 11.06, 10.43; pixels 9.08, 10.92, 10.18; JEPA latent 4.82, 4.96, 5.02 (random play 2.34). The JEPA-latent agents collect wood in 83% to 88% of episodes but place a table in 58% to 61% and craft a pickaxe in 3 to 21 of 256; the flow-latent agents in 100%, 98% to 99% and 203 to 229 of 256.

Heads fitted after training. Reward and termination heads with a recurrent memory of the episode, fitted after training on the frozen latents of either model, detect real reward events better on the flow latent than on the JEPA's (recall 0.62 against 0.30 at precision 0.92 and 0.82). Agents trained inside either model with these heads farm them: after five million imagined steps they unlock 2.33 achievements per episode inside the flow model (random play 2.34) and 0.76 inside the JEPA, while their imagined reward per step ends six to ten times above the true rate (the flow agents press DO on 61% to 83% of steps, which the heads pay 0.48 to 0.62 and the game 0.003 to 0.013). The flow model with its jointly trained heads reaches 3.54, and its agents are exploited too, by repeated drinking. Post-hoc heads are therefore a limitation of both models as reinforcement-learning environments, not a JEPA-versus-flow result, and the pickaxe-crafting flow agents of Appendix W rest on jointly trained heads.

I Capacity: distractors and a cue

World. The black gate with two added 10×10 blocks, green and blue, that move at constant velocity and bounce off the border without touching reward or termination, and a 3×3 yellow mark at the top or bottom right that shows the side of the opening from the wall's entry, seven steps before

the opening is drawn. A cube in the centre needs five moves to reach an opening but sees it only three moves before contact, so the mark decides survival. Data: 500,000 transitions of uniformly random play.

Objectives and budget. Encoders with K latent numbers carry Gaussian channel noise ($\sigma = 0.5$) that bounds their rate. Reconstruction; reconstruction plus reward and termination heads at weight 1 (Dreamer's recipe); reconstruction weighted by the gradient of a critic's value with respect to the frame (value-gradient weighting, weight 10^3 or 10^4); LeWorldModel; and the paper's grid JEPA at 512 numbers. Read-outs on held-out frames: cube and blocks R^2, mark AUROC.

Table 10: **What a rate-limited latent keeps.** Seed 0 at $K = 512$; seeds 0 / 1 / 2 at $K = 16$. Pixel error is the reconstruction's summed squared error.

K	objective	pixel error	cube R^2	mark AUROC	green block R^2	blue block R^2
512	reconstruction	1	.90	1.00	.89	.96
512	reconstruction + heads	1	.90	1.00	.86	.96
512	LeWorldModel (no rate bound)	–	.00	.56	.46	.45
512	grid JEPA (no rate bound)	–	−.13	.57	.56	.67
16	reconstruction	147 / 120 / 146	.03 / .03 / .04	.52 / .52 / .51	−.01 / .57 / −.01	.92 / .86 / .92
16	reconstruction + heads	118 / 120 / 118	.03 / .03 / .03	.52 / .52 / .52	.59 / .55 / .59	.83 / .86 / .83
16	value-gradient weight 10^3	147 / 146 / 147	.04 / .04 / .04	1.00 / 1.00 / 1.00	−.01 / −.01 / −.01	.90 / .89 / .92
16	value-gradient weight 10^4	170 / 216 / 177	.73 / .91 / .79	1.00 / 1.00 / 1.00	−.01 / .00 / −.01	.74 / .00 / .69
16	LeWorldModel (no rate bound)	–	.03 / .03 / .03	.52 / .52 / .56	.87 / .91 / .89	.92 / .93 / .93

Without the blocks, reconstruction at $K = 16$ keeps the cube on two of three seeds and the mark on all three, so the blocks are what crowd them out; LeWorldModel at $K = 16$ keeps the cube there and still loses the mark (AUROC 0.51 to 0.55), the one feature in this world whose value is not predictable from the previous frames.

J PER-SEED TABLES BEHIND EVERY CELL

Both scorecards are generated from saved artifacts by the scripts released with the code; the tables below list every fit and every policy run that enters them, so each cell of Table 1 and Table 2 can be opened. No row is excluded and no seed or checkpoint is selected: every fit that completed its registered budget appears, and cells with fewer than three entries print their n in the main tables.

What each imagination paid. Those last two columns say what each policy was rewarded for, beside what it then did in the real world. The flow model is roughly calibrated where it works, imagining 0.88 of episodes alive on the black gate against 0.97 real, and optimistic where it does not, up to 0.71 imagined against 0.11 real under a persistent background. The JEPA's currency is wrong in both directions: in the per-frame wind world its heads end at 0.04 alive with an imagined death probability of 0.89 per step, so a dream episode lasts two steps and the gradient sees almost no consequence of an action, while in the still sibling they pay an imagined return of 8.0, 2.5 and 9.5 where the real world's best is 0. Those policies are not solving the task; they are collecting the model's mistakes, which is the same reward farming the Craftax rows show.

Table 11: **Scorecard 1, per fit.** Both pre-registered read-outs for every cell: the latent probe on the frozen encoding and the pixel probe on decoded frames, on encodings of held-out observations (ENC) and at the eighth predicted step (PRED), with the exact step-8 floor, the persistence reference, and the gate quantities where they apply. The main table reports the better of the two probes; both are here for every seed. Every fit that completed its registered budget appears, in seed order, with no seed or checkpoint selection.

world	arm	seed	ENC	PRED	floor	latent_enc	latent_pred	pixel_enc	pixel_pred	wall_enc	wall_pred1	auroc_pred	updates
black, still	class-token JEPA	0	0.09	18.41	0.00	0.09	45.30	17.01	18.41	–	–	–	23750
black, still	class-token JEPA	1	0.09	18.40	0.00	0.09	44.88	17.00	18.40	–	–	–	23750
black, still	class-token JEPA	2	0.09	18.40	0.00	0.09	45.61	17.01	18.40	–	–	–	23750
black, still	grid JEPA	0	0.10	3.42	0.00	0.10	3.93	0.27	3.42	–	–	–	8500
black, still	grid JEPA	1	0.10	3.57	0.00	0.10	3.89	0.27	3.57	–	–	–	8500
black, still	grid JEPA	2	0.09	3.05	0.00	0.09	3.57	0.27	3.05	–	–	–	8500
black, still	flow	0	0.13	0.13	0.00	0.13	0.13	0.23	0.24	–	–	–	–
black, still	flow	1	0.12	0.11	0.00	0.12	0.11	0.23	0.24	–	–	–	–
black, still	flow	2	0.12	0.12	0.00	0.12	0.12	0.23	0.24	–	–	–	–
black, wind	class-token JEPA	0	0.00	8.85	7.90	0.00	9.14	0.22	8.85	–	–	–	23750
black, wind	class-token JEPA	1	0.00	8.79	7.90	0.00	9.11	0.22	8.79	–	–	–	23750
black, wind	class-token JEPA	2	0.00	8.72	7.90	0.00	8.82	0.22	8.72	–	–	–	23750
black, wind	grid JEPA	0	0.13	11.74	7.90	0.13	11.74	0.26	12.10	–	–	–	8500
black, wind	grid JEPA	1	0.14	11.49	7.90	0.14	11.52	0.29	11.49	–	–	–	8500
black, wind	grid JEPA	2	0.13	11.30	7.90	0.13	11.30	0.26	11.37	–	–	–	8500
black, wind	flow	0	0.13	11.22	11.76	0.13	11.28	0.22	11.22	–	–	–	–
black, wind	flow	1	0.14	11.27	11.76	0.14	11.30	0.22	11.27	–	–	–	–
black, wind	flow	2	0.13	10.89	11.76	0.13	10.93	0.22	10.89	–	–	–	–
per-frame noise, still	class-token JEPA	0	20.80	20.29	0.00	23.96	20.29	20.80	22.99	–	–	–	24000
per-frame noise, still	class-token JEPA	1	20.76	20.18	0.00	23.91	20.18	20.76	21.52	–	–	–	24000
per-frame noise, still	class-token JEPA	2	20.82	19.13	0.00	23.98	19.86	20.82	19.13	–	–	–	24000
per-frame noise, still	grid JEPA	0	0.12	4.27	0.00	0.12	4.27	0.27	4.46	–	–	–	8500
per-frame noise, still	grid JEPA	1	0.13	6.09	0.00	0.13	6.15	0.27	6.09	–	–	–	8500
per-frame noise, still	grid JEPA	2	0.10	4.11	0.00	0.10	4.11	0.27	4.26	–	–	–	8500
per-frame noise, still	flow	0	0.26	0.39	0.00	3.54	3.20	0.26	0.39	–	–	–	–
per-frame noise, still	flow	1	0.26	0.31	0.00	2.91	2.68	0.26	0.31	–	–	–	–
per-frame noise, still	flow	2	0.26	0.30	0.00	3.14	2.55	0.26	0.30	–	–	–	–
per-frame noise, wind	class-token JEPA	0	17.95	19.57	7.90	25.29	19.57	17.95	22.37	–	–	–	24000
per-frame noise, wind	class-token JEPA	1	18.09	19.74	7.90	25.22	19.74	18.09	22.94	–	–	–	24000
per-frame noise, wind	class-token JEPA	2	18.09	19.53	7.90	25.36	19.53	18.09	20.54	–	–	–	24000
per-frame noise, wind	grid JEPA	0	17.54	19.70	7.90	21.35	19.70	17.54	48.07	–	–	–	8500
per-frame noise, wind	grid JEPA	1	17.63	20.03	7.90	21.37	20.03	17.63	33.42	–	–	–	8500
per-frame noise, wind	grid JEPA	2	17.71	21.44	7.90	21.26	21.44	17.71	24.73	–	–	–	8500
per-frame noise, wind	flow	0	0.25	12.25	11.76	3.68	13.96	0.25	12.25	–	–	–	–
per-frame noise, wind	flow	1	0.25	12.74	11.76	2.44	13.77	0.25	12.74	–	–	–	–
per-frame noise, wind	flow	2	0.25	12.63	11.76	2.87	14.10	0.25	12.63	–	–	–	–
persistent noise, still	class-token JEPA	0	18.57	19.61	0.00	21.32	21.28	18.57	19.61	–	–	–	24000
persistent noise, still	class-token JEPA	1	18.34	19.72	0.00	20.82	21.21	18.34	19.72	–	–	–	24000
persistent noise, still	class-token JEPA	2	19.15	20.76	0.00	21.48	20.95	19.15	20.76	–	–	–	24000
persistent noise, still	grid JEPA	0	0.71	1.78	0.00	0.71	1.78	1.44	3.14	–	–	–	8500
persistent noise, still	grid JEPA	1	0.84	1.64	0.00	0.84	1.64	1.98	3.71	–	–	–	8500
persistent noise, still	grid JEPA	2	0.73	2.07	0.00	0.73	2.07	1.63	3.85	–	–	–	8500
persistent noise, still	flow	0	0.40	1.54	0.00	3.38	3.67	0.40	1.54	–	–	–	–
persistent noise, still	flow	1	0.74	2.66	0.00	12.36	16.02	0.74	2.66	–	–	–	–
persistent noise, still	flow	2	0.61	1.16	0.00	6.16	5.49	0.61	1.16	–	–	–	–
persistent noise, wind	class-token JEPA	0	15.98	19.46	7.90	22.97	27.42	15.98	19.46	–	–	–	23750
persistent noise, wind	class-token JEPA	1	15.81	19.25	7.90	22.75	25.97	15.81	19.25	–	–	–	23750
persistent noise, wind	class-token JEPA	2	16.26	19.75	7.90	22.75	25.08	16.26	19.75	–	–	–	23750
persistent noise, wind	grid JEPA	0	0.72	11.55	7.90	0.72	11.55	1.22	13.01	–	–	–	8500
persistent noise, wind	grid JEPA	1	0.67	10.77	7.90	0.67	10.77	1.09	11.08	–	–	–	8500
persistent noise, wind	grid JEPA	2	0.62	10.53	7.90	0.62	10.53	1.20	10.96	–	–	–	8500
persistent noise, wind	flow	0	0.25	13.98	11.76	2.51	14.71	0.25	13.98	–	–	–	–
persistent noise, wind	flow	1	0.25	13.14	11.76	2.33	13.49	0.25	13.14	–	–	–	–
persistent noise, wind	flow	2	0.25	13.23	11.76	2.92	14.24	0.25	13.23	–	–	–	–
gate	class-token JEPA	0	7.60	17.21	0.00	7.60	42.52	15.50	17.21	0.33	0.65	0.50	23500
gate	class-token JEPA	1	6.18	17.22	0.00	6.18	42.54	15.52	17.22	1.00	1.00	0.49	23500
gate	class-token JEPA	2	6.42	17.21	0.00	6.42	39.84	15.51	17.21	0.32	0.64	0.50	23500
gate	grid JEPA	0	0.10	3.66	0.00	0.10	3.82	0.23	3.66	1.00	0.00	0.50	8500
gate	grid JEPA	1	0.10	3.45	0.00	0.10	3.55	0.27	3.45	0.99	0.00	0.50	8500
gate	grid JEPA	2	0.10	2.61	0.00	0.10	2.61	0.23	2.80	0.99	0.53	0.56	8500
gate	flow	0	0.18	0.48	0.00	0.18	0.48	0.23	0.64	1.00	1.00	1.00	–
gate	flow	1	0.17	0.34	0.00	0.17	0.34	0.23	0.45	1.00	1.00	1.00	–
gate	flow	2	0.17	0.32	0.00	0.17	0.32	0.23	0.42	1.00	1.00	1.00	–
gate + per-frame noise	class-token JEPA	0	20.17	17.23	0.00	20.20	20.59	20.17	17.23	0.44	1.00	0.50	24000
gate + per-frame noise	class-token JEPA	1	20.04	21.03	0.00	20.27	21.03	20.04	24.18	0.44	1.00	0.50	24000
gate + per-frame noise	class-token JEPA	2	20.27	22.60	0.00	20.28	22.71	20.27	22.60	0.44	1.00	0.50	24000
gate + per-frame noise	grid JEPA	0	0.38	5.01	0.00	0.68	5.01	0.38	5.75	1.00	0.94	0.64	8500
gate + per-frame noise	grid JEPA	1	12.59	23.02	0.00	13.57	23.02	12.59	25.50	1.00	1.00	0.95	8500
gate + per-frame noise	grid JEPA	2	1.24	15.47	0.00	6.26	15.47	1.24	21.76	1.00	0.78	0.50	8500
gate + per-frame noise	flow	0	0.26	0.48	0.00	2.63	2.62	0.26	0.48	1.00	1.00	1.00	–
gate + per-frame noise	flow	1	0.26	0.38	0.00	2.62	2.55	0.26	0.38	1.00	1.00	1.00	–
gate + per-frame noise	flow	2	0.27	0.42	0.00	2.89	2.96	0.27	0.42	1.00	1.00	1.00	–
gate + persistent noise	class-token JEPA	0	18.79	21.08	0.00	20.53	21.08	18.79	21.28	0.22	0.23	0.52	24000
gate + persistent noise	class-token JEPA	1	18.30	20.93	0.00	20.78	25.03	18.30	20.93	0.22	0.22	0.52	24000
gate + persistent noise	class-token JEPA	2	18.43	21.01	0.00	20.29	23.14	18.43	21.01	0.22	0.24	0.52	24000
gate + persistent noise	grid JEPA	0	1.02	3.10	0.00	1.02	3.10	1.42	4.41	0.83	0.86	0.99	8500
gate + persistent noise	grid JEPA	1	1.10	3.41	0.00	1.10	3.41	1.26	4.44	0.83	0.91	0.98	8500
gate + persistent noise	grid JEPA	2	1.15	3.60	0.00	1.15	3.60	1.28	4.65	0.82	0.92	1.00	8500
gate + persistent noise	flow	0	0.27	1.22	0.00	2.88	5.23	0.27	1.22	1.00	1.00	1.00	–
gate + persistent noise	flow	1	0.26	1.24	0.00	2.93	6.74	0.26	1.24	1.00	0.97	1.00	–
gate + persistent noise	flow	2	0.26	0.62	0.00	2.66	3.36	0.26	0.62	1.00	1.00	1.00	–

world	arm	seed	ENC	PRED	persistence	AF	updates
Craftax-Classic	class-token JEPA	0	0.01843	0.02428	0.01333	0.702	24000
Craftax-Classic	class-token JEPA	1	0.01917	0.02413	0.01333	0.707	24000
Craftax-Classic	class-token JEPA	2	0.01821	0.02360	0.01333	0.716	24000
Craftax-Classic	grid JEPA	0	0.00860	0.01137	0.01333	0.885	8500
Craftax-Classic	grid JEPA	1	0.00789	0.01125	0.01333	0.886	8500
Craftax-Classic	grid JEPA	2	0.00779	0.01160	0.01333	0.878	8500
Craftax-Classic	flow	0	0.00007	0.00124	0.01333	0.931	9000+16000
Craftax-Classic	flow	1	0.00007	0.00122	0.01333	0.942	9000+16000
Craftax-Classic	flow	2	0.00008	0.00136	0.01333	0.921	9000+16000

Table 12: **The gate worlds' wall and gap quantities.** Decoded intensity of the wall's shared solid part on real encodings and at the model's own first and eighth predicted step, and the AUROC for the revealed opening's side on real encodings and on predicted latents; mean ± sample standard deviation over the same fits as Table 1, every fit entering with no seed or checkpoint selection. The quantities on real encodings set the **red** and **green** conditions of the gate rows. Intensity on decoded predictions saturates when a decoder renders a bright field, so AUROC is the discriminative quantity of the pair.

world	recipe	wall, real	wall, pred 1	wall, pred 8	AUROC, real	AUROC, pred
gate	class-token JEPA	0.55±0.39	0.76±0.20	0.26±0.44	0.55±0.09	0.50±0.01
gate	grid JEPA	0.99±0.00	0.18±0.31	0.00±0.01	1.00±0.00	0.52±0.03
gate	flow	1.00±0.00	1.00±0.00	0.96±0.02	1.00±0.00	1.00±0.00
gate + per-frame noise	class-token JEPA	0.44±0.00	1.00±0.00	0.00±0.00	0.50±0.00	0.50±0.00
gate + per-frame noise	grid JEPA	1.00±0.00	0.91±0.12	0.54±0.43	1.00±0.00	0.69±0.23
gate + per-frame noise	flow	1.00±0.00	1.00±0.00	0.94±0.02	1.00±0.00	1.00±0.00
gate + persistent noise	class-token JEPA	0.22±0.00	0.23±0.01	0.22±0.00	0.52±0.00	0.52±0.00
gate + persistent noise	grid JEPA	0.83±0.00	0.90±0.03	0.50±0.14	1.00±0.00	0.99±0.01
gate + persistent noise	flow	1.00±0.00	0.99±0.02	0.89±0.14	1.00±0.00	1.00±0.00

Table 13: **Scorecard 2, per policy run.** Survival, walls passed, control length and return of every policy run on the world's fixed 256-episode packet, with the model fit and policy seed it used and the number of PPO updates in its registered budget. State-blind rules are listed with the same evaluator; their control length is the quantity the noisy gate worlds are compared on, since every rule and policy there scores zero survival. The last two columns are the imagined side of the same run at its final update: the fraction of the model's own rollouts still alive and the return its own heads paid, which is what the policy was optimizing.

world	arm	model seed	policy seed	updates	survival	walls passed	control length	return	imagined alive	imagined return
per-frame noise, still	grid JEPA, imagined	0	0	50	0.965	0.00	119.9	-0.035	0.94	8.01
per-frame noise, still	grid JEPA, imagined	0	1	50	0.938	0.00	117.8	-0.062	0.82	2.45
per-frame noise, still	grid JEPA, imagined	0	2	50	0.633	0.00	90.3	-0.367	0.94	9.51
per-frame noise, still	flow, imagined	0	0	50	0.953	0.00	119.2	-0.047	0.91	-0.01
per-frame noise, still	flow, imagined	0	1	50	0.969	0.00	119.5	-0.031	0.89	-0.03
per-frame noise, still	flow, imagined	0	2	50	0.992	0.00	120.6	-0.008	0.87	-0.01
per-frame noise, still	grid JEPA decoded, real	0	0	50	1.000	0.00	121.0	0.000	0.91	0.00
per-frame noise, still	grid JEPA decoded, real	0	1	50	1.000	0.00	121.0	0.000	0.89	0.00
per-frame noise, still	grid JEPA decoded, real	0	2	50	0.996	0.00	120.7	-0.004	0.90	-0.00
per-frame noise, still	flow decoded, real	0	0	50	1.000	0.00	121.0	0.000	0.91	0.00
per-frame noise, still	flow decoded, real	0	1	50	0.992	0.00	120.8	-0.008	0.89	0.00
per-frame noise, still	flow decoded, real	0	2	50	1.000	0.00	121.0	0.000	0.90	0.00
per-frame noise, still	blind: random	–	–	–	0.238	–	74.0	–	–	–
per-frame noise, still	blind: alternate	–	–	–	1.000	–	121.0	–	–	–
per-frame noise, wind	grid JEPA, imagined	0	0	50	0.000	0.00	10.2	-1.000	0.04	-1.54
per-frame noise, wind	grid JEPA, imagined	0	1	50	0.000	0.00	11.4	-1.000	0.04	-1.53
per-frame noise, wind	grid JEPA, imagined	0	2	50	0.000	0.00	16.0	-1.000	0.04	-1.53
per-frame noise, wind	flow, imagined	0	0	50	0.945	0.00	118.1	-0.055	0.98	-0.03
per-frame noise, wind	flow, imagined	0	1	50	0.129	0.00	53.9	-0.871	0.78	-0.34
per-frame noise, wind	flow, imagined	0	2	50	0.652	0.00	102.2	-0.348	0.95	-0.10
per-frame noise, wind	grid JEPA decoded, real	0	0	50	0.000	0.00	16.8	-1.000	0.26	-0.99
per-frame noise, wind	grid JEPA decoded, real	0	1	50	0.000	0.00	16.8	-1.000	0.28	-0.97
per-frame noise, wind	grid JEPA decoded, real	0	2	50	0.000	0.00	16.9	-1.000	0.28	-0.98
per-frame noise, wind	flow decoded, real	0	0	50	0.844	0.00	112.5	-0.156	0.96	-0.08
per-frame noise, wind	flow decoded, real	0	1	50	0.668	0.00	102.2	-0.332	0.94	-0.14
per-frame noise, wind	flow decoded, real	0	2	50	0.652	0.00	101.0	-0.348	0.92	-0.20
per-frame noise, wind	blind: random	–	–	–	0.000	–	17.4	–	–	–
per-frame noise, wind	blind: alternate	–	–	–	0.000	–	18.2	–	–	–
persistent noise, wind	grid JEPA, imagined	0	0	50	0.027	0.00	32.6	-0.973	0.56	-0.67
persistent noise, wind	grid JEPA, imagined	0	1	50	0.020	0.00	31.1	-0.980	0.55	-0.69
persistent noise, wind	grid JEPA, imagined	0	2	50	0.000	0.00	16.7	-1.000	0.45	-0.71
persistent noise, wind	flow, imagined	0	0	50	0.891	0.00	115.9	-0.109	0.98	-0.05
persistent noise, wind	flow, imagined	0	1	50	0.207	0.00	69.2	-0.793	0.80	-0.62
persistent noise, wind	flow, imagined	0	2	50	0.137	0.00	51.9	-0.863	0.53	-0.71
persistent noise, wind	grid JEPA decoded, real	0	0	50	0.934	0.00	117.3	-0.066	0.98	-0.05
persistent noise, wind	grid JEPA decoded, real	0	1	50	0.574	0.00	95.8	-0.426	0.90	-0.21
persistent noise, wind	grid JEPA decoded, real	0	2	50	0.770	0.00	110.2	-0.230	0.96	-0.09
persistent noise, wind	flow decoded, real	0	0	50	0.871	0.00	113.7	-0.129	0.95	-0.10
persistent noise, wind	flow decoded, real	0	1	50	0.809	0.00	110.4	-0.191	0.95	-0.11
persistent noise, wind	flow decoded, real	0	2	50	0.891	0.00	115.5	-0.109	0.97	-0.06
persistent noise, wind	blind: random	–	–	–	0.000	–	17.4	–	–	–
persistent noise, wind	blind: alternate	–	–	–	0.000	–	18.2	–	–	–
gate	grid JEPA, imagined	1	0	150	0.027	0.73	46.2	-0.973	0.28	-0.10
gate	grid JEPA, imagined	1	1	150	0.008	0.23	36.2	-0.992	0.37	-0.05
gate	grid JEPA, imagined	1	2	150	0.027	0.70	44.0	-0.973	0.37	-0.06
gate	flow, imagined	0	0	150	0.988	3.97	120.3	-0.012	0.88	-0.16
gate	flow, imagined	0	1	150	0.941	3.86	117.7	-0.059	0.88	-0.15
gate	flow, imagined	0	2	150	0.977	3.93	119.5	-0.023	0.85	-0.18
gate	grid JEPA decoded, real	1	0	150	0.977	3.95	119.9	-0.023	0.85	-0.20
gate	grid JEPA decoded, real	1	1	150	0.980	3.96	120.1	-0.020	0.85	-0.18
gate	grid JEPA decoded, real	1	2	150	0.945	3.91	118.9	-0.055	0.85	-0.20
gate	flow decoded, real	0	0	150	0.957	3.88	118.2	-0.043	0.87	-0.17
gate	flow decoded, real	0	1	150	0.516	3.36	103.8	-0.484	0.80	-0.24
gate	flow decoded, real	0	2	150	0.953	3.92	118.9	-0.047	0.83	-0.21
gate	blind: random	–	–	–	0.000	–	24.0	–	–	–
gate	blind: alternate	–	–	–	0.000	–	24.6	–	–	–
gate + per-frame noise	grid JEPA, imagined	0	0	150	0.000	0.07	33.2	-1.000	0.89	0.48
gate + per-frame noise	grid JEPA, imagined	1	1	150	0.000	0.13	22.7	-1.000	0.11	-0.43
gate + per-frame noise	grid JEPA, imagined	2	2	150	0.000	0.00	21.9	-1.000	0.77	0.40
gate + per-frame noise	flow, imagined	0	0	150	0.980	3.98	120.6	-0.020	0.89	-0.13
gate + per-frame noise	flow, imagined	1	1	150	0.941	3.77	115.5	-0.059	0.87	-0.16
gate + per-frame noise	flow, imagined	2	2	150	0.730	3.21	101.9	-0.270	0.79	-0.28
gate + per-frame noise	grid JEPA decoded, real	0	0	150	0.953	3.89	118.5	-0.047	0.87	-0.18
gate + per-frame noise	grid JEPA decoded, real	1	1	150	0.523	2.56	84.6	-0.477	0.69	-0.45
gate + per-frame noise	grid JEPA decoded, real	2	2	150	0.492	3.35	103.5	-0.508	0.81	-0.23
gate + per-frame noise	flow decoded, real	0	0	150	0.945	3.88	118.2	-0.055	0.87	-0.17
gate + per-frame noise	flow decoded, real	1	1	150	0.582	2.83	93.4	-0.418	0.76	-0.34
gate + per-frame noise	flow decoded, real	2	2	150	0.961	3.97	120.3	-0.039	0.78	-0.26
gate + per-frame noise	blind: random	–	–	–	0.000	–	24.0	–	–	–
gate + per-frame noise	blind: alternate	–	–	–	0.000	–	24.6	–	–	–
gate + persistent noise	grid JEPA, imagined	0	0	150	0.000	0.04	26.6	-1.000	0.48	-0.45
gate + persistent noise	grid JEPA, imagined	1	1	150	0.000	0.00	19.1	-1.000	0.31	-0.68
gate + persistent noise	grid JEPA, imagined	2	2	150	0.000	0.00	15.3	-1.000	0.40	-0.56
gate + persistent noise	flow, imagined	0	0	150	0.000	0.00	21.5	-1.000	0.25	-0.99
gate + persistent noise	flow, imagined	1	1	150	0.039	0.79	47.8	-0.961	0.57	-0.76
gate + persistent noise	flow, imagined	2	2	150	0.113	1.70	67.8	-0.887	0.71	-0.46
gate + persistent noise	grid JEPA decoded, real	0	0	150	0.906	3.79	116.0	-0.094	0.84	-0.21
gate + persistent noise	grid JEPA decoded, real	1	1	150	0.961	3.88	118.3	-0.039	0.83	-0.21
gate + persistent noise	grid JEPA decoded, real	2	2	150	0.945	3.91	118.8	-0.055	0.82	-0.22
gate + persistent noise	flow decoded, real	0	0	150	0.973	3.94	119.6	-0.027	0.87	-0.17
gate + persistent noise	flow decoded, real	1	1	150	0.309	3.04	95.7	-0.691	0.82	-0.26
gate + persistent noise	flow decoded, real	2	2	150	0.922	3.84	117.2	-0.078	0.84	-0.21
gate + persistent noise	blind: random	–	–	–	0.000	–	24.0	–	–	–
gate + persistent noise	blind: alternate	–	–	–	0.000	–	24.6	–	–	–

K EVERY MODEL IN EVERY WORLD

Figure 7 and Table 14 run all six models of this paper in all nine worlds, at seed 0, with four ways of turning a model into behavior. Each cell is one run; the per-seed tables of Appendix J and Table 2 give the spread over seeds where it was measured.

Models. Flow model; grid JEPA (LeWorldModel's objective on the grid encoder); class-token JEPA (LeWorldModel's published design: one 192-dimensional token, history 3); stop-gradient JEPA; EMA teacher with SIGReg; EMA teacher without SIGReg (Section 4 and Appendix D). The last three were fitted for this grid where no fit existed, with the recipe of their gate fits (8,500 steps, heads and decoder fitted afterwards on 80,000 frames). In the black gate the grid JEPA is model 1, as in Table 2. The two black cube worlds were recorded with 100,000 training transitions instead of 500,000; every model of those worlds was trained on them, and Test 2's heads are refitted on all of them.

Agent trained inside the model. Test 2's agent, protocol and budget (Appendix P), seen through each model's decoder. For the class-token JEPA the imagination keeps its own 192-dimensional latent and history of 3 frames; nothing else changes. Blind: the better of uniform random actions and left–right alternation on the same 256 test episodes.

Planners. Both planners search every action sequence of length h in the model's imagination from the last real frames and execute the first action, replanning at every real step, on 128 episodes of 128 steps from eight random warm-up steps (Appendix Q). *Planning toward a goal image* is LeWorldModel's planner: the sequence whose last imagined latent is closest to the encoding of a frame with the cube at the centre and no wall. *Planning with reward heads* scores a sequence by the discounted reward that the model's reward and termination heads imagine along it; the JEPA's planner adds its value head at the horizon, the flow model has no value head and adds nothing. The JEPA planners search $h = 3$ in cube worlds and $h = 5$ in gate worlds; the flow planner searches $h = 3$ everywhere, one sampled future per sequence at four function evaluations, because each of its steps samples the flow model (4^5 sequences would take about four hours per world). Blind: uniform random actions on the planner's episodes.

Latent-input agents. The perceptron policy of Appendix Q reads the model's latent instead of decoded frames, trained with PPO inside the frozen model (150 updates of 512 trajectories of 16 steps) or on real transitions (1.5 million environment steps), and tested on 256 episodes of its own evaluator. Blind: uniform random actions on those episodes.

	Trained inside the model									Planning to a goal image									Planning with reward heads								
	black			per-frame noise			persistent noise			black			per-frame noise			persistent noise			black			per-frame noise			persistent noise		
	still	wind	gate	still	wind	gate	still	wind	gate	still	wind	gate	still	wind	gate	still	wind	gate	still	wind	gate	still	wind	gate	still	wind	gate
blind or random	100	0	0	100	0	0	100	0	0	22	0	0	22	0	0	22	0	0	22	0	0	22	0	0	22	0	0
flow model	100	96	99	95	95	98	100	89	0	90	0	0	31	0	0	82	0	0	100	80	2	100	77	2	100	79	1
grid JEPA	93	50	3	96	0	0	8	3	0	88	1	0	92	0	0	100	77	0	100	5	0	100	0	2	85	8	0
class-token JEPA	43	88	0	0	0	0	0	0	0	97	64	0	5	0	0	80	0	0	100	60	0	9	0	0	2	0	0
stop-gradient JEPA	100	38	1	0	0	0	15	0	0	87	2	0	13	0	0	98	0	0	100	0	0	15	0	0	41	0	0
EMA teacher	100	14	0	0	0	0	2	0	0	88	1	0	15	0	0	94	0	0	100	1	0	21	0	1	38	0	0
EMA teacher, no SIGReg	100	69	4	100	73	9	0	0	0	81	44	0	79	52	0	94	0	0	100	88	2	100	84	0	45	0	0

Figure 7: **Every model in every world, seed 0: survival (%) of an agent trained inside each model and of two planners searching each model's imagination.** Columns: the nine worlds, grouped by background (black, per-frame noise, persistent noise); rows: the six models, and on top the best strategy that ignores the screen (left panel: the better of random actions and left–right alternation; planner panels: random actions on the planners' own episodes). Darker is higher. In the still worlds alternation survives every episode, so no model can beat it there.

Table 14: **The numbers of Figure 7, and the latent-input agents.** Survival (%), seed 0, one run per cell. Colour: red when a cell is not above the better blind strategy of its world by more than 5 points (alternation survives every episode of the still worlds), green at 80% or more, amber otherwise. Blind column: the blind strategy on each block's own test episodes. A dash marks a run we did not make: in the gate worlds, latent-input agents on real experience were trained only through the flow model's and the grid JEPA's latents, where they reach at most 5%.

background	dynamics	blind	flow model	grid JEPA	class-token JEPA	stop-gradient JEPA	EMA teacher	EMA teacher, no SIGReg
Agent trained inside the model								
black	still	100	100	93	43	100	100	100
	wind	0	96	50	88	38	14	69
	gate	0	99	3	0	1	0	4
per-frame noise	still	100	95	96	0	0	0	100
	wind	0	95	0	0	0	0	73
	gate	0	98	0	0	0	0	9
persistent noise	still	100	100	8	0	15	2	0
	wind	0	89	3	0	0	0	0
	gate	0	0	0	0	0	0	0
Planning toward a goal image								
black	still	22	90	88	97	87	88	81
	wind	0	0	1	64	2	1	44
	gate	0	0	0	0	0	0	0
per-frame noise	still	22	31	92	5	13	15	79
	wind	0	0	0	0	0	0	52
	gate	0	0	0	0	0	0	0
persistent noise	still	22	82	100	80	98	94	94
	wind	0	0	77	0	0	0	0
	gate	0	0	0	0	0	0	0
Planning with reward heads								
black	still	22	100	100	100	100	100	100
	wind	0	80	5	60	0	1	88
	gate	0	2	0	0	0	0	2
per-frame noise	still	22	100	100	9	15	21	100
	wind	0	77	0	0	0	0	84
	gate	0	2	2	0	0	1	0
persistent noise	still	22	100	85	2	41	38	45
	wind	0	79	8	0	0	0	0
	gate	0	1	0	0	0	0	0
Latent-input agent trained inside the model								
black	still	20	100	100	100	100	100	100
	wind	0	96	94	99	73	57	96
	gate	0	98	0	0	0	0	0
per-frame noise	still	20	99	100	0	0	0	100
	wind	0	1	0	0	0	0	98
	gate	0	0	0	0	0	0	2
persistent noise	still	20	51	2	0	0	0	1
	wind	0	1	0	0	0	0	0
	gate	0	0	0	0	0	0	0
Latent-input agent trained on real experience								
black	still	20	99	88	98	95	100	100
	wind	0	93	98	94	95	88	96
	gate	0	4	5	–	–	–	–
per-frame noise	still	20	100	91	12	21	0	100
	wind	0	0	0	0	0	0	98
	gate	0	1	3	–	–	–	–
persistent noise	still	20	13	68	13	18	12	9
	wind	0	18	5	0	0	0	0
	gate	0	2	0	–	–	–	–

L DECODED STRIPS

Every strip in the paper, Figure 1 included, shows the first eligible held-out episode or window of its world, chosen without looking at model outputs.

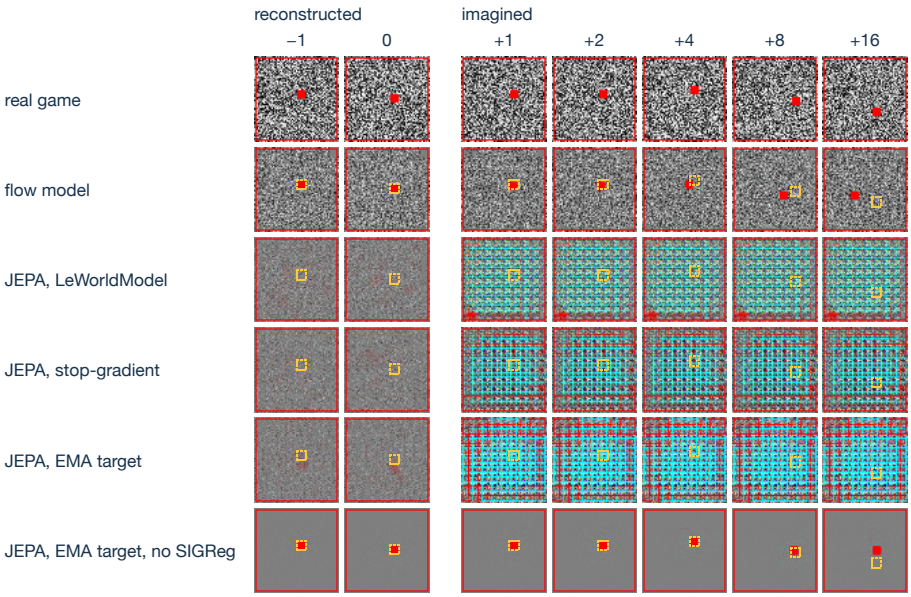


Figure 8: **With wind, every JEPA target that uses SIGReg loses the cube already in its encoder.** Per-frame noise with wind. Each row is one world model (seed 0) given the same eight real frames and the recorded actions. Left: the last two of those frames after the model's encoder and a decoder fitted after training (for the flow model, its own tokenizer). Right: the model's own predictions 1 to 16 steps ahead, decoded the same way; the flow model shows one sampled future. The JEPA rows share one encoder, predictor, data and budget and differ only in the prediction target (Appendix D). Dashed yellow square: where the cube really is.

M THE CUBE WORLDS AND THE READ-OUT PROTOCOL

Environment. The scene contains a red 6×6 object in a 63×63 RGB image with a red boundary. Four cardinal actions move the object three pixels. Reset positions are uniformly sampled from integer coordinates in $[16, 42]^2$, and initial wind is zero. In wind conditions, each wind coordinate independently increments by -1, 0, or 1 with probabilities 0.1, 0.8, and 0.1, clipped to $[-2, 2]$; the updated wind is added to the action displacement. Background pixels are uniform integers from 0 to 255; gray noise shares the same sample across RGB channels. Contact with the boundary gives reward -1 and terminates; other rewards are zero, with a 128-step cap. The gray-noise cube worlds and the gate worlds use 500,000 training transitions from 100 parallel environments; the two black cube worlds use 100,000; every world holds out 25,000 from the same collector (Appendix N).

Persistent colored background. With a persistent colored background, where only the flow model was trained, its latent loses a still cube (18.4 pixels) and locates a windblown one to 4.1 pixels.

Read-out fitting. The nonlinear latent position probe is fitted on training data and evaluated on independent held-out frames. Its spatial softmax-and-offset output is produced by two hidden layers of width 128, trained for 10,000 updates with batch 2,048 and Adam at initial learning rate 10^{-3} with cosine decay. Grid inputs are standardized using training statistics. For class-token inputs, standardization and PCA whitening are selected using the final 10% of fitting rows, then the chosen variant is refitted on the full fitting split; whitening floors singular values at 1% of the largest. The pixel probe is a separate supervised CNN read on frames from the off-budget decoder, trained with synthetic background replacement using ground-truth cube masks; both probes are diagnostics, never a policy or world-model training signal. Applying either read-out to predicted latents tests prediction and read-out extrapolation together, which is why a cell is reported as the better of the two and why the behavioural scorecard, not the probe, carries the claims about imagination. Rollout windows are sampled with replacement and overlap and exclude death or reset during the eight

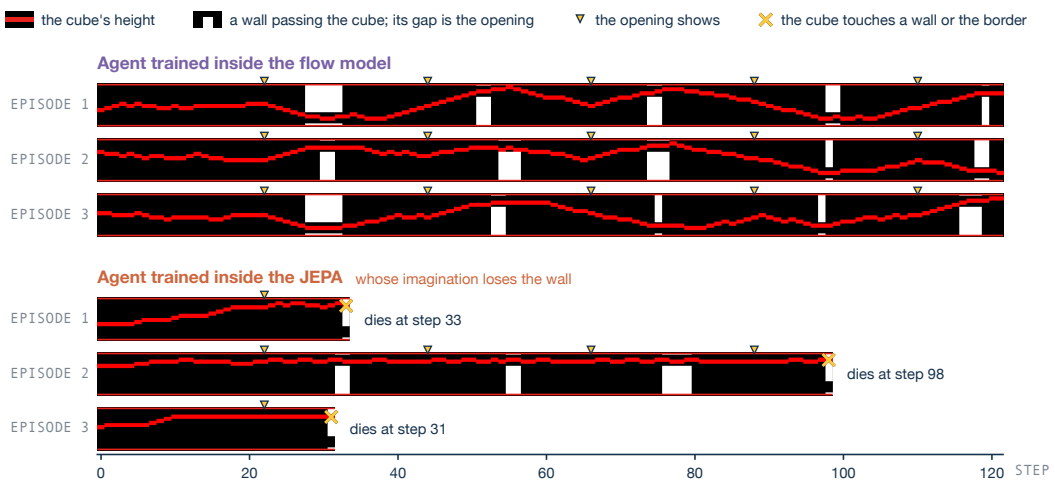


Figure 9: **On the same real test episodes, the agent trained inside the flow model goes to each opening, while the agent trained inside the JEPA dies at the first wall that opens at the bottom.** Black gate. Each strip is one test episode, one column per controlled step: the red band is the rows the cube occupies, the white bar is a wall while it overlaps the cube's columns and its gap is the opening, triangles mark when an opening shows, and a cross marks the step at which the cube touches a wall or the border; thin red lines are the border. Both agents meet the same walls, with the same openings, at the same steps. The JEPA-trained agent passes the walls that open at the top (episode 2) and dies at the first that opens at the bottom (steps 33, 98 and 31). Episodes: the first three of the 256 fixed test episodes, whose frames are stored for every agent regardless of outcome (Appendix P); agent seed 0 of each model, which survives 98.8% (flow) and 2.7% (JEPA) of all 256 episodes (Table 13). Steps count the 121 controlled steps after seven random admission actions.

context and 16 forecast frames; the benchmark uses 2,048 windows for video metrics and the first 512 for position diagnostics. Surviving windows are a selected subset of wind trajectories, so PRED does not by itself establish death prediction or closed-loop control.

Verdicts of Test 1. The colors of Table 1 were fixed before the experiments: **green** when real encodings locate the cube within one cube width and predictions come within two pixels of an ideal forecaster (wall: at least 80% intensity), **red** at chance (wall: below 20%), **amber** in between.

What the wind-world reference scores are. The two numbers printed against PRED in the wind worlds are computed exactly by a dynamic programme over the simulator's position-and-wind law from the last context frame under the recorded actions, on the same 512 windows the cells use. They are the scores of two ideal forecasters that know the true dynamics but not that the window was later selected for surviving all 24 frames: an exact conditional-mean forecast scores 7.90 pixels at step 8, an exact one-sample forecast of the same law 11.76. Neither is a lower bound on the metric. Because the realized position is that of a selected window, a forecast biased toward the safe interior scores better than either: the infimum over all point forecasts, the median of the selected law, is 7.44, and one sample of the selected law scores 10.60; the published median reference of 8.14 exceeds the mean reference for the same reason. A cell can therefore sit below its reference, as the flow arm's black-wind cell does at 11.1. The reference is assigned by what the arm's predictor is by construction, a deterministic map or a sampler, and this assignment decides one colour: the grid JEPA's black-wind cell reads 11.5 against 7.90 and is **amber**, the flow's 11.1 against 11.76 is **green**, a difference of 0.4 pixels in the measurement. Read on the one-parameter family "conditional mean plus a fraction ℓ of a sampled innovation", whose step-8 score rises from 7.90 at $\ell = 0$ through 8.76, 9.58, 10.60 and 11.17 at $\ell = 0.4, 0.6, 0.8, 0.9$ to 11.76 at $\ell = 1$, the same row reads: class token $\ell \approx 0.4$, grid JEPA $\ell \approx 0.95$, flow $\ell \approx 0.9$; the grid JEPA's predicted latents read out at nearly full dispersion. Both probes are nonlinear, so a probe applied to a predicted latent does not commute with the expectation inside it, and no sentence in the paper reads a model's predictor off its probe

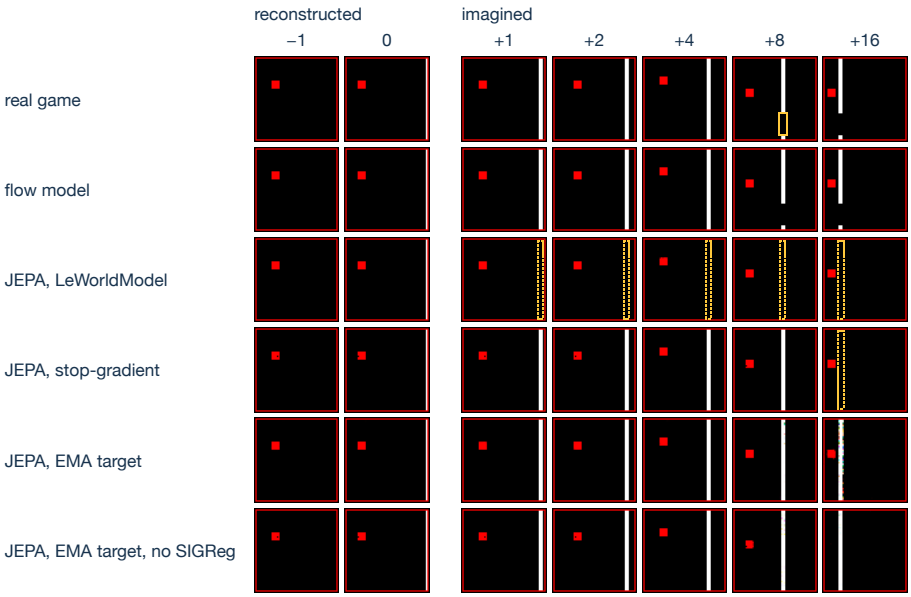


Figure 10: **Every prediction target except LeWorldModel's joint one keeps the approaching wall.** Black gate, laid out as Figure 8: the last two context frames reconstructed by each model, then its own predictions 1 to 16 steps ahead. The opening is revealed at step 7; solid yellow outline: the real opening. Dashed yellow outline: where the real wall is, drawn when a model's frame keeps less than 20% of it. LeWorldModel's encoder still reconstructs the wall in the context frames, but at a weight too small for its predictor to keep (Figure 2b). First eligible held-out window, chosen without looking at model outputs; every model seed 0.

Table 15: **Detaching the target does not rescue the wind cell and costs the deterministic one.** Position read-out error (mean absolute pixel error per coordinate), mean ± sample standard deviation over three model seeds; each seed pair shares initialization and data order, and every fit enters with no seed or checkpoint selection.

Per-frame gray noise	Joint target	Detached target
Deterministic	0.37 ± 0.05	10.93 ± 4.90
Wind	14.51 ± 0.12	14.51 ± 0.07

score. The "better of two probes" rule biases PRED downward, toward **green**; it was registered to protect a cell from a weak decoder, and the per-probe values are in Table 11.

Detached-target refits. For the two per-frame gray conditions, three model seeds per world are refitted with the prediction target detached from the encoder's gradient (`stop_gradient` on z_{t+1} in Equation 1); the predictor-input and SIGReg gradients, architecture, data order and initialization are unchanged, and the joint control of each seed is retrained by the same script so that the pair differs only in the target gradient (a bit-exact equivalence test against the original trainer is part of the test suite). A GELU read-out with two hidden layers of width 256 on the raw 512 latent coordinates is fitted on 50,000 training encodings per model and scored on 10,000 frames of an independently collected archive as mean absolute pixel error per coordinate (Table 15); this protocol differs from the probe of Table 1, so the two tables are not numerically comparable. Detaching the target removes the deletion force of Section 2.1 but not the white-noise fixed point, which only moves to $v^\star = 0.984$: the three windy detached fits and two of the three still ones sit there (Appendix C), so the wind world stays at chance and the still sibling gets worse.

Amplitude intervention. For the black still world, the same latent tile probe is read on three inputs over the first 128 metric windows of the cell evaluation, for the seed-0 fit of each arm: the

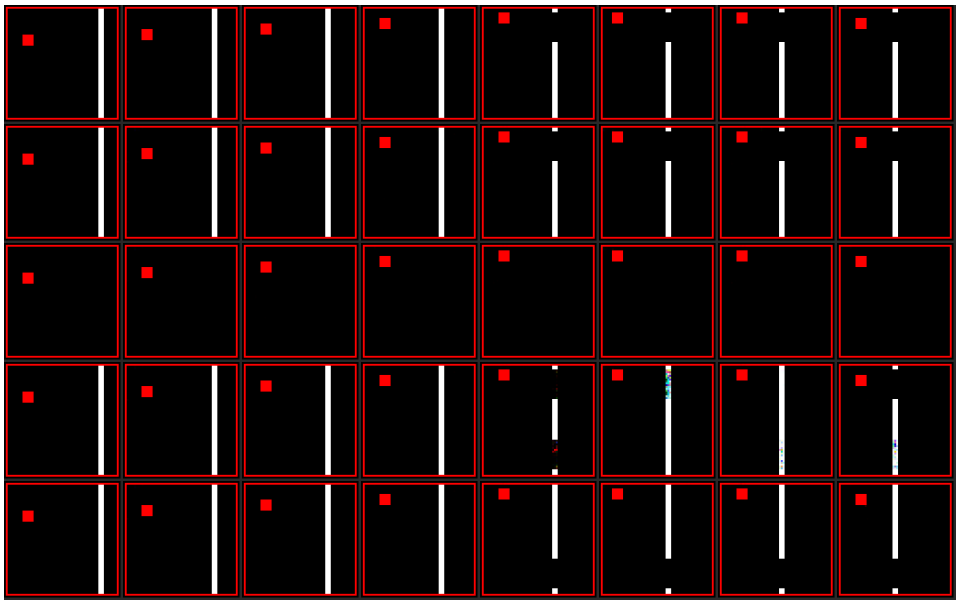


Figure 11: **Averaging over the opening's side keeps a wall in view; LeWorldModel's trained predictor does not.** A wall that has not yet arrived is a different case, erased by the average (Section 4). Black gate, JEPA model seed 0, one held-out context whose wall is still solid at $x \in \{51, 54\}$ and whose opening is revealed at the fourth or fifth predicted step; eight predicted steps under the recorded actions. Rows: the real future; the decoded encoding of the real future; the trained predictor's rollout; the decoded average of the frozen encodings of the two possible futures, the recorded opening and the other one, which is the exact conditional mean given the context before the reveal; the decoded encoding of the other-opening future. Faint artefacts at the opening in the fourth row after the reveal are the decoder rendering the average of two encodings. Quantities for every context and fit are in Section 4; the strip is the first context of its group, chosen without looking at model output.

true encodings of the recorded future, the model's own predicted latents, and the true encodings perturbed by isotropic Gaussian noise whose variance equals that arm's own rollout mean squared error at that step (3.5×10^{-4} of the variance for the class token, 2.2×10^{-4} for the grid). The perturbation therefore has exactly the size of each predictor's error but none of its structure; the 18.40 quoted in Section 4 is the cell's pixel probe over 512 windows and is not the same read-out. The class-token model's rollout error is 0.0004 of its latent variance at every step; its read-out gives 0.0005 pixels on clean latents, 39.4 on predicted ones and 46.5 under the noise. The grid model's rollout error grows from 0.0002 to 0.20 of its variance over eight steps; its read-out gives 0.09 pixels clean, 0.21 to 3.57 on predicted latents and 0.13 to 2.65 under the matched noise. Two readings follow: the read-out's sensitivity to an unstructured perturbation of the predictor's own magnitude accounts for both arms' rollout errors, and at equal *relative* error the grid latent is two orders of magnitude more robust than the class token. Nothing is trained in this diagnostic and no seed or window is selected.

N HYPERPARAMETERS AND COMPUTE

The gate and gray-noise cube conditions supply 500,000 uniform-random transitions each. The earlier black and color-noise cube datasets contain 100,000 transitions. Craftax uses its separate 500,000-transition archive. Model training uses bfloat16 compute, float32 parameters, and one A100. A 500-step calibration estimates the training length fitting 575 seconds including compilation; the grid gate model trains for 8,500 steps (17,000 and 34,000 in the earlier budget extensions of Appendix X), the class-token cube models for 23,750–24,000. Model seeds 0, 1, 2 of the grid JEPA differ only in the initialisation and data-order seed.

JEPA settings. Both JEPA architectures use batch 128 and AdamW with learning rate and weight decay 10^{-3}, 10% warm-up followed by cosine decay to zero, and gradient clipping at 1.0. SIGReg uses weight 0.09, 17 knots on $[0, 3]$, and 1024 random projections. Neither architecture uses a stop-gradient, teacher, decoder, or reward during world-model fitting. Linear–BatchNorm–GELU–linear projection heads flatten batch, time, and spatial tokens for normalization. Training-mode statistics therefore couple examples and temporal positions; at inference, running statistics define a fixed pointwise projection and the attention predictor is causal. The theoretical causal-map formulation is an inference/population idealization.

	Class-token JEPA	Grid JEPA
encoder	ViT, 7×7 patches	tile transformer + conv
width / depth / heads	192 / 6 / 3	128 / 6 / 8
encoder MLP	768	$4\times$ width
read-out	class token	4×4 grid, 32 channels
latent dimension	192	512
projector	BN projector, width 2048	same, hidden 512
predictor	AdaLN-zero AR	spatial + causal temporal
predictor depth / heads / dim	6 / 16 / 64	6 / 8 / 32
predictor MLP / width	2048 / latent	$2\times$ width / 256
history	3 frames	8 frames
dropout	0.1	0
parameters	15.25M	11.12M

Flow settings. The tokenizer uses 9,000 steps, batch 256, and learning rate 9×10^{-4}; dynamics 16,000 steps, batch 64, the same learning rate, width 256, six layers, eight spatial and temporal attention heads, a 32-frame context, independent per-frame noise levels, a shortcut loss of weight 0.25 every fourth step after the first quarter of training, history dropout 0.25, two history frames resampled every eight steps after half of training, and a temporal loss weight of 0.25. The symlog reward head has 255 bins over $[-20, 20]$; reward and termination losses have weight 0.1 and enter the trunk at full strength for half of training, then at factor 0.1. Inference uses four function evaluations per frame. We corrected terminal-transition masking in the recipe's reward loss (Appendix V).

JEPA read-outs. Gate post-hoc heads use the frozen encoder and the first 800 time rows (80,000 frames) of the same archive used for world-model training. The following 200 time rows are held out from head fitting only; the independent world-model evaluation archive is `eval.npz`. The pixel decoder uses 8,000 steps, batch 256, learning rate 10^{-3}, width 256, six residual blocks; it supports the figures and the decoded-latent control, in which its frozen output, clipped to $[0, 255]$, replaces the pixels of the real-gate PPO; no true pixel enters that policy after the encoder. Reward, termination, and behaviour-value heads are two-layer perceptrons of width 512 on the latent and the one-hot action, 6,000 steps, batch 1024, learning rate 10^{-3}, Monte Carlo value targets at discount 0.99, value loss weight 0.1. The residual predictor models $z_{t+1} - m$ with $m = g(z_{t-7:t}, a_{t-7:t})$ the original grid prediction, conditioned on m, z_t, and the action: three residual blocks of width 1024, 8,000 steps, and batch 2048. The final 20,000 valid windows of the same world-model training archive are excluded from residual fitting and used for its quality check. They are not independent world-model evaluation data. For encoder seed 0, coordinate-mean MSE is 3.07×10^{-4} for the original predictor, 2.05×10^{-4} for one residual sample, and 1.32×10^{-4} for the mean of eight. This

quality check uses eight function evaluations per sample; the reported PPO runs instead use four. Averaging eight learned samples is a random Monte Carlo average, not an exact conditional mean.

Policy optimisation (historical protocols). The settings in this paragraph are those of the earlier gate and cube studies of Appendix X; the shared-policy comparison of the main text uses Appendix P and the Craftax comparison Appendix W. PPO uses discount 0.99, GAE parameter 0.9, clipping 0.2, value coefficient 0.5, entropy coefficient 0.01, AdamW at 2.5×10^{-4} with weight decay 0.01, four epochs per batch, and gradient clipping at 1.0. Pixels pass through a three-block Impala torso (Espeholt et al., 2018), latents through a two-layer perceptron; a GRU receives that embedding and the previous action; the critic uses 64 categorical bins with histogram-Gaussian targets (Farebrother et al., 2024). Real-environment PPO uses 128 environments and 64 steps per iteration with four minibatch groups, truncated backpropagation over 16 steps, and 1.5M environment steps. In-model PPO uses 150 updates of 512 trajectories of 16 steps (1.2M imagined steps) with minibatch size 128; half of the starts are resets, half dataset states with their real eight-frame context. All reported policy rows use final checkpoints.

Compute. On a shared A100 the seed-0 grid-gate JEPA completes its $8{,}500$ updates in 573 seconds against the 575-second budget, with a logged total of 586.5 seconds including compilation. The flow model takes 732 seconds because of its separate tokenizer stage. Every read-out of a frozen model is off budget, and each such stage — probe, head and decoder fitting, the residual sampler, one policy run, the latent diagnostic — costs minutes rather than hours; the sample-averaging rollouts of Appendix X are the most expensive, tens of minutes per run. Per-stage wall-clock is logged beside every run.

O EVALUATION PROTOCOL

This section states the earlier gate protocol of Appendix X; the main text's policy comparison follows Appendix P instead, and the Craftax one Appendix W. Every gate-policy endpoint uses the final checkpoint and 256 real episodes from reset, with the 128-step cap. Survival is the fraction still alive after step 128; death on that step is not survival. Walls passed counts wall exits before or on the terminal transition, including an exit on the same step as death. Discovery is final survival strictly above 50%. Real-training and imagined-training runs advance their random keys under different iteration schedules; their final evaluations are not paired episode trials. Policy seeds are separate runs conditional on a fitted model; independently fitted encoders provide a different replication level. That protocol has a single flow checkpoint, against three independent fits per recipe in the campaign of Table 2. Reported means weight policy runs equally; standard deviations describe variation across those runs, not uncertainty across model families.

Gate specification. The gate uses the same image and object sizes and cardinal action displacement. A white vertical wall of width three pixels spawns at $x = 60$ every 22 steps and moves left three pixels per step. Its 16-pixel gap is sampled at spawn, top or bottom with equal probability, and becomes visible when the wall reaches $x \leq 40$. The gap location is hidden state before that reveal. Border or wall contact produces reward -1 and termination; the time cap is 128 steps.

Latent diagnostic. For each encoder, the diagnostic uses 64 real contexts with the wall still solid at $x \in \{42, 45\}$. Reconstructed simulator states generate recorded futures and matched wall-free futures under the recorded actions. The recorded branch survives the diagnostic horizon by construction. At each step, wall amplitude is the squared Euclidean distance between encodings of the wall-present and wall-absent frames. The teacher-forced residual uses real encoded history and is a total squared error, summed over latent coordinates. The wall coordinate of a prediction is its projection onto the wall-present minus wall-absent latent difference, with endpoints 0 and 1. Decoded wall intensity averages the minimum colour channel on shared wall pixels, excluding either possible gap band. These measurements compare features within one encoder; raw distances across models are not an information metric.

P SHARED-POLICY COMPARISON: PROTOCOL

This is the protocol behind Section 5 and the behavioral rows of Section 4. It was fixed before any of its policy outcomes existed (`research/shared_decoded_cnn_protocol.md` and, for the two worlds added afterwards, `research/scorecard_protocol.md` in the repository).

Matrix. Six worlds: per-frame gray noise without wind, per-frame gray noise with wind, persistent gray noise with wind, the gate, and the gate with a per-frame or a persistent gray background. For each world, the grid JEPA and the flow model each train three policy seeds $(0, 1, 2)$ inside imagination; all three gate worlds additionally receive three decoded-real control policies per family. That is 54 policies, all of which finished their registered budget and are reported; two of them, policy seeds 1 and 2 of the flow model's imagination in the gate world with a persistent background, were stopped once by an infrastructure failure and rerun from scratch under the same protocol. The three worlds without a gate received the same decoded-real controls afterwards (three policy seeds per family on the model fit of their imagined rows, 18 more policies, same budget and test episodes). Their recorded episodes carry no wall, so the simulator state the control resets to has the wall fields set to "no wall" as the simulator itself does; the per-episode background of the persistent world is rebuilt by replaying the recorder, which reproduces every recorded frame exactly.

In the four worlds of the original campaign, the three policy seeds run on one model fit each: gray-world JEPA checkpoints are model seed 0, the black gate's JEPA is model seed 1 (preselected on prior development evidence that its decoded-real CNN could discover the strategy), and every flow checkpoint is the seed-0 fit. In the two gate worlds added for this version, the design is stronger: policy seed k runs on model fit k, so the three runs of each cell are three independent world-model fits as well as three policy initializations. Policy seeds alone are not independent world-model fits, and the table states which regime each row uses.

Policy and optimizer. The policy is the unmodified pixel actor-critic of the original study: a three-block Impala CNN on native 63×63 RGB, a GRU that also receives the previous action, a categorical actor and a 64-bin histogram critic; 1,702,980 parameters. Both families use the same recurrent PPO: AdamW at 2.5×10^{-4} with weight decay 0.01, four epochs, minibatch 128, 16-step truncated backpropagation, $\gamma = 0.99$, GAE $\lambda = 0.9$, value coefficient 0.5, entropy coefficient 0.01, clipping 0.2, global gradient clipping 1.0, batch 512 and horizon 48. The three non-gate worlds use 50 updates (1,228,800 positions, 2,400 optimizer steps); every gate policy, imagined or decoded-real, uses 150 updates (3,686,400 positions, 7,200 optimizer steps). The training script refuses any other budget unless run as an explicitly non-scientific two-update smoke.

Observations. Each policy sees only its own model's decoded RGB: decoded posterior encodings of real frames during burn-in and evaluation, decoded model predictions in imagination. No raw pixel reaches the actor after the encoder. Training starts are sampled uniformly from valid eight-frame archive histories with no reset in the history; burn-in processes the first seven decoded frames with their real incoming actions. Latents keep the native $4 \times 4 \times 32$ interface without resizing, whitening or amplification; decoder quantization is retained.

Task signals. Flow uses its native sampled dynamics at four function evaluations and its jointly fitted reward and termination heads. The JEPA uses its deterministic predictor, the existing 80,000-frame decoder, and reward and termination heads of the original architecture refitted on all 500,000 training transition labels with the original 6,000-update recipe. The refit never updates encoder, predictor or decoder. Equal label counts do not equalize supervision stage: the flow heads send full-strength gradients into the dynamics trunk for the first half of its training and 0.1-scaled gradients afterwards, while JEPA world-model fitting is reward-free.

Evaluation. 256 fresh reset episodes are admitted by surviving seven uniformly random actions; the admission never looks at the policy. The same accepted prefix identities and all environment and action keys are reused across families and policy seeds, so every contrast is paired. The actor then controls 121 steps under the 128-step cap. Survival is return above -0.5; walls passed counts wall exits before or on the terminal transition. Every policy is its final checkpoint. Native-RGB trajectories of the first three accepted episodes are stored for each policy regardless of outcome.

Verdicts of Test 2. The colors of Table 2 were fixed before the runs: **green** for at least 80% mean survival and 50% in every run, **red** when the agent beats the better blind strategy by at most five points, **amber** in between. In the gate world with per-frame noise, a rerun of the same configuration moved survival from 5% to 94% (Appendix F), so differences inside that spread are read as ties.

Validation before the runs. Independent CPU checks of the training and evaluation code; GPU codec parity between the imagination path and the evaluation path for every checkpoint (context versus single-step encoding, 256- and 512-frame batches, head-cache encoding); unchanged frozen-state hashes; exact gate-state reconstruction; shared initial-context and evaluation-packet fingerprints. Two-update smokes per family and path supplied wall-clock estimates used only for scheduling.

For the two gate worlds added in this version, imagination needs the simulator state behind each archive context, and the archives store position, wind, wall position and gap side but not the per-episode background texture. A sidecar replays the collector from the recorded seed and emits the texture key, and is accepted only when the replayed frames, actions, positions, wall positions and gap sides equal the archive's exactly. The reconstructed state is then checked against the archive frame by frame on $1{,}024$ sampled transitions per world before any policy runs: pixel-exact in the black and persistent-background worlds, and structurally exact on the drawn pixels where the background is redrawn each frame. All three gate worlds passed on every sampled transition.

State-blind reference rows. Two rules that never read an observation are run through the same paired evaluator on the same packets: uniform random actions, and strict left–right alternation, which is the trivial survival strategy of the still worlds (Table 16). They calibrate the survival numbers of Table 2. Alternation survives the per-frame still world in every episode; both rules survive no episode of any wind world or of any gate world. A trained policy whose control length falls below these values is steering into the boundary faster than chance. Because a state-blind rule ignores the observation and the three gate worlds share reset and action keys, their blind rows are identical by construction; only the background differs between them.

Table 16: State-blind rules on the shared evaluation packets: survival, control length and, for the gate worlds, walls passed, over the same 256 paired episodes as Table 2. Both rules are run through the identical evaluator on the identical episode keys; no episode or seed selection applies.

world	rule	survival	length	walls
per-frame noise, still	uniform random	0.24	74.0	–
per-frame noise, still	left–right alternation	1.00	121.0	–
per-frame noise, wind	uniform random	0.00	17.4	–
per-frame noise, wind	left–right alternation	0.00	18.2	–
persistent noise, wind	uniform random	0.00	17.4	–
persistent noise, wind	left–right alternation	0.00	18.2	–
gate	uniform random	0.00	24.0	0.07
gate	left–right alternation	0.00	24.6	0.05
gate + per-frame noise	uniform random	0.00	24.0	0.07
gate + per-frame noise	left–right alternation	0.00	24.6	0.05
gate + persistent noise	uniform random	0.00	24.0	0.07
gate + persistent noise	left–right alternation	0.00	24.6	0.05

Imagined termination probe. To separate what the policy learns from what its training environment shows it, each frozen model is rolled forward for 48 steps from 256 uniformly sampled real eight-frame contexts under the recorded actions (the data policy is uniform random), and its own termination head is read on two inputs: the model's predicted latent, exactly as in imagination, and the encoder latent of the real frame at the same step. The recorded termination labels give the ground truth. Nothing is trained and the adapters are byte-identical to those of the campaign. Figure 12 plots the implied probability that an imagined episode is still alive after k steps against the recorded survival. Per-step outputs, including latent distances between predicted and encoder latents, are stored with the campaign results.

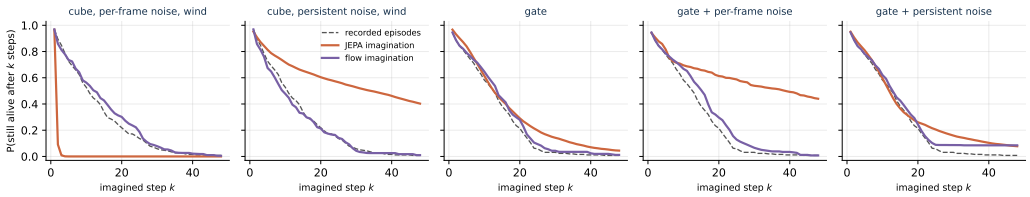


Figure 12: **What each model's imagination tells an agent about dying.** Each frozen model is rolled forward from 256 real starting points under recorded random actions, and its own termination head is read along the rollout. Solid curves show the implied probability that the imagined episode is still alive after k steps; the dashed curve shows how many of the same recorded episodes actually were. One model of each kind, those of Table 2. In the cube world with per-frame noise and wind, the JEPA's head predicts termination with probability 0.91 at every step, so its imagined episodes end at once; elsewhere it is optimistic, most of all in the gate world with per-frame noise, where after 48 steps it expects 44% of episodes to be alive and fewer than 1% are.

Q DECODER-FREE INTERFACES ON THE NOISY GATE WORLDS

Section 5 reads both models through a decoder. The two interfaces here read the frozen latent directly, so that no decoder stands between a model and the policy or planner. Both use the same three model fits per world as Table 2, policy seed k on fit k, and every number is a final update; no seed, checkpoint or episode is selected.

Latent-observation PPO. The policy replaces the pixel CNN by a two-layer perceptron on the 512-dimensional latent, keeps the GRU, the previous-action input, the categorical actor and the histogram critic, and is trained by the same recurrent PPO with the same settings for 150 updates of 512 trajectories of 16 steps inside each frozen model, half of the starts from resets and half from recorded eight-frame contexts, as in the historical protocol of Appendix N. The JEPA supplies its deterministic predictor and refitted heads; the flow model its sampled dynamics at four function evaluations and its jointly fitted heads. Evaluation is 256 fresh episodes from reset under the 128-step cap with lengths counted from the reset and no admission prefix, so the lengths are not those of Table 2; the two state-blind rules are run through the same evaluator on the same keys and printed in the table. Real-transition controls through the same latent interface, one policy per fit, are listed where they completed.

Planners on the JEPA. Three planners run on the frozen JEPA in the closed-loop evaluator of the historical gate study, 128 environments for 128 steps from a uniformly random eight-step warm-up, lengths counted from the reset, the cap not counted as a death. *Goal-latent search* is the LeWorld-Model planner: every five-action sequence is rolled through the predictor and the one whose final latent is closest to the encoding of a goal frame, the cube at the frame's centre with no wall, is executed one step. *Head-scored search* rolls the same sequences and scores them by the refitted reward and termination heads with a bootstrapped value at the horizon; *one step* does the same at horizon one. Random and a fixed centre-holding rule are run through the same evaluator.

Reading. The latent interface does not qualify. Trained on *real* transitions through the frozen latent, the perceptron policy survives at most 0.03 of episodes through the JEPA's latent and at most 0.02 through the flow's, in either world, where the same PPO through the decoders reaches 0.49 to 0.98 (Table 13); on the black gate, in the earlier single-seed protocol, the same interface reached 0.98 inside the flow model, so it is the noisy backgrounds that defeat it at this budget. The twelve imagined-latent policies all end at 0.000 survival with 0.71 to 0.99 of their imagined rollouts alive, but with the control failing they say nothing about imagination that the decoded rows do not already say, and the paper's interface claims rest on the decoder alone. What the runs do establish is negative: the flow model's decoded-interface success in the per-frame gate world is not reproduced through its latent by this policy, so the repair Section 5 credits to sampling is a repair *through the decoder*. Among the planners, the head-scored ones pass walls, up to 0.81 per episode in one fit, and extend control by ten steps on average under a per-frame background, which shows that the refitted heads carry the wall's position; the goal-latent planner, which reads only the predictor, is at or below

Table 17: **Decoder-free interfaces on the two noisy gate worlds.** Survival to the cap, mean episode length from reset and walls passed per episode, per seed (0 / 1 / 2), policy seed k on model fit k; planners are deterministic given the fit and the evaluator's keys. PPO rows use the 256-episode evaluator of `studies/stoch/rl.py`, planner rows the 128-environment evaluator of `studies/stoch/control.py`; the two blind rows are from the PPO evaluator, and the random-action row among the planners is from the planner evaluator.

World	Interface / training	Survival	Length	Walls
gate + per-frame noise	blind, uniform random	0.00	30.4	0.06
gate + per-frame noise	blind, left–right alternation	0.00	32.4	0.02
gate + per-frame noise	real transitions, latent, JEPA	0.031 / 0.000 / 0.000	59.5 / 39.2 / 39.0	0.91 / 0.19 / 0.06
gate + per-frame noise	real transitions, latent, flow	0.008 / 0.000 / 0.016	46.4 / 33.4 / 49.1	0.61 / 0.25 / 0.80
gate + per-frame noise	imagined, latent, JEPA	0.000 / 0.000 / 0.000	40.6 / 18.7 / 29.4	0.03 / 0.00 / 0.00
gate + per-frame noise	imagined, latent, flow	0.000 / 0.000 / 0.000	24.5 / 21.9 / 33.3	0.09 / 0.00 / 0.03
gate + per-frame noise	planner on the JEPA, uniform random	0.000 / 0.000 / 0.000	32.2 / 29.2 / 30.0	0.09 / 0.02 / 0.06
gate + per-frame noise	planner on the JEPA, goal-latent search, horizon 5	0.000 / 0.000 / 0.000	28.0 / 19.5 / 28.2	0.02 / 0.00 / 0.02
gate + per-frame noise	planner on the JEPA, head-scored search, horizon 5	0.016 / 0.000 / 0.000	48.7 / 35.3 / 37.1	0.55 / 0.06 / 0.06
gate + per-frame noise	planner on the JEPA, head-scored one step	0.008 / 0.000 / 0.000	52.9 / 33.8 / 33.5	0.81 / 0.03 / 0.05
gate + persistent noise	blind, uniform random	0.00	30.4	0.06
gate + persistent noise	blind, left–right alternation	0.00	32.4	0.02
gate + persistent noise	real transitions, latent, JEPA	0.000 / 0.000 / 0.000	33.0 / 34.6 / 32.5	0.04 / 0.06 / 0.07
gate + persistent noise	real transitions, latent, flow	0.016 / 0.004 / 0.000	43.3 / 39.4 / 35.6	0.54 / 0.48 / 0.29
gate + persistent noise	imagined, latent, JEPA	0.000 / 0.000 / 0.000	9.4 / 11.0 / 12.1	0.00 / 0.00 / 0.00
gate + persistent noise	imagined, latent, flow	0.000 / 0.000 / 0.000	25.0 / 30.7 / 31.1	0.07 / 0.05 / 0.07
gate + persistent noise	planner on the JEPA, uniform random	0.000 / 0.000 / 0.000	32.2 / 29.2 / 30.0	0.09 / 0.02 / 0.06
gate + persistent noise	planner on the JEPA, goal-latent search, horizon 5	0.000 / 0.000 / 0.000	30.5 / 29.6 / 30.3	0.02 / 0.00 / 0.01
gate + persistent noise	planner on the JEPA, head-scored search, horizon 5	0.000 / 0.000 / 0.000	26.7 / 29.6 / 26.5	0.02 / 0.11 / 0.02
gate + persistent noise	planner on the JEPA, head-scored one step	0.000 / 0.000 / 0.000	23.5 / 25.8 / 23.6	0.02 / 0.04 / 0.01

random in both worlds, as it was on the black gate (32.0 steps against 32.2, historical protocol). A policy or planner that reads the grid latent with a convolutional torso, or an amplified wall direction, was not run.

R THE REVERSAL ACROSS REGULARIZER WEIGHTS AND BUDGETS

The reversal of Section 4 is measured at one regularizer weight, $\lambda = 0.09$, and one budget, 8,500 updates. Eight further grid-JEPA fits, all seed 0, vary one of the two: the SIGReg weight at 0.03, 0.27 and 0.9 in both wind worlds, and the budget at 17,000 and 25,500 updates, two and three times the registered one, in the per-frame wind world. Everything else is the campaign recipe, and each fit is scored by the same cell evaluation as Table 1, both probes, the same 512 windows.

Table 18: **Per-frame noise plus wind against persistent noise plus wind, one fit each, as the regularizer weight and the budget change.** ENC and PRED as in Table 1, the better of the two probes; the registered cell's seed-0 values are repeated for reference. Every fit that was run appears.

World	Fit	ENC	PRED	pred. loss	SIGReg
per-frame noise, wind	$\lambda = 0.09$, 8,500 updates (registered)	17.54	19.70	0.7345	2.29
per-frame noise, wind	$\lambda = 0.03$	0.14	9.13	0.0343	2.16
per-frame noise, wind	$\lambda = 0.27$	17.71	19.11	0.8821	1.24
per-frame noise, wind	$\lambda = 0.9$	17.78	20.13	0.9504	1.05
per-frame noise, wind	17,000 updates	17.56	20.77	0.7355	2.28
per-frame noise, wind	25,500 updates	17.62	23.11	0.7343	2.26
persistent noise, wind	$\lambda = 0.09$, 8,500 updates (registered)	0.72	11.55	0.0031	1.07
persistent noise, wind	$\lambda = 0.03$	0.42	10.15	0.0013	1.16
persistent noise, wind	$\lambda = 0.27$	8.70	16.20	0.0099	0.96
persistent noise, wind	$\lambda = 0.9$	18.14	22.02	0.0180	1.04

Budget. Two and three times the registered budget change nothing in the failing cell: ENC 17.56 and 17.62 pixels, prediction loss 0.7355 and 0.7343 against 0.7345 at 8,500 updates, the same latent to three digits. The fit is in a stable basin, not on its way somewhere else.

Regularizer weight. The weight moves the odds. At $\lambda = 0.03$ the per-frame wind world keeps the cube in seed 0, ENC 0.14 pixels with prediction loss 0.034, but not in two further seeds, run after the rest of this study (ENC 17.7 pixels in both, loss 0.48). In seed 0 it is once more the *easier* background, as in the still worlds, with the persistent sibling at 0.42. At 0.09 the per-frame cell fails and the persistent one keeps the cube, which is the reversal of Table 1. At 0.27 the persistent cell degrades to 8.70 pixels, and at 0.9 both are at chance. The two backgrounds fail at different weights and with different loss signatures: every failing per-frame fit has a prediction loss between 0.73 and 0.95 per coordinate, a latent driven by the unpredictable noise, while the failing persistent fit at $\lambda = 0.9$ has a loss of 0.018, a nearly predictable latent that carries the episode's texture instead of the cube, which is the cheap non-collapsed optimum of Proposition 2 in all but the exactly-Gaussian clause.

What the objective prefers. The final training values of the two loss terms let each fit be scored under the other's weight. Under the registered $\lambda = 0.09$ the failing seed-0 fit scores $0.734 + 0.09 \times 2.29 = 0.94$, while the cube-keeping $\lambda = 0.03$ fit scores $0.034 + 0.09 \times 2.16 = 0.23$ under the same weight; under $\lambda = 0.27$ the comparison is 1.22 against 0.62; only at $\lambda = 0.9$ does the failing fit score better, 1.90 against 1.98, a gap inside the batch noise of the SIGReg statistic. So at the registered weight, and at three times it, the objective prefers a latent that keeps the cube by a factor of two to four, and every fit we trained at those weights, three seeds and three budgets, lands in the other basin. The failing cell is where optimization goes, not what the objective asks for, which is the part of the phenomenon Section A declines to characterize. One fit per variant is a check of direction, not a spread; the registered cell's three fits differ by 0.08 pixels.

S PREDICTOR SWAP

The swap of Section 4 keeps every part of the flow recipe except its tokenizer. The frozen grid-JEPA encoder of a gate fit, the tokenizer's encoder trunk trained with the JEPA objective instead of reconstruction (it differs only in reading one frame instead of two and in ending with LeWorldModel's projector), replaces the tokenizer's encoder, its 512-dimensional latent read as the $4 \times 4 \times 32$ grid the flow predictor expects, and the JEPA's own pixel decoder, fitted after training, replaces the tokenizer's decoder. The flow predictor is then trained exactly as in the flow recipe: the same 16,000 updates, per-coordinate latent standardization computed on the training archive, and reward and termination heads trained jointly. No tokenizer is trained, so the swap costs about 400 seconds of training. Both the swap and the JEPA's own predictor are scored by the gate evaluation of Table 12, run by the same script on the same contexts: the decoded shared-wall intensity on 256 contexts before the opening is revealed, and the opening-side AUROC averaged over the eight predicted steps after it. The script is `studies/scorecard/swap.py`.

Table 19: **Predictor swap on the frozen grid-JEPA encoder.** Wall intensity at predicted steps 1 and 8 (encoded = real frames) and the mean opening-side AUROC over the eight predicted steps, for the JEPA's own predictor and for the flow predictor trained on the same frozen encoder. One row per encoder seed; every fit that was run appears.

			JEPA predictor		flow predictor (swap)	
world	seed	encoded	wall 1 / 8	AUROC	wall 1 / 8	AUROC
black gate	0	1.00	0.00 / 0.00	0.50	0.00 / 0.00	0.50
black gate	1	0.99	0.00 / 0.01	0.50	0.00 / 0.00	0.50
black gate	2	0.99	0.53 / 0.00	0.56	0.50 / 0.01	0.51
gate + per-frame noise	0	1.00	0.94 / 0.14	0.64	1.00 / 0.99	0.99
gate + per-frame noise	1	1.00	1.00 / 1.00	0.95	1.00 / 0.99	1.00
gate + per-frame noise	2	1.00	0.78 / 0.50	0.50	0.95 / 0.57	0.53

The reverse swap. To separate the encoder from the predictor, we also trained deterministic predictors on the frozen reconstruction tokenizer: the grid JEPA's factored predictor with its squared loss (A), and the flow recipe's own network with a squared loss instead of the flow loss (B). Table 20 scores them on the same contexts, with 93 contexts before and 90 after the reveal. In the black gate both keep the wall and the opening, as the flow predictor does. On LeWorldModel's encoder every

predictor loses the wall within one step, including the flow predictor and the squared-loss predictor trained on latents whitened over all 512 coordinates, so the loss is not a scale problem: the decoder draws the wall from the exact encoding (1.00), but the encoding changes almost nothing when the wall is removed (Appendix D).

Table 20: **Every predictor keeps the black-gate wall on the tokenizer and loses it on LeWorldModel's encoder.** Wall intensity at predicted steps 1 and 8 and opening-side AUROC at step 8.

encoder	predictor (model seeds)	wall 1	wall 8	AUROC 8
tokenizer	A, squared loss (0, 1)	1.00, 1.00	0.90, 0.97	1.00, 1.00
tokenizer	B, squared loss (0, 1)	1.00, 1.00	0.98, 0.98	1.00, 1.00
tokenizer	flow sampler (0, 1)	1.00, 1.00	0.94, 0.96	1.00, 1.00
LeWorldModel	flow, whitened (0, 1, 2)	0.00, 0.00, 0.74	0.00, 0.00, 0.00	0.50, 0.50, 0.57
LeWorldModel	A, whitened (0)	0.03	0.00	0.50

T POLICY BUDGET: EVERY IMAGINED CELL AT THREE TIMES THE REGISTERED UPDATES

Table 2 reports each world's registered budget, fixed before any policy ran: 150 updates in the gate worlds, 50 in the cube worlds. Two of the flow arm's cube cells had a wide seed spread at that budget, which is exactly the shape a policy-optimization artifact takes, so before reading anything into them we reran *every* imagined cube cell, both families and all three seeds, at 150 updates. Nothing else changed: same checkpoints, same head refits, same archive, same start protocol, same evaluation packet, same optimizer, same evaluator, and the extended budget is recorded in each run's provenance (`--budget extended`).

Table 21: **The same imagined cells at the registered and the extended policy budget.** Final-checkpoint survival in per cent on each world's fixed 256-episode packet, mean ± sample standard deviation over the three policy seeds, with the per-seed values beside it. Only the number of PPO updates differs between the columns; every run is reported and no seed or checkpoint is selected.

world	recipe	50 updates	150 updates	seeds (50)	seeds (150)
per-frame noise, wind	grid JEPA	0±0	0±0	0.000, 0.000, 0.000	0.000, 0.000, 0.000
per-frame noise, wind	flow	58±41	90±1	0.945, 0.129, 0.652	0.891, 0.918, 0.895
persistent noise, wind	grid JEPA	2±1	18±29	0.027, 0.020, 0.000	0.512, 0.000, 0.016
persistent noise, wind	flow	41±42	65±5	0.891, 0.207, 0.137	0.703, 0.602, 0.633

Three readings, in order of what they cost us.

First, the flow arm's weak cells were the budget. With a per-frame background its three seeds go from 0.945, 0.129, 0.652 to 0.891, 0.918, 0.895, and with a persistent background from 0.891, 0.207, 0.137 to 0.703, 0.602, 0.633: in both worlds the spread collapses and the low seeds were unfinished optimization, not a model that cannot teach. The persistent-noise cell is still below the colour threshold at the longer budget, mean 0.65, and it stays **amber**.

Second, the JEPA's per-frame-noise cell is not a budget artifact. It is 0.000 in all three seeds at 50 updates and 0.000 in all three at 150. That is the world where the encoder lost the cube (Section 4), and no amount of policy optimization inside such a model recovers it.

Third, the JEPA's persistent-noise cell does move: 0.027, 0.020, 0.000 becomes 0.512, 0.000, 0.016. One seed of three reaches half the episodes. We report this because it qualifies the sentence it bears on: in the world where the cube survives encoding, the JEPA's imagination is not uniformly unable to teach the task, it is unreliable, and one fit in three teaches a policy that survives half the time given three times the budget. The ordering against the flow reference is the same at either budget, mean 0.18 against 0.65 and worst seed 0.000 against 0.602.

What this changes about how to read the paper: the cube worlds' imagined cells are budget-sensitive in both families, so the matched rows of Table 2 should be read as what one fixed budget buys, not

as a ceiling. The gate worlds, which carry the paper's claims, are registered at the longer budget for both families and for both training sources, and their cells are not close to a threshold: the JEPA's imagination is at 0.00 to 0.03 and the flow's decoded-real controls and black-gate imagination are above 0.88.

U One repair attempt: a Muesli-inspired unroll auxiliary over the frozen models

One cell of Table 2 fails for neither of the paper's two reasons. In the gate world with a persistent background both families' representations keep the wall, both rollouts render it, the decoded-real controls pass on the same checkpoints, and yet both imaginations fail. In that cell the flow arm's policies *improve inside the dream* — the last update of the three runs has 0.25, 0.57 and 0.71 of imagined rollouts alive — and transfer nothing, 0.000, 0.039 and 0.113. That signature is model exploitation rather than lost state, so it is the one place where a change to the *algorithm* rather than to the model deserves a measurement, and we ran exactly one such change. (The flow cell was later traced to the sampler's context signal: at 0.8 the same models teach agents that survive 91% to 100% of episodes; Appendix G.)

What was added. The design note was written before the runs and fixes everything below. The world model stays frozen: no gradient from either new term reaches an encoder, predictor, decoder or head, and nothing is trained on imagined data except the policy and its critic. The policy keeps the campaign's CNN + GRU, the same parameter count and the same optimizer. Two terms are added to the campaign's PPO loss, both read off the frozen model's own lookahead, in the spirit of Muesli's unroll auxiliary and its regularized policy improvement (Hessel et al., 2021):

1. a K-step value target along the imagined on-policy trajectory, $y_t^{(K)} = \sum_{j<K} \gamma^j c_{t,j} \hat{r}_{t+j} + \gamma^K c_{t,K} V_{\bar{\theta}}(s_{t+K})$ with $c_{t,j}$ the model's own continuation product and $\bar{\theta}$ the rollout-time critic, regressed with the campaign's HL-Gauss loss at weight 0.5; and
2. a clipped advantage-weighted policy target. For every action a — including the ones the policy did not take — the frozen model is stepped once under a, continued $K-1$ steps under the policy's greedy action and bootstrapped, giving $\hat{q}_K(s_t, a)$; with $\hat{A} = \hat{q}_K - V_{\bar{\theta}}$ and β the standard deviation of $\hat{A}$ over the imagined step, the target is $\pi_{\mathrm{CMPO}}(a \mid s_t) \propto \pi_{\bar{\theta}}(a \mid s_t) \exp(\mathrm{clip}(\hat{A}/\beta, -1, 1))$ and $\mathrm{KL}(\pi_{\mathrm{CMPO}} \,\|\, \pi_\theta)$ enters at weight 0.5 on every eighth imagined row.

The clip is what makes this safe on a wrong model: each unnormalized weight lies in $[e^{-1}, e]$, so after normalization an advantage the model invents can move the target policy's likelihood ratio $\pi_{\mathrm{CMPO}}(a \mid s)/\pi_{\bar{\theta}}(a \mid s) = e^{\mathrm{clip}(\hat{A}_a/\beta, -1, 1)}/Z$ by at most a factor e^2 in either direction, whatever it claims, since the normalizer Z is itself in $[e^{-1}, e]$. $K = 3$ was chosen before the runs from $K \in \{2, 3, 4\}$, both families get the intervention in the same world with the same K and the same three seeds, and the matched rows of Table 2 are untouched.

That the rest is unchanged is measured, not asserted. The campaign's collector and the auxiliary collector were run on the same model, the same contexts and the same key, and every field the campaign's PPO reads was compared. Bit-identity is not available in this stack for a reason the campaign's own code records — bfloat16 kernels change with the batch shape, so adding one unrelated codec call to a program already moves the JEPA's decoded context by up to 9 levels of 255 — so the check measures that floor directly, by recompiling the campaign's collector and by putting it in a program that holds one extra codec call. At the first imagined step the flow collector deviates by exactly zero on every field, and the JEPA collector's deviations equal the extra-call floor to every printed digit, with both models' own reward and termination diagnostics inside one bfloat16 unit in the last place.

It does not work, and the way it fails is informative. Neither family improves. The grid JEPA stays at 0.000 in all three seeds, and the flow arm moves from 0.000, 0.039, 0.113 to 0.000, 0.016, 0.070 — slightly worse, well inside the spread of a cell whose three matched seeds already differ by

Table 22: **The gate + persistent-noise imagination cells with and without the unroll auxiliary.** Final-checkpoint survival in per cent on the world's fixed 256-episode packet, three policy seeds, policy seed k on model fit k, with the per-seed values; "imagined alive" is the mean fraction of the model's own rollouts still alive at the last update, which is what the policy was paid. Only the two auxiliary loss terms differ between a recipe's two rows. The verdict column applies the note's rule, fixed before the runs: improved if the three-seed mean gains at least 0.10 with no seed worse than its own matched counterpart by more than 0.05.

recipe	imagination	survival	per seed	imagined alive	verdict
grid JEPA	matched PPO	0±0	0.000, 0.000, 0.000	0.40	
grid JEPA	+ unroll aux (K=3)	0±0	0.000, 0.000, 0.000	0.47	unchanged
flow	matched PPO	5±6	0.000, 0.039, 0.113	0.51	
flow	+ unroll aux (K=3)	3±4	0.000, 0.016, 0.070	0.54	unchanged

a factor of three. By the rule fixed before the runs both rows read *unchanged*, so no sentence about the intervention enters the main text and $K = 2$ and $K = 4$ were not run: the note conditioned them on $K = 3$ moving a cell.

What did change is the wrong quantity. Both families end *more* successful inside the model they were trained in: the mean fraction of imagined rollouts still alive at the last update rises from 0.40 to 0.47 for the JEPA and from 0.51 to 0.54 for the flow model, while real survival does not follow. The added terms were live, not silently zero — the K-step value loss runs at 1.2–2.1, comparable to the PPO critic's own, and the policy term's mean KL per update is $2.2\text{–}2.6 \times 10^{-2}$ for the JEPA and 5.9×10^{-3}–2.8×10^{-2} for the flow model. A lookahead through a frozen model can only sharpen what that model already reports, and in this cell the report itself is what is wrong: the deficit is in what the imagination pays, so making the policy better at collecting it moves survival no closer. We also checked the cheapest alternative explanation, that the imagined policies simply collapse to a single action and no target could move them: only one matched seed and one auxiliary seed end below a policy entropy of 0.05, and the two matched seeds that end at 0.94 and 0.89 fail just as completely (`output/scorecard/aux_entropy.json`).

V TERMINAL REWARDS IN THE FLOW RECIPE

The original recipe masks transitions entering reset frames out of reward training. Here the only nonzero reward occurs on terminal transitions, so that mask removes the reward signal. The corrected head predicts the next reward from the current frame summary and action with terminal transitions retained in its loss. The reported flow-policy outcomes use this corrected convention. The single-checkpoint reference therefore denotes the complete tokenizer, dynamics, reward and termination package, rather than an isolated change in transition stochasticity.

W CRAFTAX INVENTORY COMPARISON: PROTOCOL

The protocol behind the Craftax paragraph of Section 5; fixed before the policies were trained (`research/craftax_native_inventory_protocol.md` and `research/craftax_inventory_protocol.md`). Table 23 lists every policy.

Environment. Craftax-Classic-Pixels version 1.5.0 with unchanged defaults: 17 actions, 64×64 procedurally generated maps, a 10,000-step native episode limit, 22 achievements. The native reward is the change in the achievement count plus 0.1 times the change in health; the evaluator checks the telescoping identity between summed rewards and terminal achievements and health on every episode. Termination occurs at the step limit, on lava, or at non-positive health. The evaluator uses the no-auto-reset environment, stops each episode at its first native termination and freezes its metrics.

World models. Both models are trained on the same 500,000-transition archive from 250 parallel streams, recorded by a frozen pretrained pixel policy that samples at temperature 1 on 20% of the

Table 23: **Craftax-Classic, one pixel policy trained for five million imagined steps inside each model,** 256 **native episodes per policy, final checkpoints.** Wood and table: fraction of episodes with the achievement; pickaxe: episodes ending with a wooden pickaxe. "Imagined r": mean reward per imagined step paid by the model's own head at the last update. All three policy seeds are listed; no seed is selected, and the state-blind rows use the identical evaluator on the same episode keys.

Policy trained in	seed	wood	table	pickaxe	achievements	return	imagined r
flow imagination	0	100%	96%	242/256	6.04	5.14	0.06
flow imagination	1	100%	91%	15/256	3.97	3.03	0.03
flow imagination	2	100%	97%	242/256	5.87	4.94	0.06
JEPA imagination	0	22%	0%	0/256	0.27	-0.65	0.90
JEPA imagination	1	4%	0%	0/256	0.07	-0.86	0.96
JEPA imagination	2	2%	0%	0/256	0.02	-0.91	1.00
uniform random	–	43%	9%	1/256	2.34	1.28	–
no-op	–	0%	0%	0/256	0.00	-0.92	–

transitions and at an inverse temperature drawn uniformly from $(0, 1)$ once per episode on the rest, under the same ten-minute budget with the same $4 \times 4 \times 32$ latent shape. The JEPA is the seed-0 grid fit of Table 1, with one-step factored predictor and eight-frame history; its pixel decoder and its reward and termination heads are refitted afterwards on the frozen latent with a fixed recipe (decoder: 8,000 steps, batch 256; heads: 6,000 steps, batch 1,024, hidden 512) whose arguments and resulting hashes are checked at load time. The flow model is the native recipe with jointly fitted heads, trained separately for this comparison under the same recipe and budget; it is not one of the three Craftax flow fits of Table 1, and the two checkpoints' hashes are recorded beside the runs. The frozen model weights of the JEPA are hash-verified to be byte-identical before and after head fitting.

Policy training in imagination. The policy is the original Craftax pixel actor-critic, a non-recurrent CNN of $6{,}092{,}177$ parameters with the original Craftax pixel-PPO settings; it is not the $1{,}702{,}980$-parameter CNN + GRU of the cube and gate worlds. It is trained for a nominal five million imagined steps (4,998,400 executed) inside each model from a shared bank of genuine eight-frame episode-start contexts. The JEPA adapter reads reward and termination from (z_t, a_t), predicts once per action and samples termination from the head's probability; the flow adapter runs its native expanding-window rollout with four function evaluations. Imagined episodes are capped at 1,024 steps. No behavior cloning, action masking, reward alteration, resource injection or checkpoint selection is used; a non-finite loss fails the run rather than being skipped. Three policy seeds per family, one model seed.

Evaluation. Each final policy is evaluated on 256 native episodes with evaluation seed 123. Environment keys derive from episode identity and step only, action-sampling keys from a separate stream; the construction is stored with the results. The policy observes native RGB and its previous action, whereas during training it observed its model's decoded RGB. Unlike the gate worlds, this comparison has no decoded-real control and no evaluation on decoded frames of the native game, so an interface mismatch between decoded and native pixels is not separated from an imagination failure by any run; the imagined-reward column and the in-model achievement logs are the quantities that do not depend on the evaluation channel. Building that matched evaluator was outside the scope of this revision and is listed as unavailable. Primary endpoint: the fraction of episodes in which the wooden-pickaxe achievement is true at the end of the episode. Retained: all 22 achievement rates with Wilson intervals, achievement count, native return, executed steps, survival, termination causes, first-acquisition steps, final inventory, and the terminal native score where a native termination occurred. Uncertainty intervals are across evaluation episodes for one fixed policy and do not estimate variation across model fits.

Reference rows. Two state-blind policies are run through the identical evaluator, keys and aggregation: uniform random actions and the no-op action. Uniform random play reaches one wooden pickaxe in 256 episodes, collects wood in 43% of episodes, places a table in 9% and averages 2.34 achievements per episode; the no-op policy earns no achievement and a return of -0.92 from health loss. A trained policy below the random row has learned something that hurts.

Imagined reward. The training logs record the mean reward per imagined step paid by each model's reward head. In the real game uniformly random play collects about 0.01 per step and the best policy of this campaign 0.03 (return over executed steps). Both readings are reported in Table 23; a policy whose imagined reward exceeds what any real policy collects is exploiting its reward head rather than the game.

X EARLIER UNMATCHED GATE EXPERIMENTS

The experiments in this appendix predate the matched shared-policy protocol of Appendix P: real and imagined policies did not share evaluation keys, latent-MLP and decoded-CNN policies are mixed, and the summary table (Table 24) is regenerated from the last evaluation of each run. They are reported because they cover interventions the matched campaign does not, in particular sampling from the JEPA's predictor. Of 27 policies trained inside the JEPA's imagination, none discovered the gate strategy, including nine that used a sampling head fitted on the predictor's residual and nine that averaged eight of its samples. Those runs share the fitted heads and the PPO protocol of that era, so they test this particular intervention rather than distributional prediction in general; the licensed statement in the main text is only that the deficit is in the rollout.

Reference model. For the gate, a reconstruction-trained tokenizer and causal flow-matching predictor provide a feasibility reference (Lipman et al., 2023; Chen et al., 2024; Hafner et al., 2025). It uses the same latent shape, data, and nominal model-training budget, with a 32-frame context and jointly fitted reward and termination heads. Architecture, supervision, and history length differ from the JEPA. One dynamics checkpoint is evaluated, so its policy runs show variation conditional on that fit rather than variation across fitted flow models.

X.1 USEFUL WALL INFORMATION REMAINS IN THE FROZEN REPRESENTATION

To test whether the wall information visible in decoded real encodings can support behavior, we train a convolutional policy in the real gate on the frozen decoder's output. Two of nine runs discover the survival strategy, both using encoder seed 1; the corresponding real-pixel CNN discovers it in two of three runs. This establishes useful information in at least one fitted encoder under the decoder-and-CNN read-out, and it is the reason the matched campaign qualifies every interface on real transitions before reading an imagination result. The decoder adds reconstruction supervision outside the world-model budget, so this is a test of recoverable information rather than unsupervised control sufficiency.

The policy interface matters. With an MLP torso, real pixels yield 0.91 walls per episode and frozen latents yield 0.94, with no discoveries in either group. An MLP on decoded latents similarly yields 0.92 walls. Thus failure of this MLP training recipe is not evidence that the encoder has lost the obstacle. These comparisons do not set a capacity limit for all MLP policies; they locate a limitation of the tested read-out and optimization procedure, which is why the matched campaign fixes one stock convolutional policy for every arm.

X.2 THE TESTED IMAGINATION PIPELINE DOES NOT DISCOVER THE STRATEGY

In the earlier unmatched protocols, all three JEPA predictor variants produce poor downstream control (Table 24). The original predictor yields 0.27 walls per episode; one residual sample and the mean of eight each yield 0.26. No policy among these 27 runs discovers the strategy; the highest final survival is 3.1%. Real-transition PPO on the same frozen encoders yields 0.94 walls. The difference measures the combined effect of learned transitions, fitted heads, and the changed PPO protocol. Even the stronger real-latent MLP result falls far short of reliable survival.

The flow reference supports feasibility at this task and nominal model budget: one of its nine policies discovers the strategy. The remaining eight pass 0.72–1.18 walls per episode. Since all nine use one dynamics fit with a different architecture and supervision, this reference does not establish a model-family ranking. Training the gate JEPA for twice and four times the steps also leaves imagined-policy survival at 0–3% in the tested extensions (encoder seed 0, policy seed 0, each predictor variant).

Table 24: **Final gate outcomes of the earlier unmatched campaign, with predictor variants kept separate.** Mean $\pm$ sample standard deviation across policy runs; discovery is final survival above 50%. JEPA imagination variants and real-latent conditions each contain 3 models $\times$ 3 policies; pixel controls have three policies. Every run enters at its last evaluation, with no selection over seeds or checkpoints. Real training changes rewards, terminations, and PPO protocol alongside dynamics. Flow is a single-model reference.

Training / observation	Predictor or torso	Walls	Discovery
Imagined JEPA	Original	0.27 ± 0.22	$0/9$
	Residual sample	0.26 ± 0.32	$0/9$
	Residual mean of 8	0.26 ± 0.30	$0/9$
Real / frozen JEPA	MLP	0.94 ± 0.04	$0/9$
Real / pixels	MLP	0.91 ± 0.08	$0/3$
Real / decoded JEPA	CNN	1.57 ± 1.26	$2/9$
Real / pixels	CNN	2.91 ± 1.65	$2/3$
Imagined flow	Sample	1.45 ± 1.23	$1/6$
	Mean of 8	0.88 ± 0.14	$0/3$

X.3 Prediction and fitted heads provide distinct failure signals

The same decoder that renders real encoded walls removes them from original predicted rollouts: across all three encoder seeds, decoded shared-wall intensity is at most 0.02 by the second predicted step. The shared wall pixels occur in both possible gap futures, so a pixelwise mixture of those futures would retain them. This observation rules out that simple pixel-space picture; it does not determine what averaging or distribution shift does in the learned latent space.

A within-encoder counterfactual diagnostic makes the scale of the feature explicit. On 64 pre-reveal contexts for seed 0, the first-step squared distance between wall-present and wall-absent encodings is 0.035, while total teacher-forced prediction error is 0.145; the same ordering holds for all three seeds. This comparison concerns sensitivity to wall removal versus error in all features. It neither identifies a causal mechanism nor permits comparing information content through raw distances across independently fitted latent spaces.

The fitted heads expose another limitation. For encoder 0, termination AUC is 0.96 for border deaths but 0.65 for wall deaths. Predicted rewards for the two death causes are -0.89 and -0.05, respectively, against the true -1. An imagined policy therefore trains on a weak wall-collision signal. The residual-sampling variants retain these heads and the policy protocol, so their poor control outcomes test this particular intervention rather than sampling in general.